มหาวิทยาลัยอัสสัมชัญ
ASSUMPTION UNIVERSITY
of THAILAND

VINCENT MARY
SCHOOL of ENGINEERING, SCIENCE and TECHNOLOGY

Senior Project Report

Kiddee Lab Registration System

Sai Aik Sei Mouk,
Noel Paing Oak Soe,
Khin Myo Aye

Asst. Prof. Dr. Darun Kesrarat (Advisor)

ITX3009/CSX3010 Senior Project 1 (1/2025)

Senior Project Approval

Project title:	Kiddee Lab Registration System
Academic Year:	1/2025
Authors:	Sai Aik Sei Mouk (6520057)
	Noel Paing Oak Soe (6530183)
	Khin Myo Aye (6540211)
Project Advisor:	Asst. Prof. Dr. Darun Kesrarat

The Senior Project committee's cooperation between the Department of Computer Science and Information Technology, Vincent Mary School of Engineering, Science and Technology, Assumption University had approved this Senior Project. The Senior Project in partial fulfilment of the requirement for the degree of Bachelor of Science in Computer Science and Information technology.

Approval Committee:

……..
(Asst. Prof. Dr Darun Kesrarat)
Project Advisor

……..
(Asst. Prof. Dr Anilkumar Kothalil Gopalakrishnan)
Committee Member

……..
(Dr. Phyo Min Tun)
Committee Member

Abstract

Kiddee Lab Registration System

This project aims to develop a registration system designed to minimize the effort required by registrars in scheduling classes and to serve as a centralized platform and foundation for future system enhancements. The system consists of a web application that enables registrars to schedule classes without manually verifying classroom availability or teacher schedules. Additionally, teachers can use the application to record student attendance and provide feedback, eliminating the need for manual paper-based records and physical logbooks. The system also incorporates an invoice and receipt creation feature, allowing registrars to generate invoices for course packages, apply management fees or discounts, select payment methods, and produce receipts for successful payments. This digital approach significantly reduces the time and effort needed to find conflict-free class schedules and manage administrative tasks. The primary focus of the system is to streamline class scheduling and attendance recording while providing an integrated billing solution, thereby establishing a comprehensive and centralized foundation system for Kiddee Lab company.

Table Of Contents

0

Chapter 1: Introduction 1

1.1 About 1

1.2 Problem Statement 1

1.3 Scope of the Project 2

1.4 Objectives 2

Chapter 2: Related Work 3

Chapter 3: Proposed Methodology 4

3.1 Methodology 4

3.2 Solving the Problem 5

3.3 Building the System – Key Features 6

3.4 ROI Analysis 7

Chapter 4: Design of The System (or Work) 8

4.1 Functional Requirements Specification 8

4.1.1 Stakeholders 9

4.1.2 Actors and Goals 9

4.1.3 Use Cases 10

4.2 System Design 11

4.2.1 Data Flow Diagram 12

4.2.2 Entity Relationship Diagram (ERD) 17

4.2.3 Class Diagram 26

4.2.4 Data Dictionary 32

4.4 Technology Used 43

4.6 Site Map 44

4.5 UX/UI Design 44

4.5.1 Log in Page 45

4.5.2 Sign in page screen 45

4.5.3 Forgot password screen 46

4.5.4 Manager UI 46

4.5.5 Registrar UI 76

4.5.6 Teacher UI 77

Chapter 5: Result 79

5.1 Functionality Achievements 79

5.2 Requirement Gathering & Testing 82

5.3 User Acceptance Testing & Adjustments 83

5.4 Limitations 83

Chapter 6: Conclusion 84

6.1 Future Work 84

References 85

Table Of Figures

Figure 1 Use Case Diagram 1.....11
Figure 2 Use Case Diagram 2.....11
Figure 3 Use Case Diagram 3.....12
Figure 4 Today Features Data Flow Diagram.....13
Figure 5 Schedule Data Flow Diagram.....14
Figure 6 Course Data Flow Diagram.....14
Figure 7 Enroll Data Flow Diagram.....15
Figure 8 Feedback Data Flow Diagram.....16
Figure 9 Create Invoice Data Flow Diagram.....17
Figure 10 ERD Overview.....18
Figure 11 ERD Detail 1.....19
Figure 12 ERD Detail 2.....20
Figure 13 ERD Detail 3.....21
Figure 14 ERD Detail 4.....23
Figure 15 Class Diagram Overview.....27
Figure 16 Class Diagram Detail 1.....27
Figure 17 Class Diagram Detail 2.....28

Table Of Tables

Table 1ERD Diagram Table.....23
Table 2 ERD Diagram Relationship Table.....26
Table 3 Class Diagram Table.....29
Table 4 Students Data Dictionary Table.....32
Table 5 Parents Data Dictionary Table.....34
Table 6 Teachers Data Dictionary Table.....34
Table 7 Courses Data Dictionary Table.....36
Table 8 Users Data Dictionary Table.....36
Table 9 ParentStudents Data Dictionary Table.....37
Table 10 TeacherCourses Data Dictionary Table.....38
Table 11 Sessions Data Dictionary Table.....38
Table 12 Schedules Data Dictionary Table.....40

Table 13 Technology Used Table.....44
Table 14 Requirement Gathering and Testing Table.....83

Chapter 1: Introduction

1.1 About

Kiddee Lab is a small children's activity school that offers a variety of courses for kids, such as coding, robotics, and design. It currently operates as a small-scale learning center located in Bangkok, Thailand, where parents can register their children for classes. The school also offers seasonal camps, workshops, and events that promote creative and technical learning for children.

The management team currently handles all student registration, schedule creation, and payment processes manually through spreadsheets, messages, and paper-based invoices. As the number of students and classes has grown, managing these tasks has become more complicated and time-consuming.

This project aims to design and implement a registration system for Kiddee Lab that helps the school efficiently manage its students, teachers, and course schedules, as well as automate the process of invoice and payment tracking.

1.2 Problem Statement

The current registration process at Kiddee Lab is inefficient and prone to human error because it heavily relies on manual coordination between staff members. The process of creating schedules, tracking student attendance, and managing payments often requires multiple communications between parents and teachers through LINE or other messaging platforms.

For example, when a parent wants to register a child, staff members must manually check class availability, confirm with teachers, and then update the schedule in separate spreadsheets. In addition, invoices and receipts are created manually, which increases the chances of mistakes in payment tracking.

As the school continues to grow, the existing manual methods are no longer scalable. There is also no centralized database to store and retrieve information about students, courses, and teachers, making it difficult to ensure data consistency and accessibility.

1.3 Scope of the Project

The scope of this project covers the development of an internal web-based registration system for Kiddee Lab. The system is primarily designed for administrative staff (managers and registrars) to handle class registration, scheduling, and payments efficiently. Teachers can also use the system to view their teaching schedules and student lists.

The scope does not include features for external users (such as parents or students) to log in or register directly. Instead, the system focuses on improving internal operations by reducing manual work and automating repetitive tasks.

The system will include modules for managing students, teachers, courses, schedules, invoices, and payments. Reports and summaries will be generated to help the management team make better operational decisions.

Limitations of this project include the fact that it is designed for use within a single organization (Kiddee Lab) and may require additional customization to be applied to other schools.

1.4 Objectives

The objectives of the project are as follows:

- To develop an internal web-based registration and management system for Kiddee Lab.
- To enable the management team to handle registrations, payments, and scheduling in one centralized platform.
- To reduce the workload of staff by automating the generation of invoices and receipts.
- To allow teachers to easily check their classes, student lists, and schedules.
- To improve data accuracy and ensure consistent records for all students, teachers, and courses.

Chapter 2: Related Work

In recent years, the increasing demand for efficient digital school management systems has led to the development of numerous registration and scheduling solutions. Many small to medium-sized educational institutions, such as private tutoring centers and after-school programs, still rely on manual methods — including spreadsheets, chat messages, and paper-based invoices — to manage their administrative workflows. These methods, while simple at first, become inefficient and error-prone as the number of students and classes grows.

Related systems in the education technology (EdTech) domain can generally be categorized into two types: **commercial off-the-shelf systems** and **custom-built internal systems**. Commercial systems, such as *ClassDojo*, *Teachworks*, and *Mindbody for Education*, provide ready-made solutions for student registration, scheduling, and billing. However, these systems often include features that are not relevant to smaller institutions, and they require monthly subscription fees that may not be cost-effective for small schools. Moreover, customization in such systems is usually limited, preventing institutions from adapting the software to fit their unique operational workflows.

On the other hand, custom-built systems are developed to meet specific institutional needs. These systems often focus on optimizing internal workflows, integrating with communication platforms, and reducing manual administrative tasks. For instance, many modern school management systems implement a **centralized database** architecture that allows administrators to access student information, course schedules, and teacher assignments from a single platform. They also support **role-based access control**, where each type of user (e.g., manager, teacher, registrar) is assigned different permissions depending on their responsibilities.

In the context of Kiddee Lab, a small creative learning center that offers courses such as coding and robotics, most commercial solutions available on the market are too broad and complex. They tend to include general features while Kiddee Lab has a unique workflow. Therefore, building a **tailored internal registration system** is a more practical approach.

Similar projects in previous studies have focused on building registration systems using various technologies such as PHP with MySQL, Node.js with MongoDB, or modern frameworks like React for front-end interfaces. These systems commonly adopt a modular design approach to

separate student management, scheduling, and payment modules. Some also include analytics dashboards for tracking course popularity or student attendance trends.

Another relevant aspect is **integration with communication platforms** such as LINE, WhatsApp, or email services. Educational institutions in Southeast Asia frequently use LINE as their primary communication channel between staff, parents, and teachers. Therefore, incorporating integration features or designing database structures compatible with LINE-based workflows can significantly improve usability and adoption among staff.

In summary, most existing registration systems share the same fundamental goal: to centralize data, automate repetitive processes, and reduce administrative errors. The Kiddee Lab Registration System follows a similar principle but focuses specifically on the operational workflow of a small-scale children's learning center. The system aims to bridge the gap between manual spreadsheet-based management and complex, expensive commercial solutions by providing a simple, efficient, and maintainable platform tailored to the organization's internal needs.

Chapter 3: Proposed Methodology

3.1 Methodology

The proposed methodology for this project is designed to ensure that the development of the Kiddee Lab Registration System meets the needs of both the management team and teaching staff. The development process began with a clear understanding of the school's existing registration workflow and the major challenges faced in managing student information, class schedules, and payments.

The methodology followed a **requirement-driven approach**, which involved gathering detailed business and functional requirements from the management team through observation, interviews, and process analysis. The system was designed iteratively, with continuous feedback from stakeholders to ensure that the final product aligns with the operational needs of Kiddee Lab.

The main focus was on usability, reliability, and accuracy — ensuring that the system would minimize human error and simplify the daily workload of the staff. The development approach can be summarized in the following phases:

1. **Requirement Gathering:** Understanding and documenting how the school currently operates.
2. **System Design:** Structuring a scalable database and intuitive user interface for internal use.
3. **System Development:** Building the system using modern web technologies and implementing all required functionalities.
4. **Testing:** Conducting functional and user acceptance testing to ensure that each feature meets its intended purpose.

By following these stages, the development process maintained a logical flow from problem identification to solution implementation.

3.2 Solving the Problem

The Kiddee Lab Registration System is designed to solve the major operational issues faced by the management team by introducing automation and centralization into daily administrative activities.

Previously, managing schedules, class registrations, and payments required the use of multiple spreadsheets and communication channels. This caused frequent inconsistencies, delayed updates, and difficulty in tracking invoices. To address these issues, the new system centralizes all information into a single database and provides distinct modules for managers, registrars, and teachers.

The **Business Model** of the system is straightforward — it focuses on internal efficiency rather than external sales or marketing. Managers and registrars use the system to handle course scheduling, student registration, and invoice generation. Teachers can view their assigned classes, student lists, and attendance. Each role has a dedicated user interface that allows them to perform only the functions related to their responsibilities, ensuring security and clarity.

The **System Objectives** reflect the school's core operational needs:

- Eliminate the need for manual spreadsheets in managing student data and schedules.

- Enable quick access to student profiles, teacher assignments, and payment history.
- Automate the invoice and receipt generation process.
- Provide real-time visibility into upcoming classes.
- Maintain accurate data consistency across all records.

In practice, the system acts as an integrated tool that links all necessary components — from class creation to invoice payment — reducing duplication of work and improving data accuracy. By streamlining these processes, the staff can focus more on academic quality and less on administrative tasks.

3.3 Building the System – Key Features

The Kiddee Lab Registration System is designed with modularity and scalability in mind. Each module serves a specific purpose and interacts with others through well-defined relationships. The key features of the system include:

- **Student Management:** Allows staff to add, edit, and view student details, including contact information, enrolled courses, and payment status.
- **Teacher Management:** Enables teachers to view their assigned classes and schedules, while allowing the management team to assign courses and update teacher information.
- **Course and Schedule Management:** Centralized control of class creation, scheduling, and session adjustments. This module ensures that overlapping or conflicting class times are minimized.
- **Invoice and Receipt Generation:** The system automatically generates invoices when a registration is confirmed, and updates payment status once received. It also allows for discount management.
- **Access Control System:** Implements different levels of access for managers, registrars, and teachers to maintain data privacy and prevent unauthorized modifications.

These features were derived directly from the functional and non-functional requirements collected during the analysis phase. The system's functional requirements primarily focus on what the system should do, while the non-functional requirements emphasize system performance, security, and ease of use.

Functional Requirements:

- The system must allow administrators to manage students, teachers, and courses.
- The system must allow registrars to generate and manage invoices.

- The system must allow teachers to view their class schedules.
- The system must generate financial summaries and attendance records.

Non-Functional Requirements:

- The system must be user-friendly.
- The system must ensure data accuracy and security.
- The system should handle multiple users simultaneously without performance degradation.
- The system must be maintainable and scalable for future feature additions.

By adhering to these requirements, the development ensures that the Kiddee Lab Registration System is both functional and reliable, providing a robust foundation for the school's internal operations.

3.4 ROI Analysis

Since Kiddee Lab is a small institution, the return on investment (ROI) for the registration system is primarily measured in **time savings and reduced administrative workload**, rather than direct financial profit. Before the implementation of this system, staff members spent considerable time managing registrations, following up on payments, and reconciling schedules manually.

With the new system, administrative processes become significantly faster and more accurate. The estimated ROI is reflected through the reduction in:

- Manual scheduling conflicts.
- Payment errors and lost invoices.
- Time spent cross-checking student and teacher records.

In addition, the digital nature of the system enables easy data backup and quick access to past transactions, reducing the risk of information loss. Over time, the system provides value by improving operational efficiency, freeing staff time, and increasing overall satisfaction among both administrators and teachers.

Chapter 4: Design of The System (or Work)

4.1 Functional Requirements Specification

The system's functional and non-functional requirements were gathered and documented after multiple discussions with the management team of Kiddee Lab. These requirements define how the system should operate and what specific functionalities it must support to ensure a smooth transition from manual to automated processes.

The overall goal is to provide a centralized system where managers, registrars, and teachers can efficiently perform their duties without relying on scattered spreadsheets or manual calculations.

Business Requirements:

- To manage all information regarding students, teachers, and classes in a single system.
- To enable the registrar to create, update, and cancel class schedules without data duplication.
- To support invoice and receipt generation linked to each registration.
- To allow teachers to view their assigned classes and corresponding student lists.
- To store all data in a structured database for easy retrieval and analysis.

Functional Requirements:

- The system must allow administrators to manage courses, sessions, and invoices.
- The system must allow the registrar to perform registration and payment processes.
- The system must allow the teacher to view schedules and student lists.

Non-Functional Requirements:

- The system must provide an intuitive and easy-to-use interface for staff.
- The system must ensure data consistency, accuracy, and security.
- The system should maintain fast performance even as the number of records grows.
- The system must be designed to allow future scalability and integration with communication platforms such as LINE.

By satisfying both functional and non-functional requirements, the Kiddee Lab Registration System ensures high efficiency and maintainability, providing a strong foundation for daily operational tasks.

4.1.1 Stakeholders

The primary stakeholders of the system include:

- **Manager:** Oversees all operations, including student management, class creation, financial tracking, and teacher assignments.
- **Registrar:** Handles registrations, payment records, and parent communication.
- **Teacher:** Views assigned classes, checks student lists, and tracks attendance.
- **Parents:** Indirect stakeholders who benefit from faster registration and more accurate invoicing.
- **Student:** The primary subject of the system whose enrollment, schedule, attendance, and learning progress are managed. As the end-recipient of the educational services, the student benefits from an organized and accurate learning experience facilitated by the system.

Each stakeholder interacts with the system differently, and permissions are granted based on their roles to ensure proper access control and data security.

4.1.2 Actors and Goals

The following summarizes the primary actors and their main goals within the system:

- **Manager:** Manage all school operations.
- **Registrar:** Efficiently register students, update schedules, and generate invoices.
- **Teacher:** Check assigned classes, attendance, and student information.
- **System:** Maintain accurate, up-to-date information and automate repetitive administrative tasks.

These actors represent the main operational workflow at Kiddee Lab and serve as the basis for the system’s use case design.

4.1.3 Use Cases

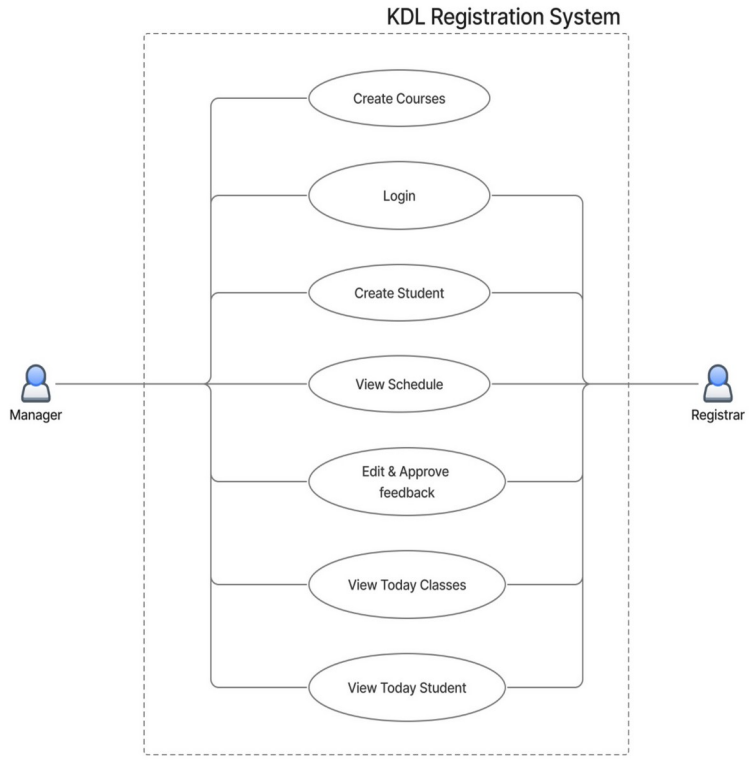


Figure 1 Use Case Diagram 1

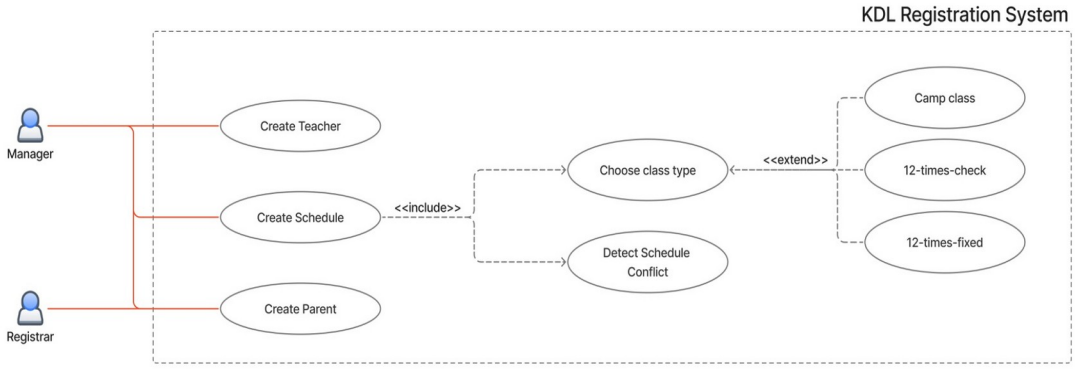


Figure 2 Use Case Diagram 2

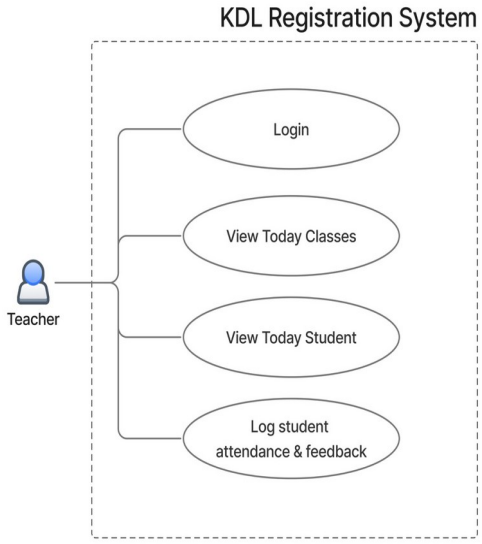


Figure 3 Use Case Diagram 3

The use case diagram describes the relationships between different user roles and system functionalities. It identifies the core actions each actor can perform:

- **Manager:** Create and manage course, user and fees.
- **Registrar:** Register students, update payment records, generate invoices, and communicate updates.
- **Teacher:** View teaching schedule, class details, and student lists.

The system supports a clear separation of roles, ensuring that each user only accesses relevant features.

4.2 System Design

4.2.1 Data Flow Diagram

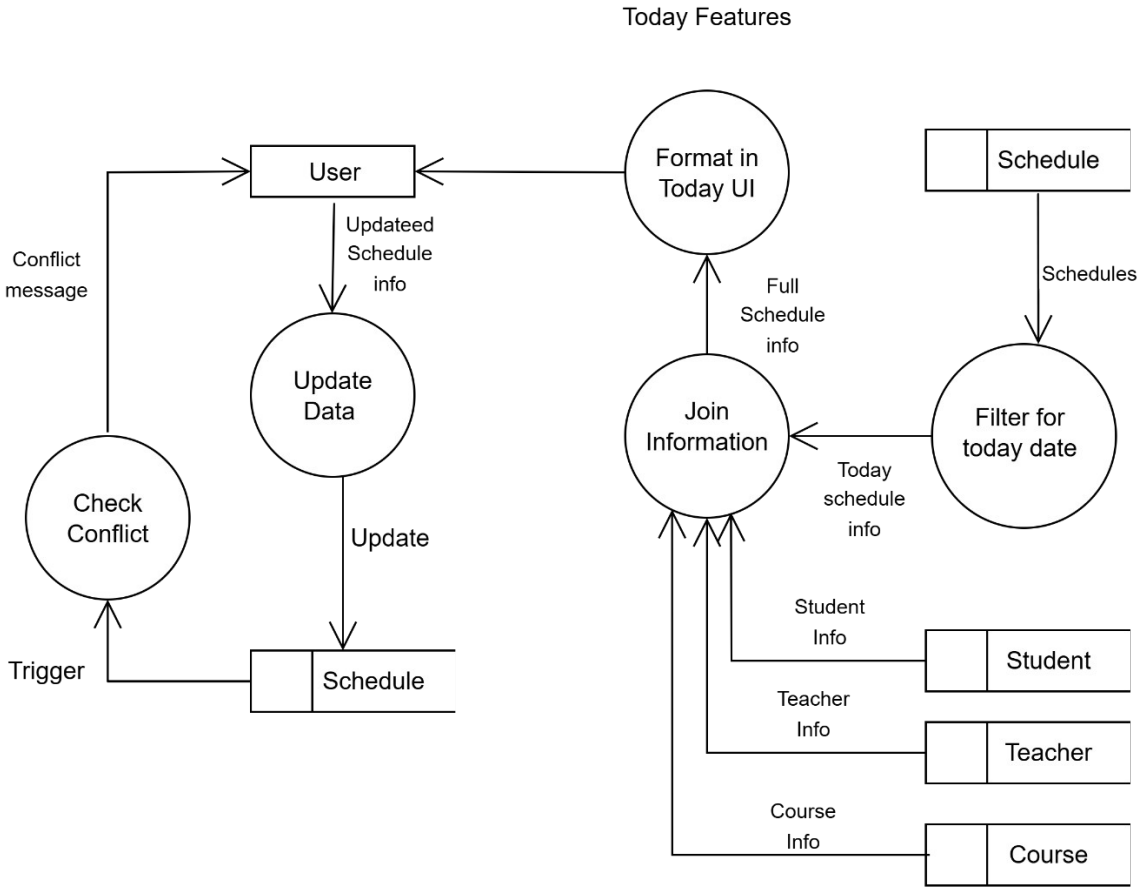


Figure 4 Today Features Data Flow Diagram

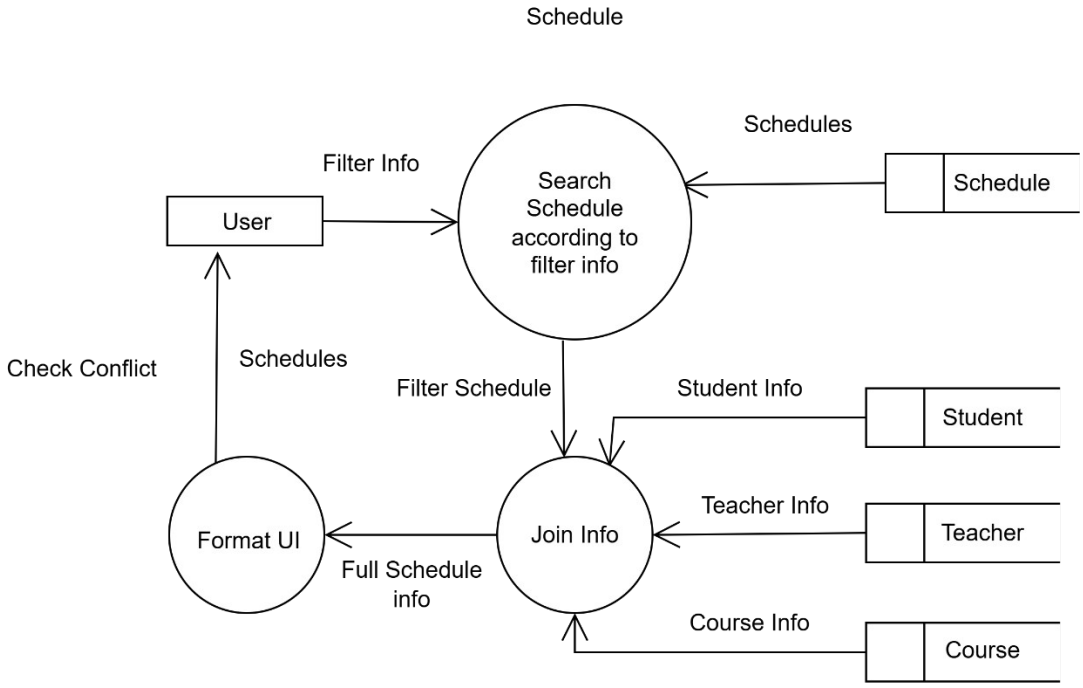


Figure 5 Schedule Data Flow Diagram

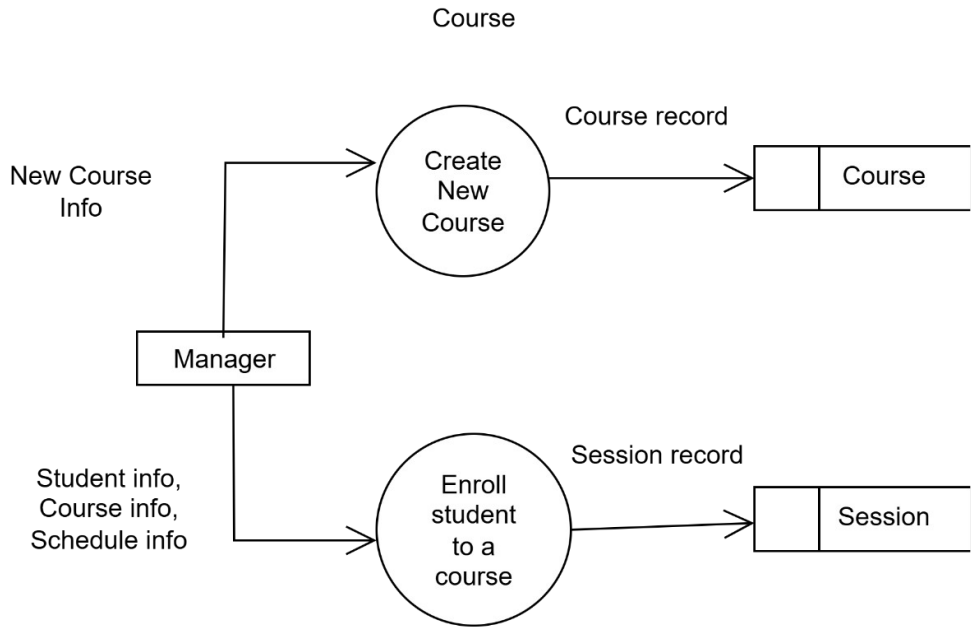


Figure 6 Course Data Flow Diagram

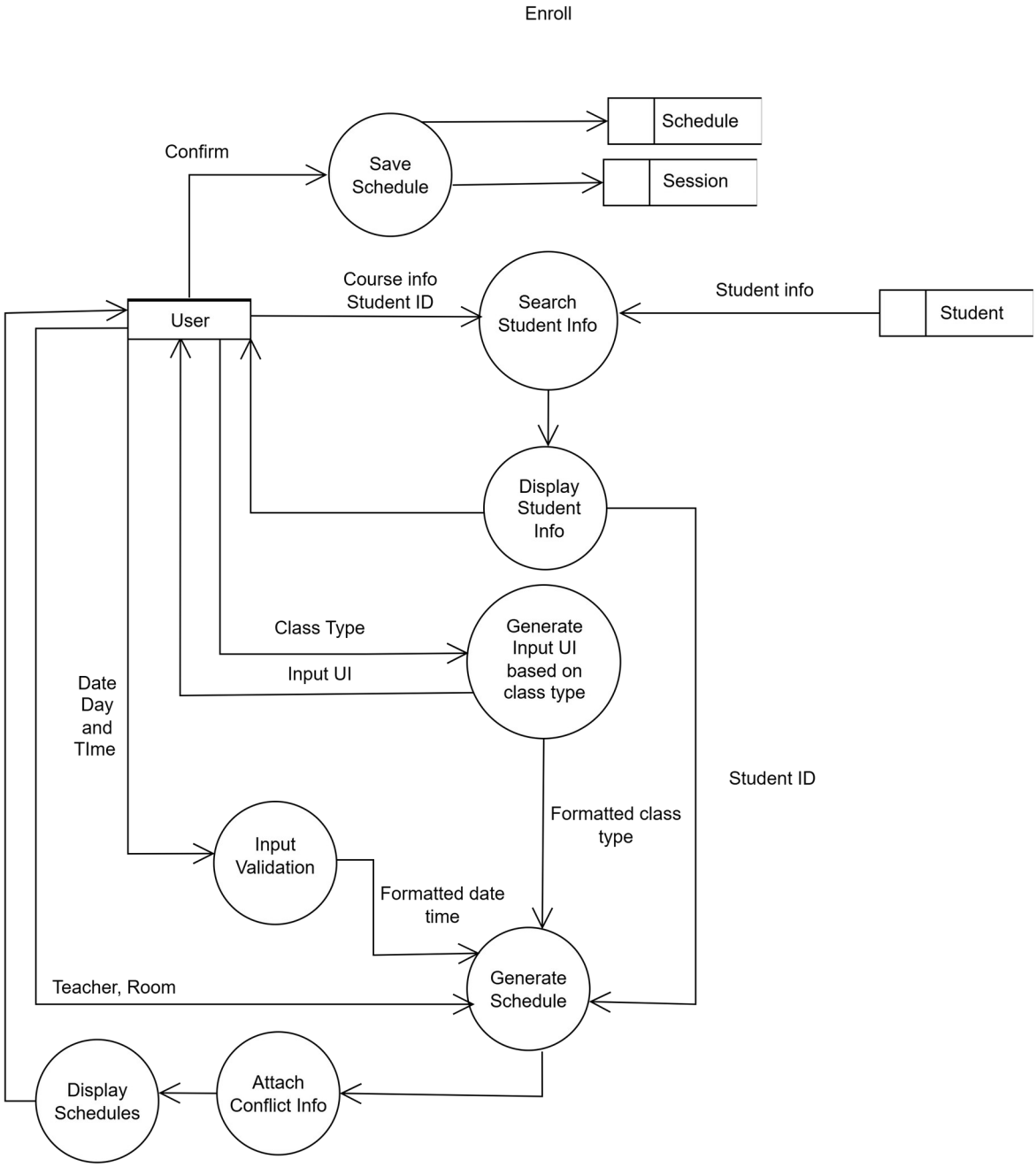


Figure 7 Enroll Data Flow Diagram

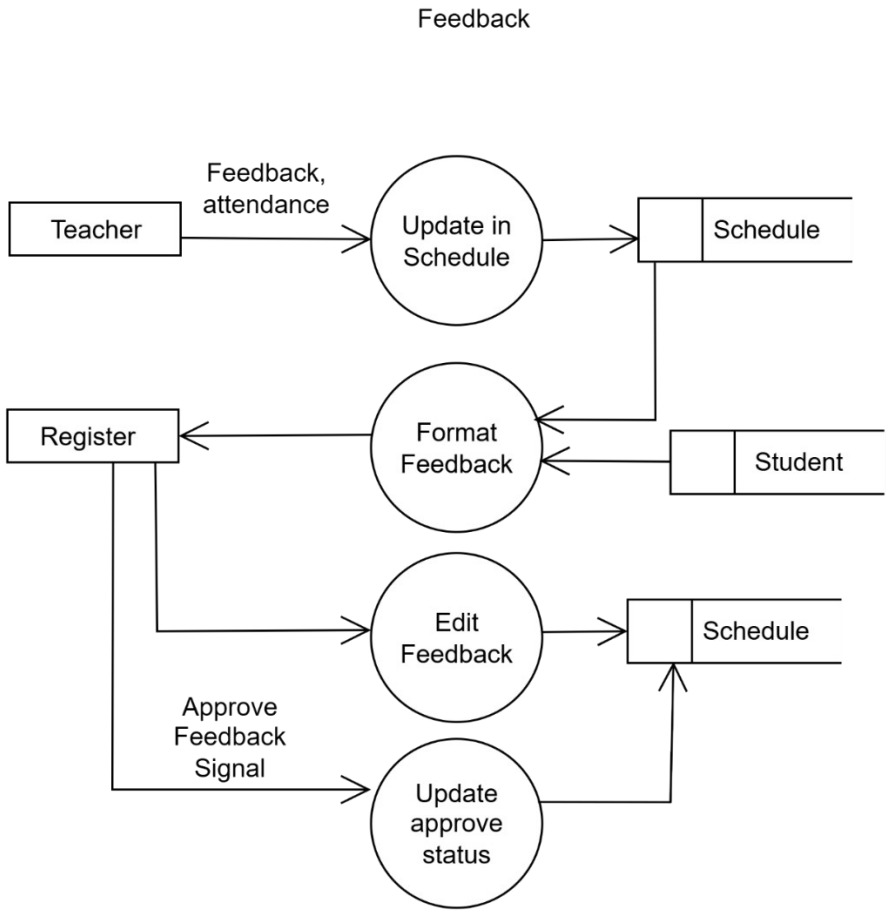


Figure 8 Feedback Data Flow Diagram

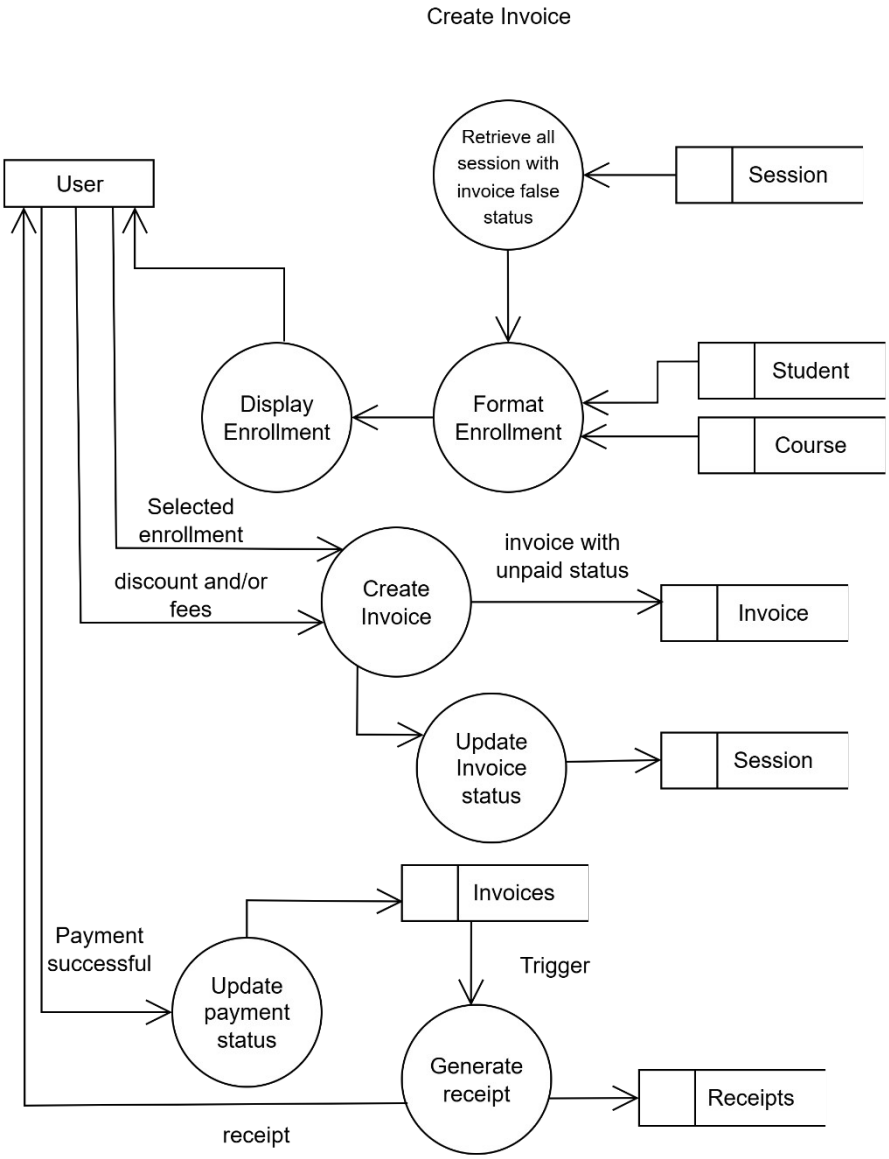


Figure 9 Create Invoice Data Flow Diagram

The Data Flow Diagram (DFD) represents how data moves between different components of the system. It provides an overview of the logical processes and data stores that make up the registration system.

The main processes identified in the DFD include:

- **Student Registration Process:** Handles the entry and storage of student information.
- **Course Management Process:** Involves adding, editing, and deleting courses.
- **Schedule Generation Process:** Automatically creates timetables based on course and teacher availability.
- **Invoice Process:** Generates and tracks invoices related to each registration.

Data stores such as *Student Database*, *Teacher Database*, and *Invoice Database* ensure that all records remain consistent and easily retrievable.

4.2.2 Entity Relationship Diagram (ERD)

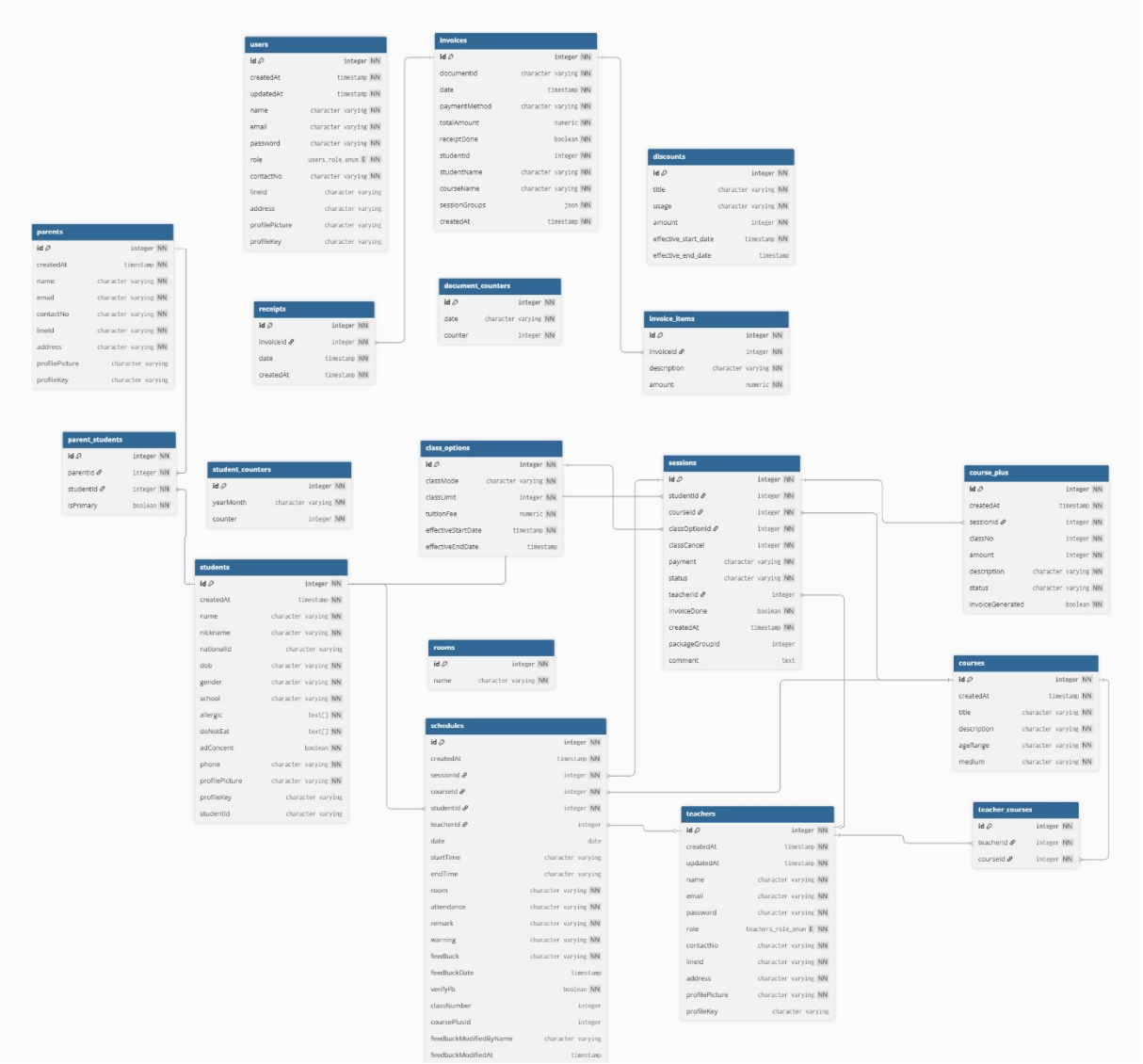


Figure 10 ERD Overview

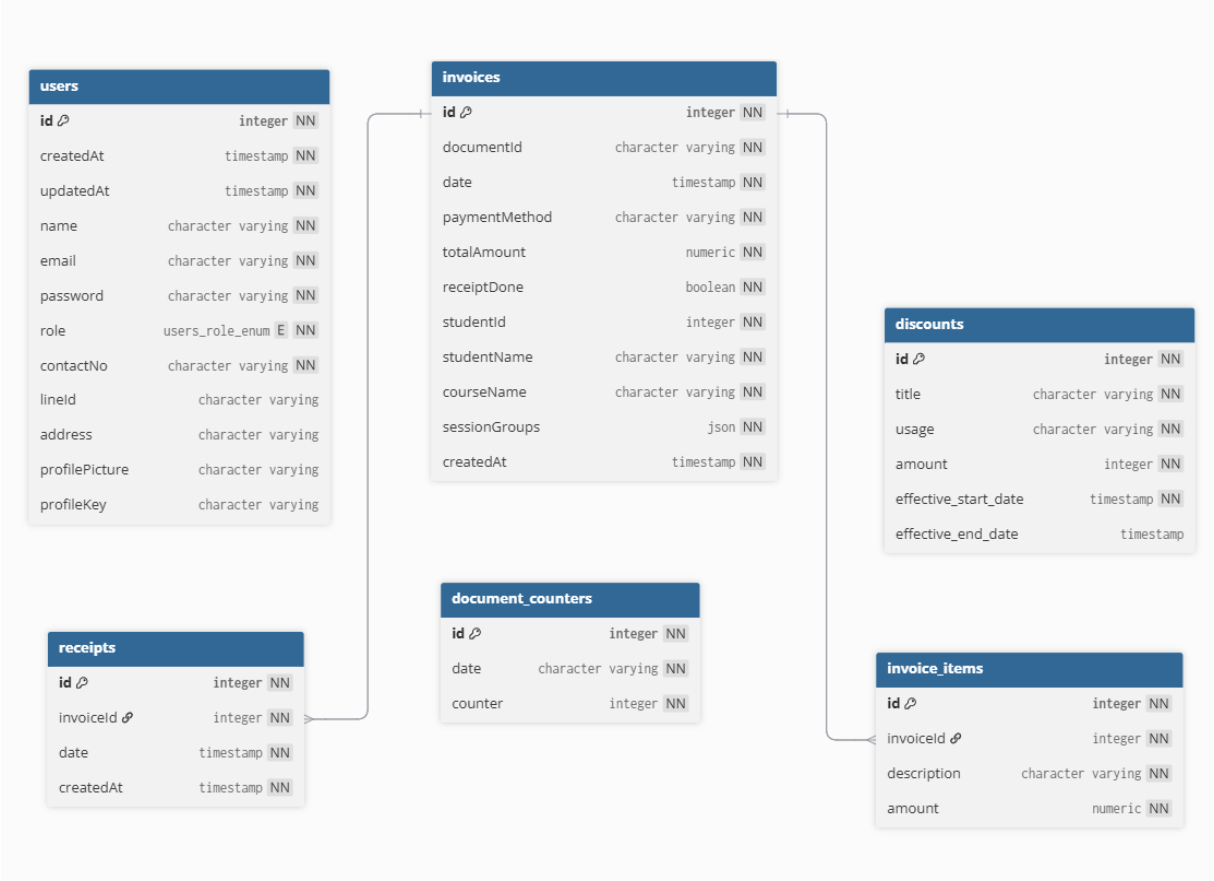


Figure 11 ERD Detail 1

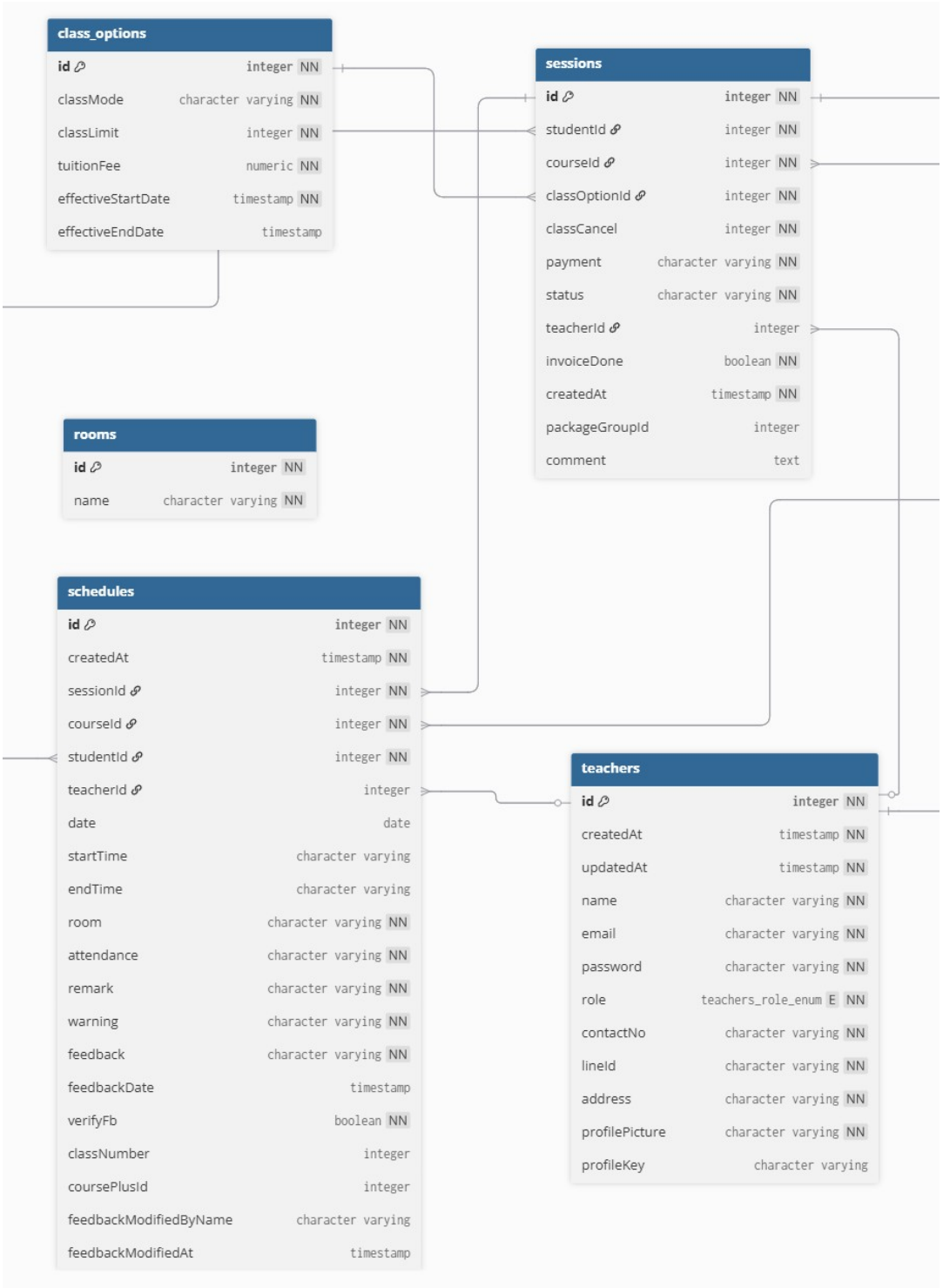


Figure 12 ERD Detail 2

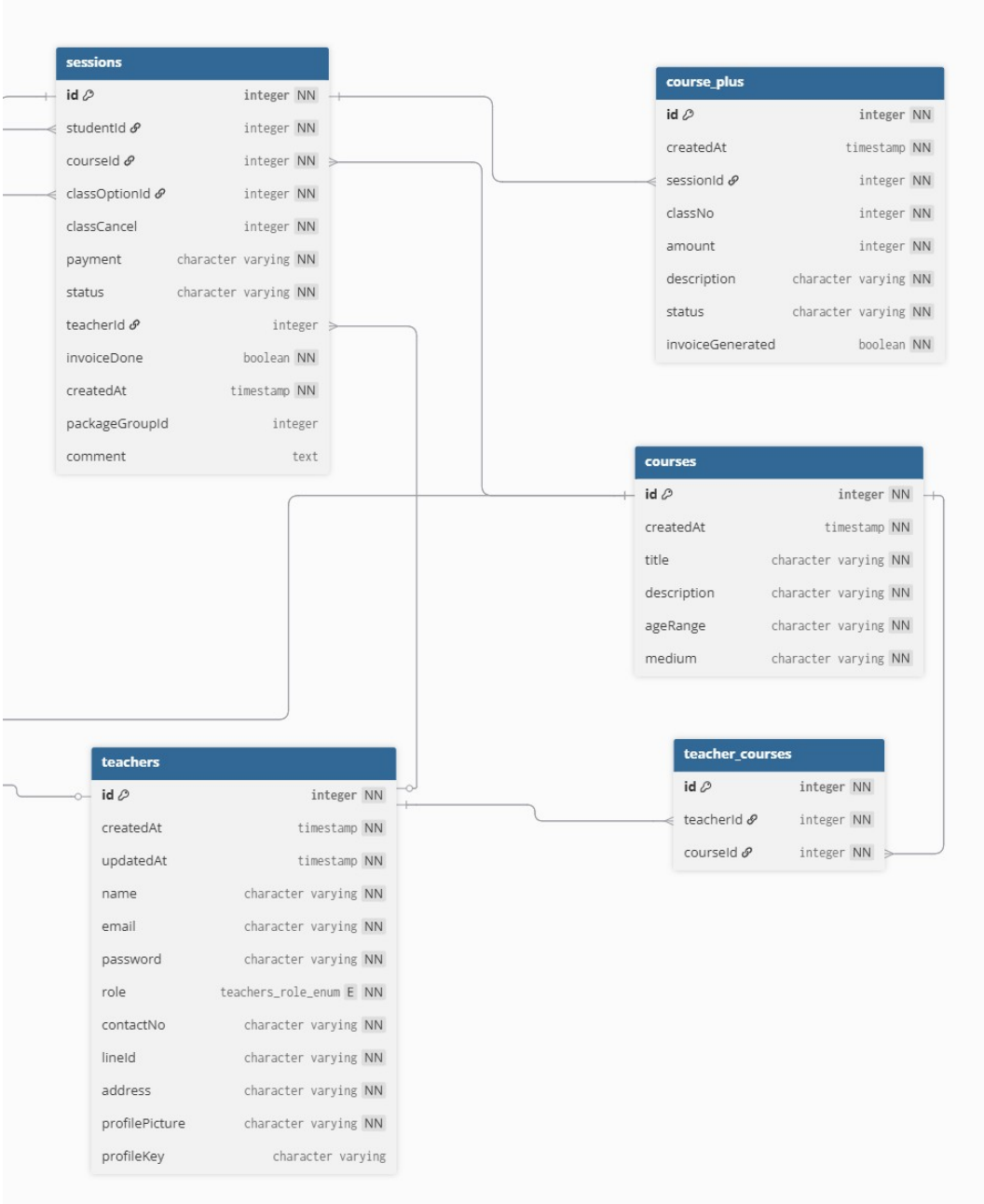


Figure 13 ERD Detail 3

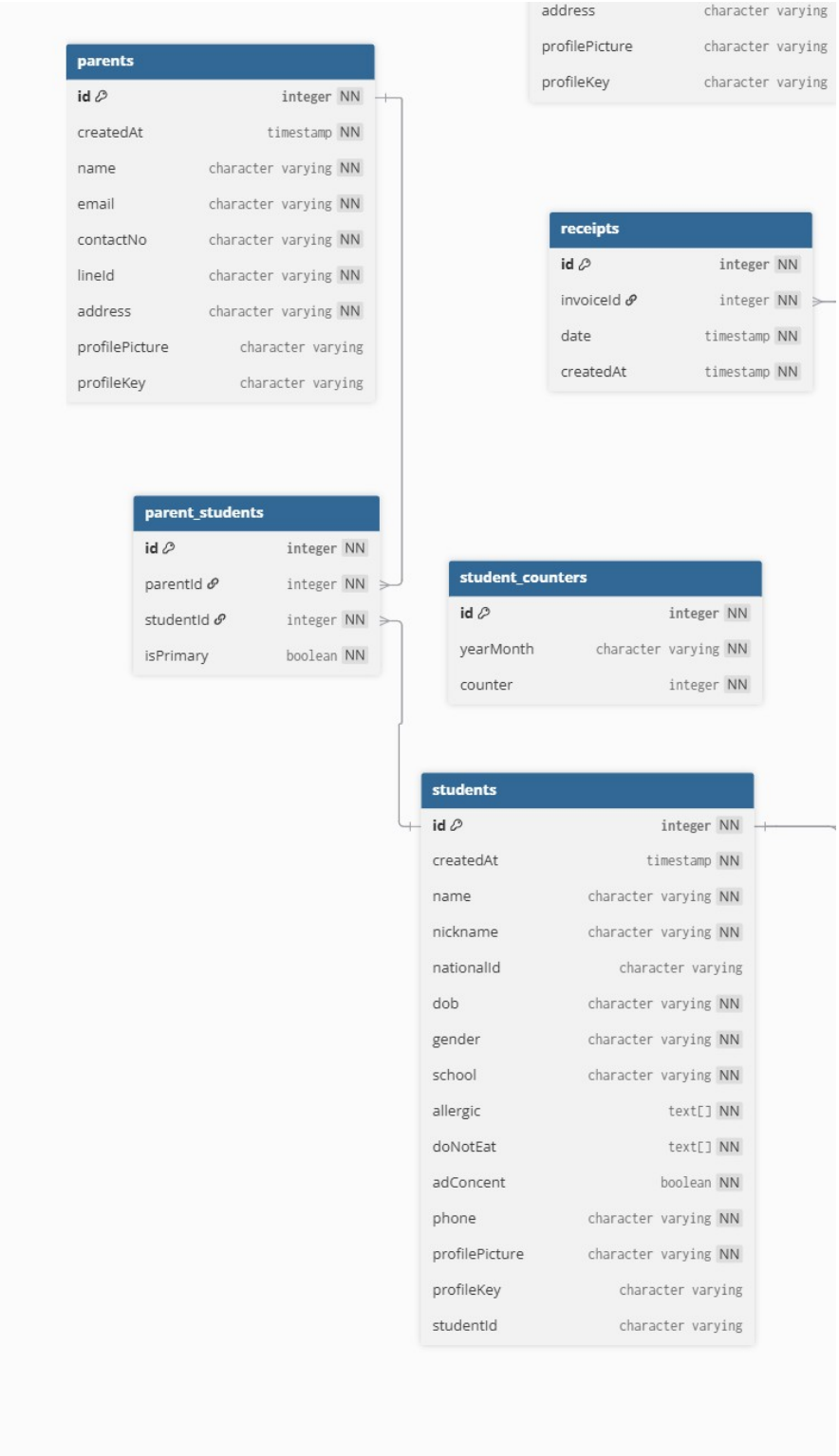


Figure 14 ERD Detail 4

The Entity-Relationship Diagram (ERD) illustrates the logical structure of the Kiddee Lab Registration System database. It identifies the main entities, their relationships, and how data flows within the system. The ERD ensures that all information about students, parents, courses, teachers, schedules, and financial transactions is well-organized and interconnected. By

visualizing these relationships, the diagram helps maintain data consistency, reduce duplication, and support smooth operations such as registration, enrollment tracking, invoicing, and attendance management.

Table 1 ERD Diagram Table

Entity	**Description**	**Relationships**
Student	Represents each child registered in the system. The student entity holds the main identity for learning activities, attendance, enrollment, and billing.	One student can have many parents (via *Parent_Student*), many sessions, many schedules, and multiple invoices.
Parent	Represents the guardians responsible for the students. Used mainly for communication, billing, and record linkage.	A parent can have multiple students, and a student can have multiple parents (many-to-many via *Parent_Student*).
Parent_Student	Serves as a link table between parents and students to manage many-to-many relationships. Also identifies the primary guardian for each student.	Connects *Parent* and *Student*.
Course	Defines the available programs or subjects offered by the institution. It is the foundation for scheduling and enrollment.	One course can have many sessions and many teachers (via *Teacher_Course*).
Class_Option	Represents the configuration of a course — such as learning mode,	One class option can apply to

	class size, and tuition type.	multiple sessions.
Teacher	Represents instructors who conduct classes. Each teacher can be linked to multiple courses and sessions.	Many-to-many with *Course* (via *Teacher_Course*); one-to-many with *Session* and *Schedule*.
Teacher_Course	Junction table that connects teachers and courses to define which teacher can teach which subject.	Connects *Teacher* and *Course*.
Session	Represents a student's enrollment in a specific course under certain conditions (teacher, schedule, payment, etc.).	One student, one course, one teacher per session; one-to-many with *Schedule*, *Course_Plus*, and *Invoice*.
Course_Plus	Represents additional or extension sessions that belong to an existing enrollment.	Many *Course_Plus* records can be linked to one *Session*.
Schedule	Defines the date, time, and location for each class session. Also used for attendance tracking.	One schedule belongs to one session, one student, one teacher, and one course.
Invoice	Records billing information for students' course enrollments or other fees.	One student can have multiple invoices; each invoice can have many *Invoice_Items* and one *Receipt*.
Invoice_Item	Represents detailed billing entries within an invoice (e.g., tuition,	Many items belong to one

	materials).	invoice.
Receipt	Confirms payment for a particular invoice.	One-to-one relationship with *Invoice*.
Discount	Stores available discount programs and validity periods. Applied during invoicing or registration.	Can be associated logically with *Invoices* or *Courses* depending on policy (no direct FK).
User	Represents general system users (admin, registrar) for authentication and access control.	Independent but may correspond to teachers or registrars.
Room	Represents physical or virtual classrooms where sessions are held.	Referenced by *Schedule* (each schedule occurs in one room).
Document_Counter	Keeps track of document numbering sequences, ensuring unique invoice or receipt codes.	Independent utility table.
Student_Counter	Keeps track of student ID numbering for consistent registration codes.	Independent utility table.

Summary of Major Relationships

Table 2 ERD Diagram Relationship Table

Relationship	**Type**	**Explanation**

Student ↔ Parent	Many-to-Many	A student can have multiple parents; a parent can have multiple students.
Student → Session	One-to-Many	Each student can enroll in many sessions.
Course → Session	One-to-Many	Each course can have multiple enrolled sessions.
Teacher ↔ Course	Many-to-Many	Teachers can teach multiple courses; courses can have multiple teachers.
Teacher → Session	One-to-Many	A teacher can conduct multiple sessions.
Session → Schedule	One-to-Many	Each session can have many scheduled classes.
Session → Invoice	One-to-Many	Each session may generate multiple invoices.
Invoice → Invoice_Item	One-to-Many	Each invoice can contain several detailed items.
Invoice → Receipt	One-to-One	Each invoice is linked to a single payment receipt.

4.2.3 Class Diagram

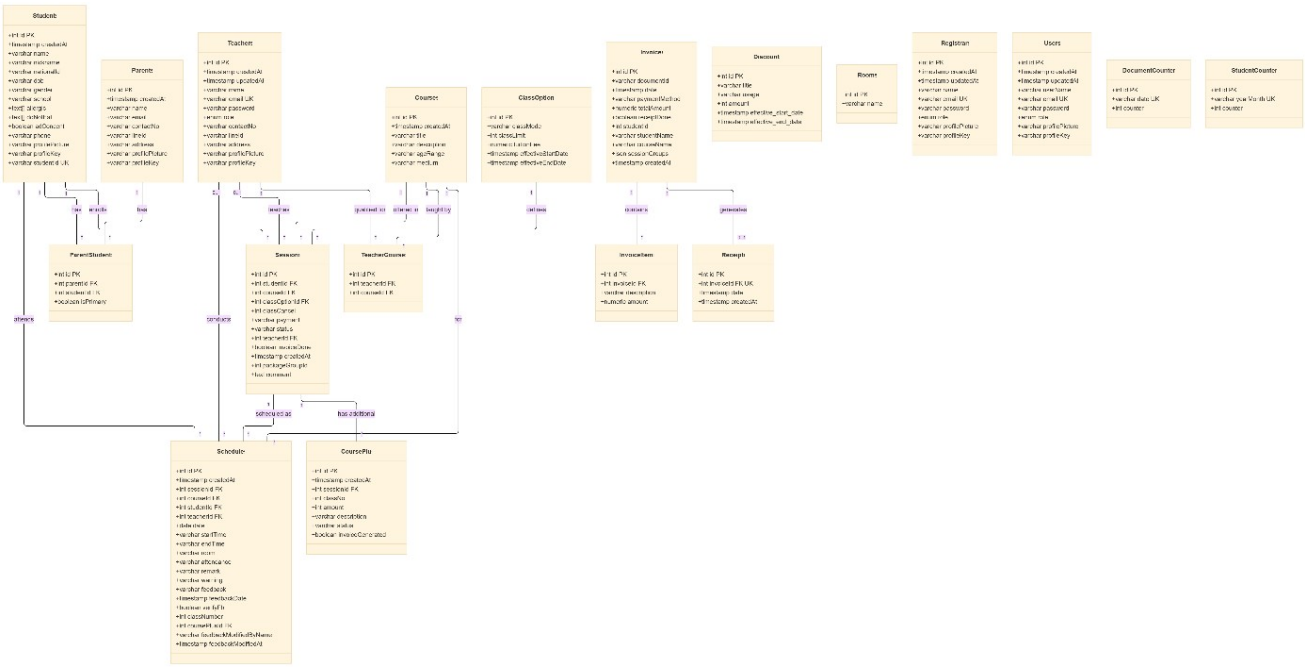


Figure 15 Class Diagram Overview

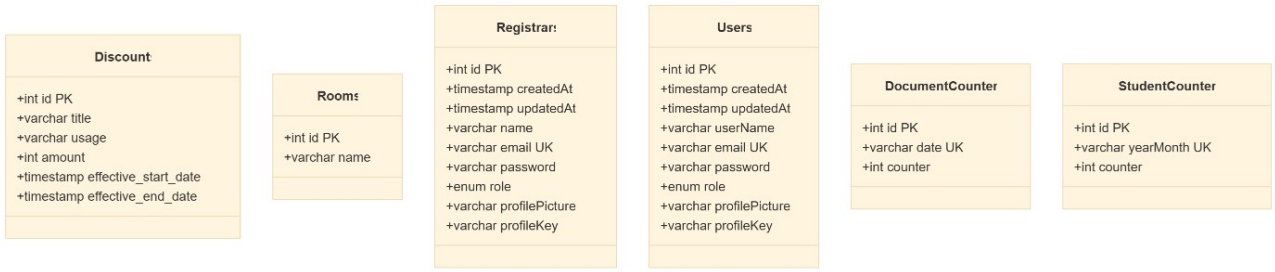


Figure 16 Class Diagram Detail 1

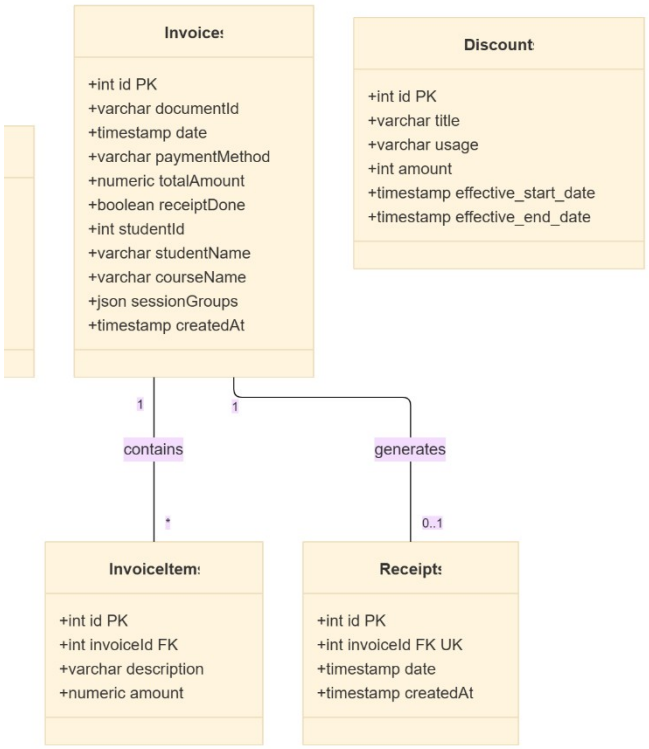


Figure 17 Class Diagram Detail 2

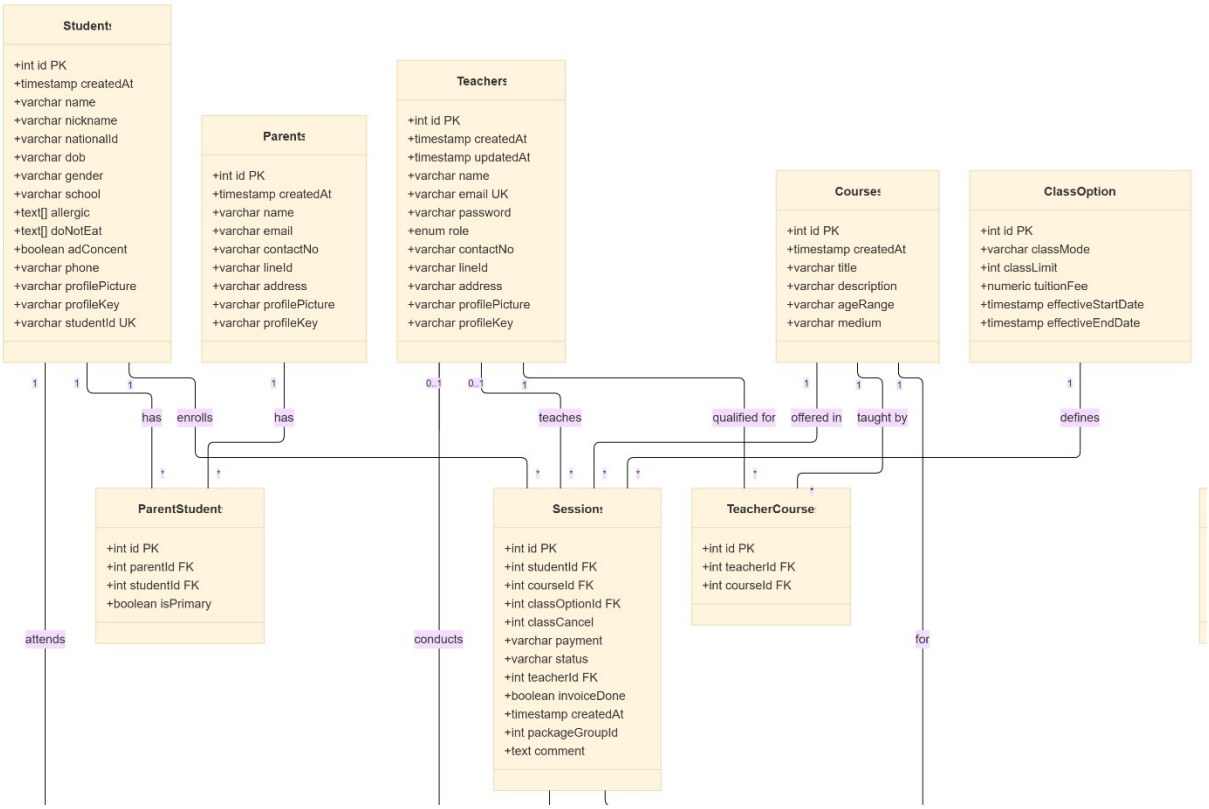


Figure 18 Class Diagram Detail 3

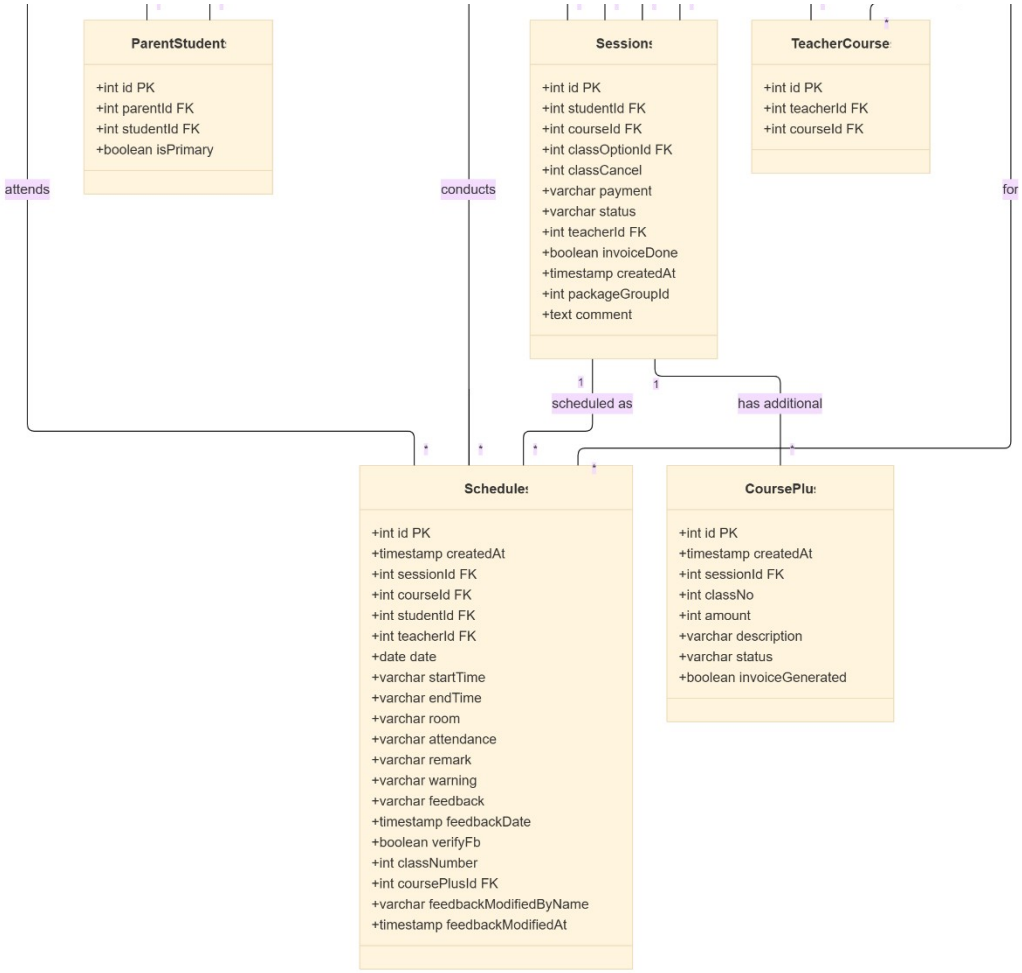


Figure 19 Class Diagram Detail 4

The class diagram represents the object-oriented design of the Kiddee Lab Registration System. It identifies the key classes, their responsibilities, and the relationships among them. Each class corresponds to an important functional part of the system, such as managing students, courses, schedules, or billing. The diagram helps illustrate how data entities in the database are transformed into interactive software objects that support system operations.

Table 3 Class Diagram Table

Class	**Description**	**Relationships / Interactions**
Student	Represents a student object in the system. Responsible for managing	Interacts with *Parent*, *Session*, *Invoice*, and

	student-related actions such as enrollment, attendance tracking, and profile updates.	*Schedule* classes.
Parent	Represents a parent or guardian. Handles actions like viewing student progress, managing contact details, and linking to multiple students.	Linked to *Student* (many-to-many via *ParentStudent*).
ParentStudent	Acts as an association class connecting *Parent* and *Student*. It ensures each relationship is clearly defined (e.g., primary guardian).	Connects *Parent* and *Student* classes.
Course	Represents a course or subject offered. Defines course details, duration, and content. Responsible for grouping sessions and schedules.	Connected to *Session*, *Teacher*, and *Schedule*.
ClassOption	Represents course configuration options such as class size, delivery mode, and tuition fee.	Used by *Session* to determine course settings.
Teacher	Represents instructor details and responsibilities. Manages teaching sessions, student progress, and schedules.	Linked to *Course* (many-to-many) and *Session*.
TeacherCourse	Association class defining which teacher can teach which courses.	Connects *Teacher* and *Course*.

Session	Represents an enrollment instance connecting a student, course, and teacher. Handles attendance, payment, and performance tracking.	Central link between *Student*, *Course*, and *Teacher*. Connected to *Schedule*, *CoursePlus*, and *Invoice*.
CoursePlus	Represents extended or special sessions added to an existing enrollment.	Linked to *Session*.
Schedule	Manages the date, time, and attendance of each class meeting. Responsible for recording attendance and feedback.	Linked to *Session*, *Student*, *Teacher*, and *Room*.
Invoice	Handles billing for a student's course or session. Responsible for generating, updating, and tracking payments.	Linked to *Student*, *InvoiceItem*, and *Receipt*.
InvoiceItem	Represents each line item on an invoice, such as tuition or materials.	Belongs to one *Invoice*.
Receipt	Represents proof of payment. Handles payment confirmation and linkage to invoices.	One-to-one relationship with *Invoice*.
Discount	Manages discount rules and validity. Applied during invoice creation.	Can interact with *Invoice* or *Course*.

User	Represents any authenticated user in the system (admin, registrar). Handles login, roles, and permissions.	Administrative Staffs
Room	Represents physical or virtual class locations used for scheduling.	Used by *Schedule*.
DocumentCounter / StudentCounter	Utility classes for generating unique document or student IDs.	Standalone, used internally for sequence control.

4.2.4 Data Dictionary

Students

Description: Stores information about students enrolled in the system.

Table 4 Students Data Dictionary Table

Column Name	Data Type	Constraints	Description
id	INTEGER	PK, AUTO_INCREMENT	Unique student identifier
createdAt	TIMESTAMP	NOT NULL, DEFAULT NOW()	Record creation timestamp
name	VARCHAR	NOT NULL	Full legal name of the student
nickname	VARCHAR	NOT NULL	Preferred name or nickname
nationalId	VARCHAR	NULLABLE	National identification

			number
dob	VARCHAR	NOT NULL	Date of birth
gender	VARCHAR	NOT NULL	Student's gender
school	VARCHAR	NOT NULL	Current school name
allergic	TEXT[]	NOT NULL, DEFAULT []	List of allergies
doNotEat	TEXT[]	NOT NULL, DEFAULT []	List of food restrictions
adConcent	BOOLEAN	NOT NULL	Advertisement consent flag
phone	VARCHAR	NOT NULL	Contact phone number
profilePicture	VARCHAR	NOT NULL	URL to profile picture
profileKey	VARCHAR	NULLABLE	Storage key for profile picture
studentId	VARCHAR	UNIQUE	System-generated student ID

Parents

Description: Contains information about parents/guardians of students.

Table 5 Parents Data Dictionary Table

Column Name	Data Type	Constraints	Description

id	INTEGER	PK, AUTO_INCREMENT	Unique parent identifier
createdAt	TIMESTAMP	NOT NULL, DEFAULT NOW()	Record creation timestamp
name	VARCHAR	NOT NULL	Full name of parent/guardian
email	VARCHAR	NOT NULL	Email address
contactNo	VARCHAR	NOT NULL	Primary contact number
lineId	VARCHAR	NOT NULL	LINE messaging app ID
address	VARCHAR	NOT NULL	Residential address
profilePicture	VARCHAR	NULLABLE	URL to profile picture
profileKey	VARCHAR	NULLABLE	Storage key for profile picture

Teachers

Description: Stores teacher/instructor information and credentials.

Table 6 Teachers Data Dictionary Table

Column Name	Data Type	Constraints	Description
id	INTEGER	PK, AUTO_INCREMENT	Unique teacher identifier
createdAt	TIMESTAMP	NOT NULL, DEFAULT	Record creation timestamp

		NOW()	
updatedAt	TIMESTAMP	NOT NULL, DEFAULT NOW()	Last update timestamp
name	VARCHAR	NOT NULL	Full name of teacher
email	VARCHAR	NOT NULL, UNIQUE	Email address for login
password	VARCHAR	NOT NULL	Hashed password
role	ENUM	NOT NULL, DEFAULT 'teacher'	User role (admin/registrar/teacher)
contactNo	VARCHAR	NOT NULL	Contact phone number
lineId	VARCHAR	NOT NULL	LINE messaging app ID
address	VARCHAR	NOT NULL	Residential address
profilePicture	VARCHAR	NOT NULL	URL to profile picture
profileKey	VARCHAR	NULLABLE	Storage key for profile picture

Courses

Description: Defines available courses/programs offered by the institution.

Table 7 Courses Data Dictionary Table

Column Name	Data Type	Constraints	Description

id	INTEGER	PK, AUTO_INCREMENT	Unique course identifier
createdAt	TIMESTAMP	NOT NULL, DEFAULT NOW()	Record creation timestamp
title	VARCHAR	NOT NULL	Course title/name
description	VARCHAR	NOT NULL	Detailed course description
ageRange	VARCHAR	NOT NULL	Target age range for students
medium	VARCHAR	NOT NULL	Language/medium of instruction

Users

Description: General user accounts for system access.

Table 8 Users Data Dictionary Table

Column Name	**Data Type**	**Constraints**	**Description**
id	INTEGER	PK, AUTO_INCREMENT	Unique user identifier
createdAt	TIMESTAMP	NOT NULL, DEFAULT NOW()	Record creation timestamp
updatedAt	TIMESTAMP	NOT NULL, DEFAULT NOW()	Last update timestamp
userName	VARCHAR	NOT NULL	Display username
email	VARCHAR	NOT NULL, UNIQUE	Email address for login

password	VARCHAR	NOT NULL	Hashed password
role	ENUM	NOT NULL, DEFAULT 'none'	User role (admin/registrar/teacher/none)
profilePicture	VARCHAR	NOT NULL, DEFAULT ''	URL to profile picture
profileKey	VARCHAR	NULLABLE	Storage key for profile picture

ParentStudents

Description: Junction table linking parents to their children (students).

Table 9 ParentStudents Data Dictionary Table

Column Name	Data Type	Constraints	Description
id	INTEGER	PK, AUTO_INCREMENT	Unique relationship identifier
parentId	INTEGER	FK → parents.id, NOT NULL	Reference to parent
studentId	INTEGER	FK → students.id, NOT NULL	Reference to student
isPrimary	BOOLEAN	NOT NULL, DEFAULT false	Indicates primary guardian

TeacherCourses

Description: Junction table linking teachers to courses they are qualified to teach.

Table 10 TeacherCourses Data Dictionary Table

Column Name	Data Type	Constraints	Description
id	INTEGER	PK, AUTO_INCREMENT	Unique relationship identifier
teacherId	INTEGER	FK → teachers.id, NOT NULL	Reference to teacher
courseId	INTEGER	FK → courses.id, NOT NULL	Reference to course

Sessions

Description: Represents a student's enrollment in a specific course with defined parameters.

Table 11 Sessions Data Dictionary Table

Column Name	Data Type	Constraints	Description
id	INTEGER	PK, AUTO_INCREMENT	Unique session identifier
studentId	INTEGER	FK → students.id, NOT NULL	Enrolled student
courseId	INTEGER	FK → courses.id, NOT NULL	Selected course
classOptionId	INTEGER	FK → class_options.id, NOT NULL	Class configuration
classCancel	INTEGER	NOT NULL	Number of cancelled classes
payment	VARCHAR	NOT NULL	Payment status

status	VARCHAR	NOT NULL	Session status
teacherId	INTEGER	FK → teachers.id, NULLABLE	Assigned teacher
invoiceDone	BOOLEAN	NOT NULL, DEFAULT false	Invoice generation flag
createdAt	TIMESTAMP	NOT NULL, DEFAULT NOW()	Session creation timestamp
packageGroupId	INTEGER	NULLABLE	Package grouping identifier
comment	TEXT	NULLABLE	Additional notes/comments

Schedules

Description: Individual class schedules and attendance records.

Table 12 Schedules Data Dictionary Table

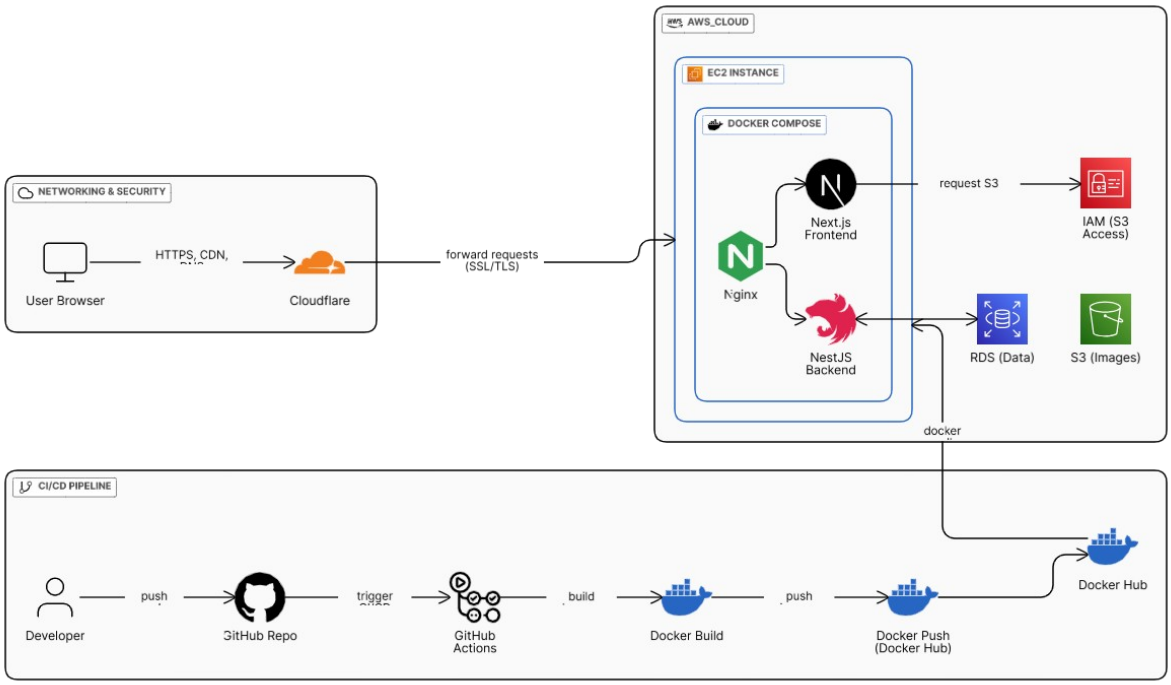
Column Name	**Data Type**	**Constraints**	**Description**
id	INTEGER	PK, AUTO_INCREMENT	Unique schedule identifier
createdAt	TIMESTAMP	NOT NULL, DEFAULT NOW()	Record creation timestamp
sessionId	INTEGER	FK → sessions.id, NOT NULL	Related session
courseId	INTEGER	FK → courses.id, NOT	Course being taught

		NULL	
studentId	INTEGER	FK → students.id, NOT NULL	Attending student
teacherId	INTEGER	FK → teachers.id, NULLABLE	Teaching instructor
date	DATE	NULLABLE	Scheduled date
startTime	VARCHAR	NULLABLE	Class start time
endTime	VARCHAR	NULLABLE	Class end time
room	VARCHAR	NOT NULL	Room/location name
attendance	VARCHAR	NOT NULL	Attendance status
remark	VARCHAR	NOT NULL	General remarks
warning	VARCHAR	NOT NULL	Warning/alert notes
feedback	VARCHAR	NOT NULL	Teacher feedback
feedbackDate	TIMESTAMP	NULLABLE	Feedback submission date
verifyFb	BOOLEAN	NOT NULL	Feedback verification flag
classNumber	INTEGER	NULLABLE	Class sequence number
coursePlusId	INTEGER	FK → course_plus.id, NULLABLE	Related additional course
feedbackModif	VARCHAR	NULLABLE	Last modifier name

iedByName			
feedbackModifiedAt	TIMESTAMP	NULLABLE	Last modification timestamp

These tables form the backbone of the system's relational database. All CRUD (Create, Read, Update, Delete) operations are performed through the web interface, ensuring efficient data management and consistency across all records.

4.3 System Architecture



The system follows a **client-server model** with a modern web stack and containerized deployment. The architecture is designed for scalability, modularity, and ease of maintenance.

1. **Client Layer**:
    - Users access the system through a web browser.
    - Requests are routed through **Cloudflare**, which provides HTTPS security, CDN caching, and DDoS protection.

2. **Server Layer**:
    - Hosted on an **AWS EC2 instance**, running services inside **Docker Compose** for containerization.
    - **Nginx** serves as a reverse proxy, routing requests between the frontend and backend.
    - **Next.js** handles the frontend, providing a dynamic and responsive user interface.
    - **NestJS** implements the backend, managing business logic, validation, and API endpoints.
3. **Data Layer**:
    - **PostgreSQL (RDS)** stores structured application data.
    - **AWS S3** stores image assets, with backend access managed via **IAM** permissions.
4. **CI/CD Pipeline**:
    - Developers push code to **GitHub**.
    - **GitHub Actions** automatically build Docker images.
    - Docker images are pushed to **Docker Hub** and deployed to the EC2 instance.

This setup ensures high modularity, security, and scalability, supporting future integration with other cloud services or APIs.

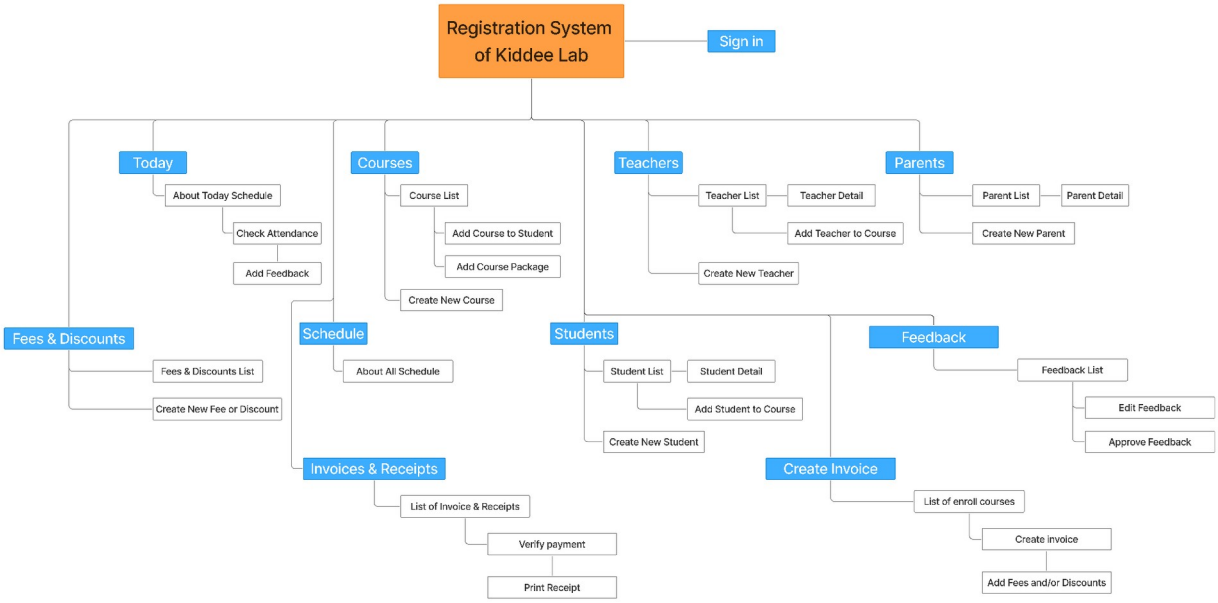
4.4 Technology Used

Table 13 Technology Used Table

Layer	**Technology**	**Purpose**
Frontend	**Next.js**	React-based framework for server-side rendering and dynamic UI

Backend	**NestJS**	Node.js framework for scalable and structured server logic
Database	**PostgreSQL (RDS)**	Relational database for storing application data
Cloud & Deployment	**AWS EC2, S3, IAM**	Hosting, file storage, and secure access management
Reverse Proxy	**Nginx**	Routing and load balancing between frontend and backend
Containerization	**Docker, Docker Compose**	Environment consistency and portability
CI/CD	**GitHub Actions, Docker Hub**	Automated build and deployment pipeline
Security & Performance	**Cloudflare**	HTTPS, CDN caching, and DDoS protection
Design & Prototyping	**Figma**	UI/UX design and prototyping

4.6 Site Map

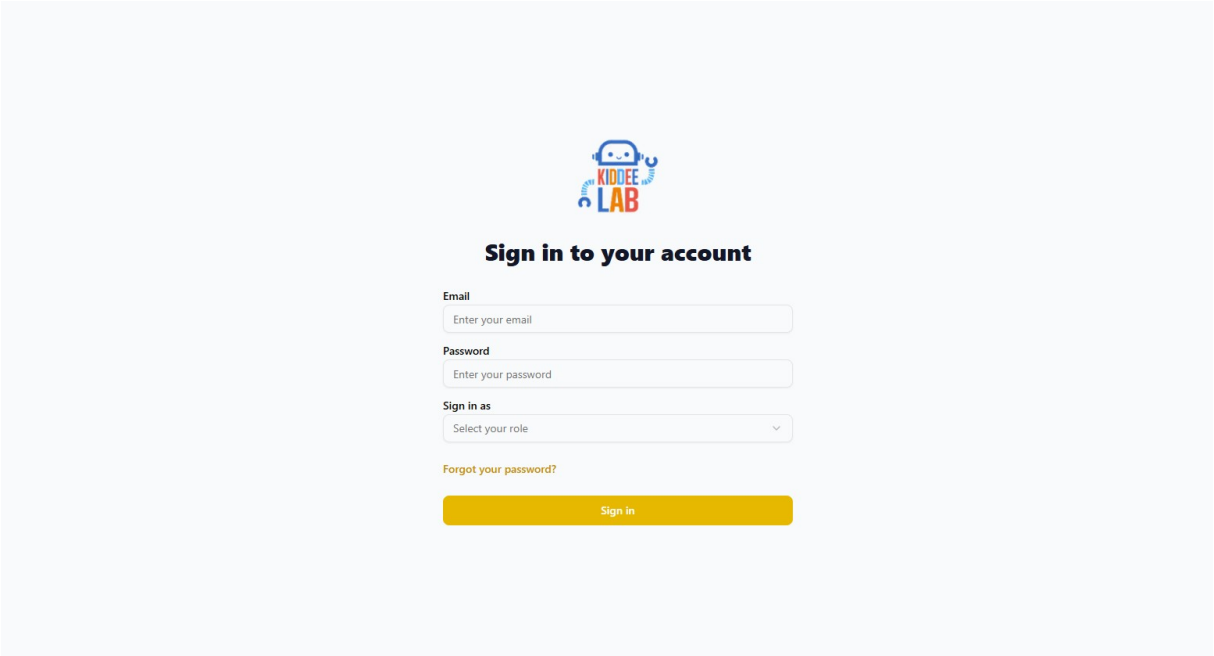


4.5 UX/UI Design

The system includes validation mechanisms to ensure all required fields are completed correctly. If any input is missing or in an incorrect format, the system will display an appropriate error message.

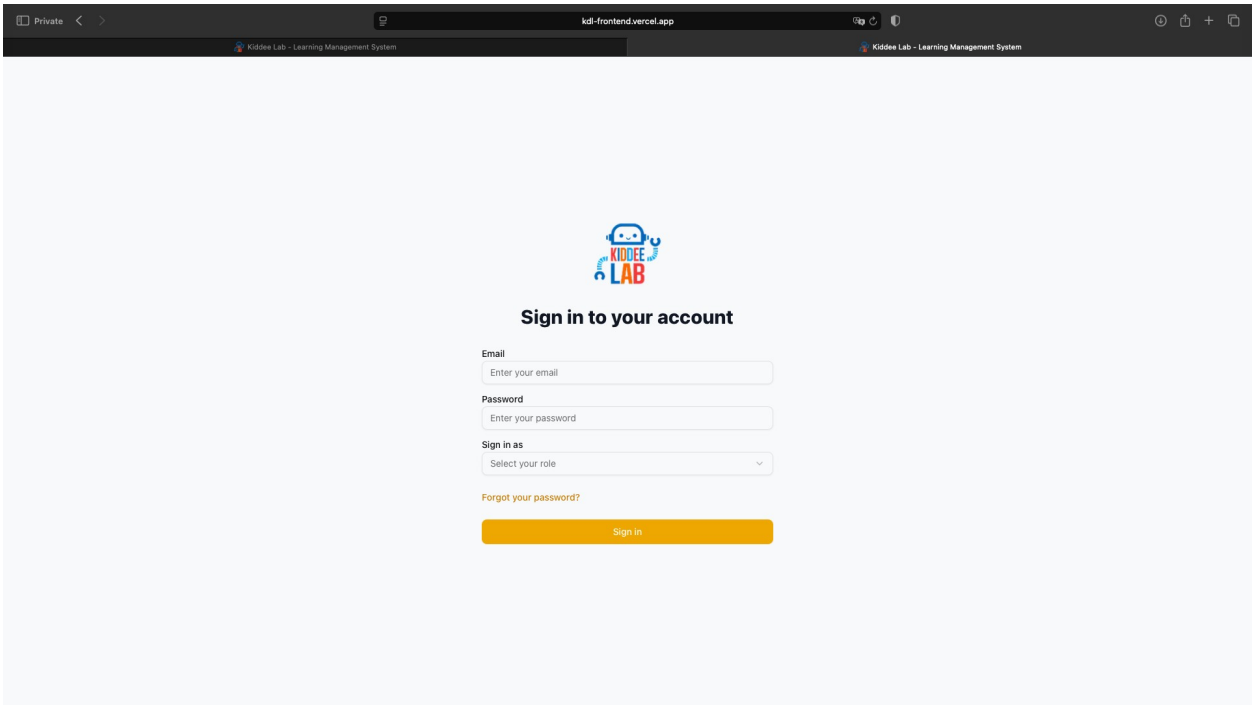
To enhance readability and avoid redundancy, user interfaces that are identical or similar across roles will be presented only once in this document, provided users have access to the same functionality.

4.5.1 Log in Page

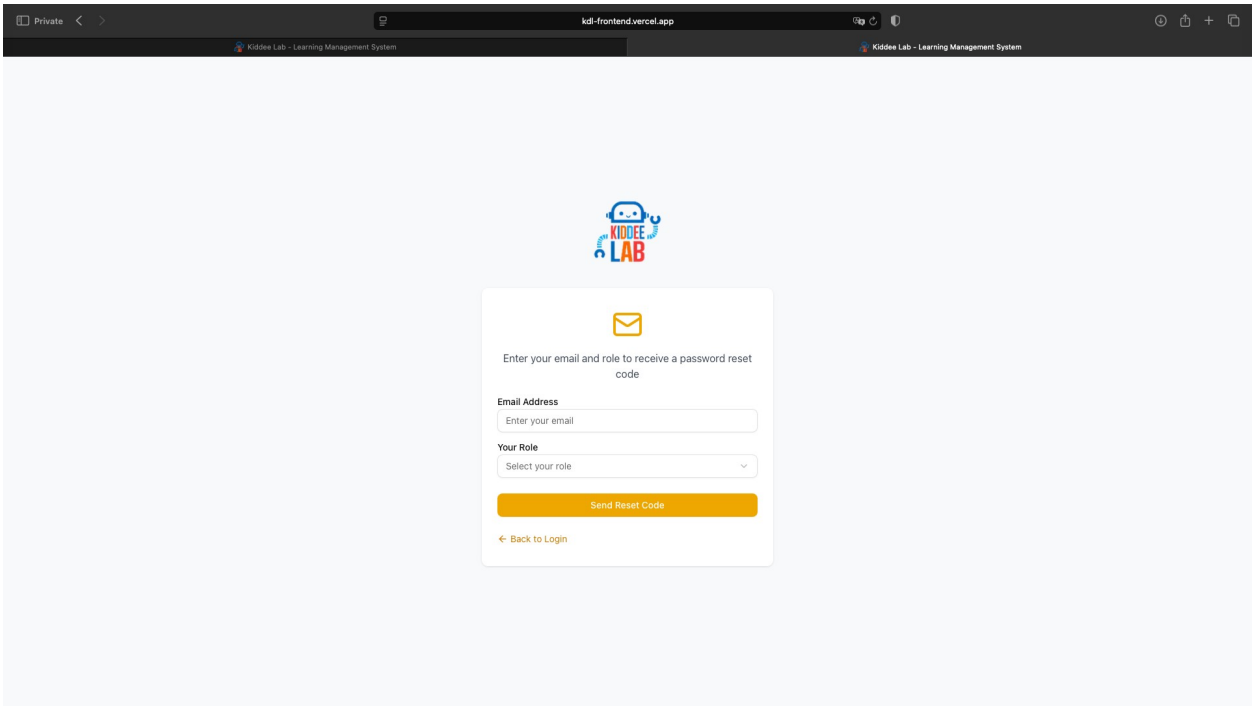


Once users access the system via www.registrar.kiddeelab.co.th, they are required to sign in based on their assigned roles. A "Forgot Password" feature is also available for users who have forgotten or wish to reset their passwords.

4.5.2 Sign in page screen



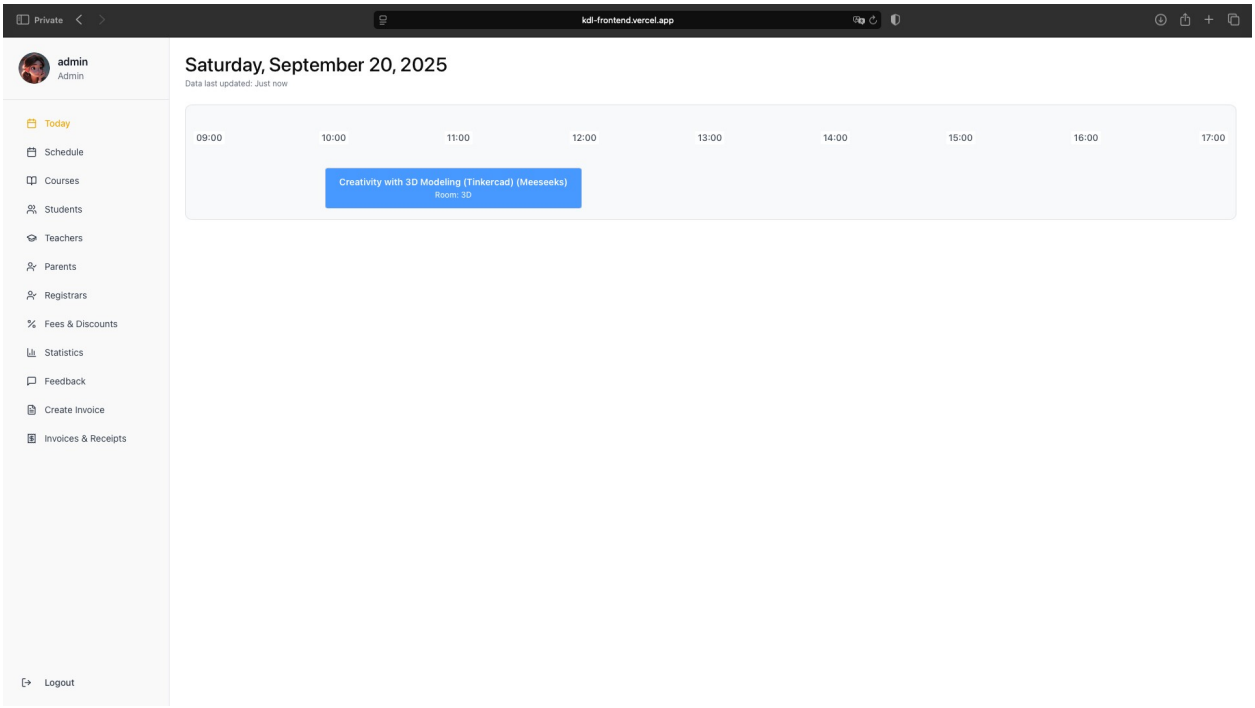
4.5.3 Forgot password screen



4.5.4 Manager UI

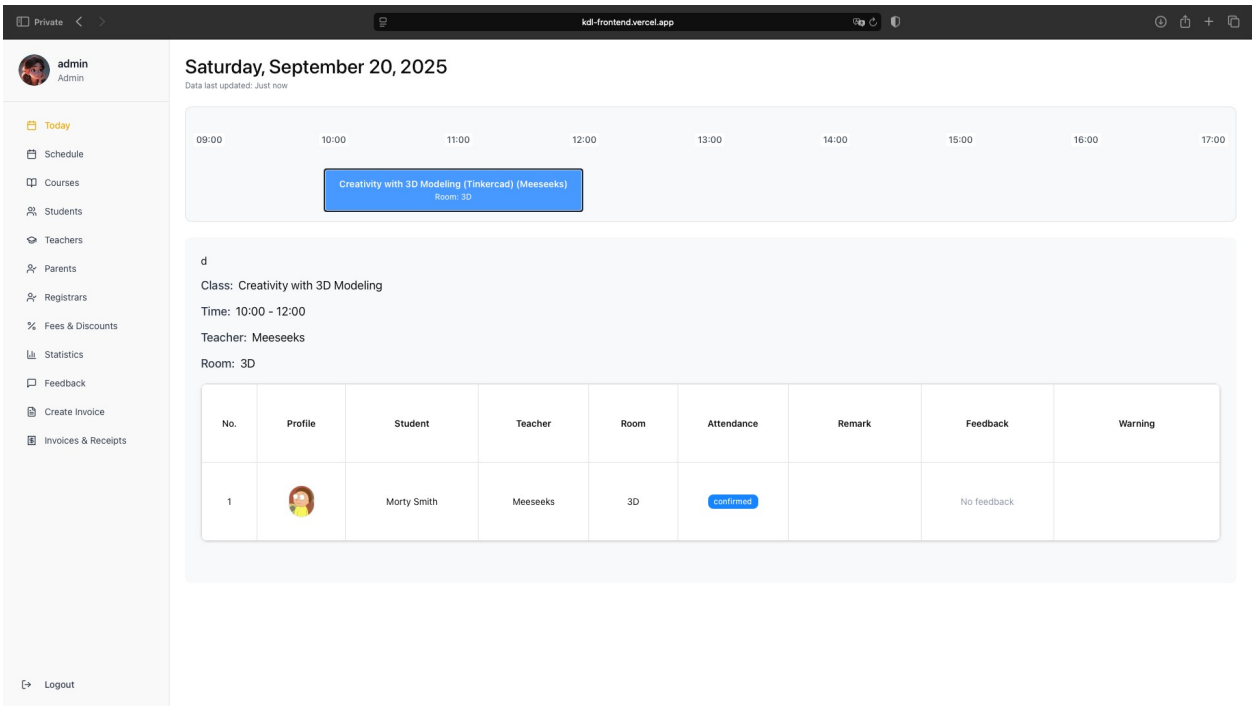
Today Page

This is the default landing page for managers upon signing in to the system.

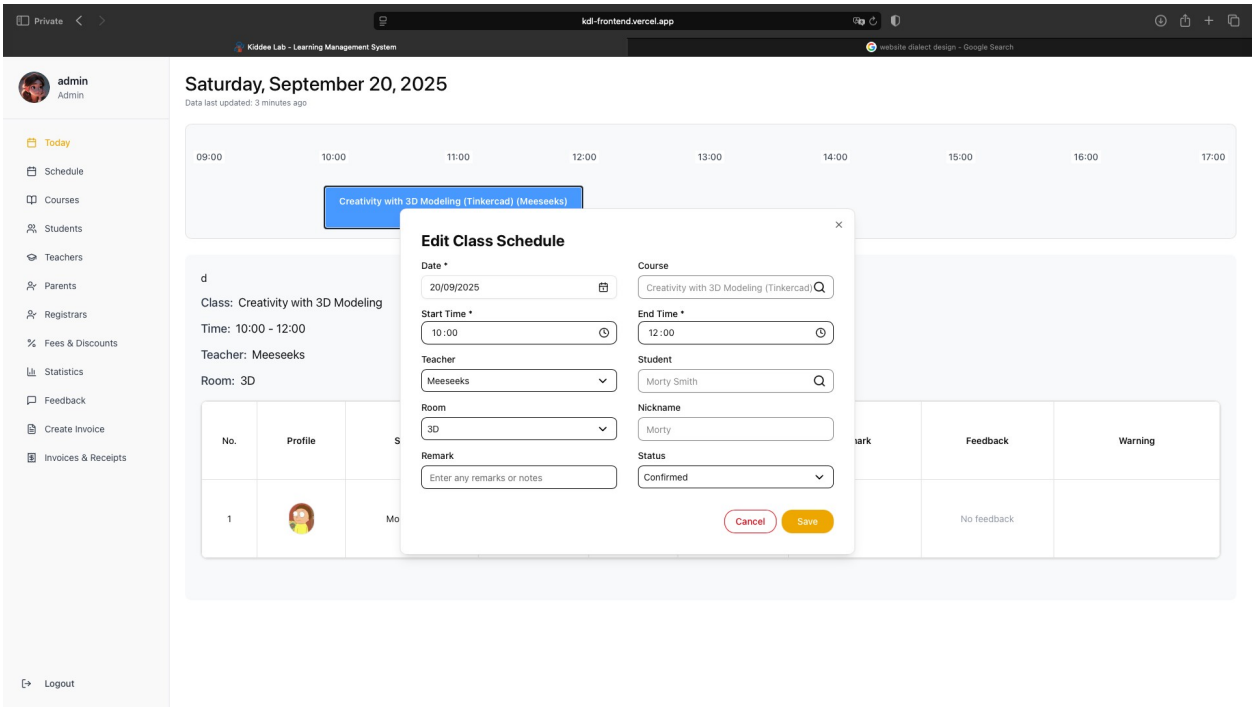


The **Today** feature displays all classes scheduled for the current day. By clicking on a specific class, the manager can view detailed information including the course name, assigned teacher, classroom, time, and the list of students expected to attend. By double-clicking a student's row, the manager can edit the student's details.

Today screen with class details



Editing class schedule in today screen

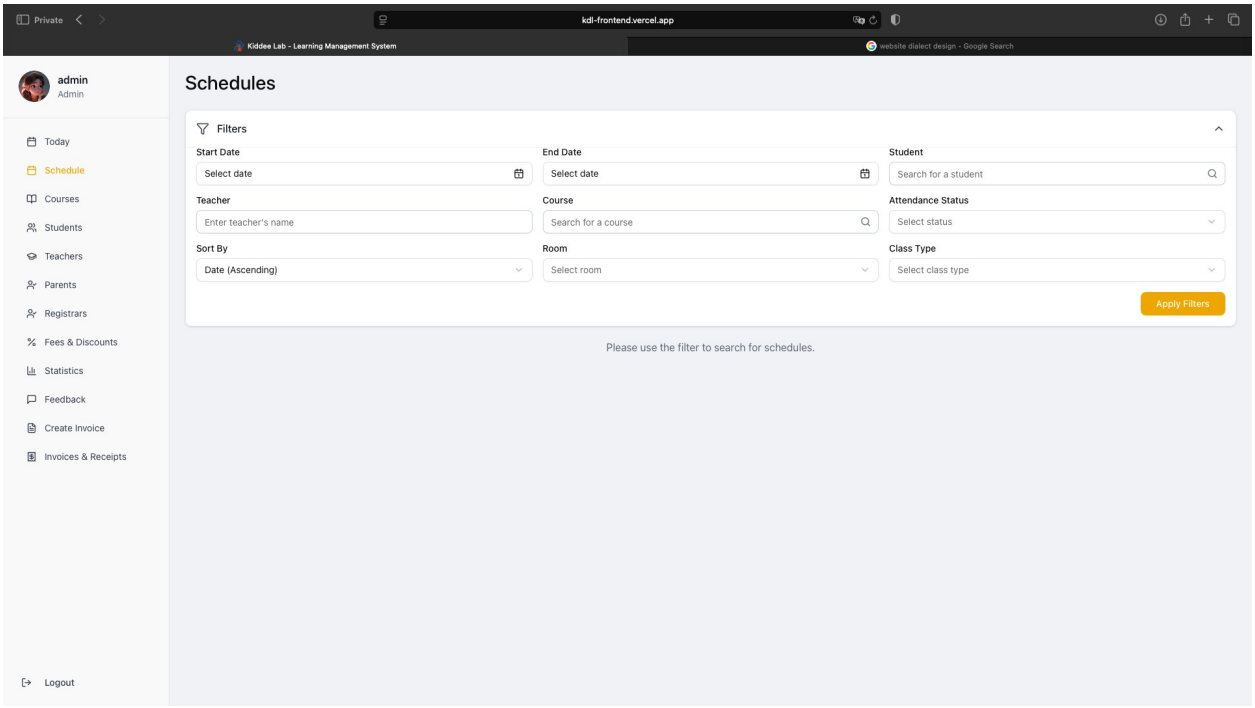


Schedules Page

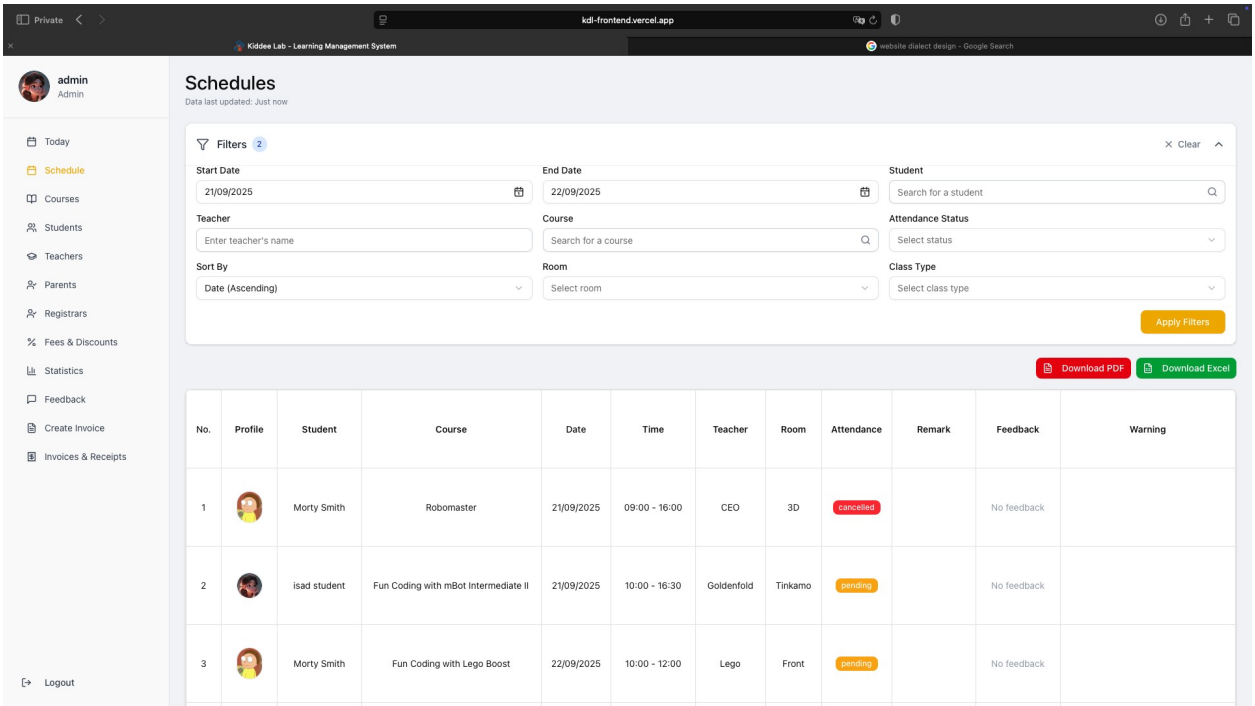
On the **Schedules** page, managers can use the filtering function to search and narrow down student schedules. Available filters include start date, end date, student name, teacher, course, attendance status, sort order, room, and class type.

By double-clicking a schedule row, the manager can access and edit the associated student's details.

Initial Schedules Screen



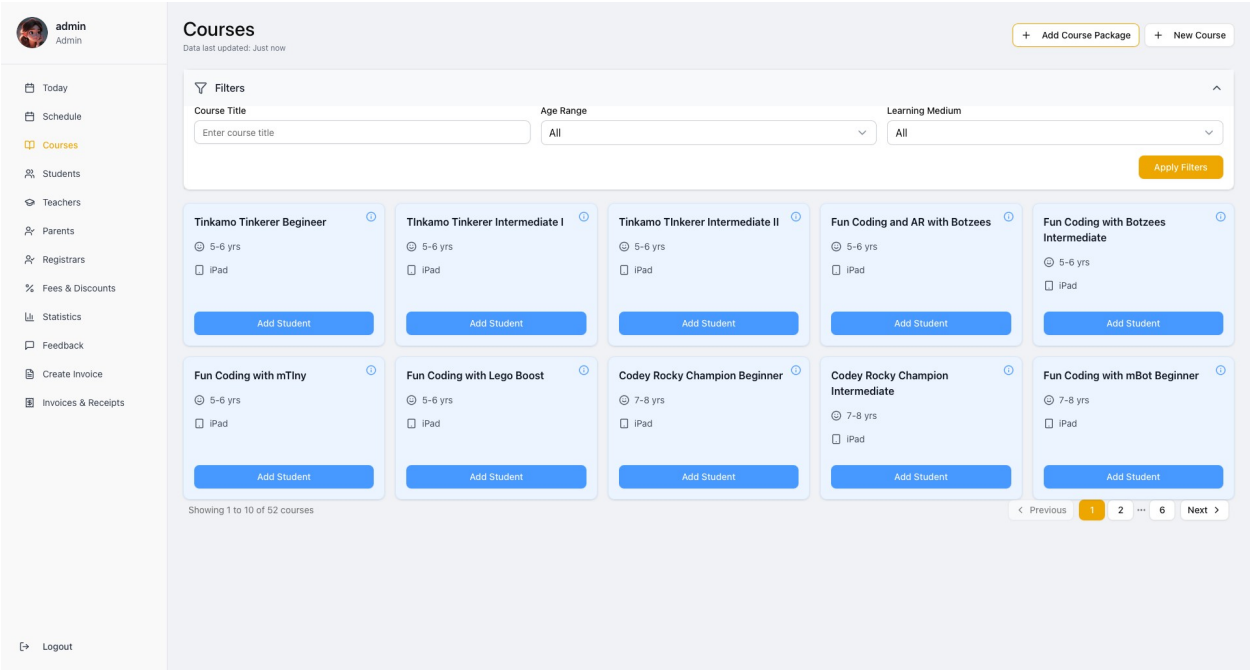
The following is an example of filtering schedules using the start date and end date. After applying the filter, the manager can download the filtered schedule in either PDF or Excel format.



Courses Page

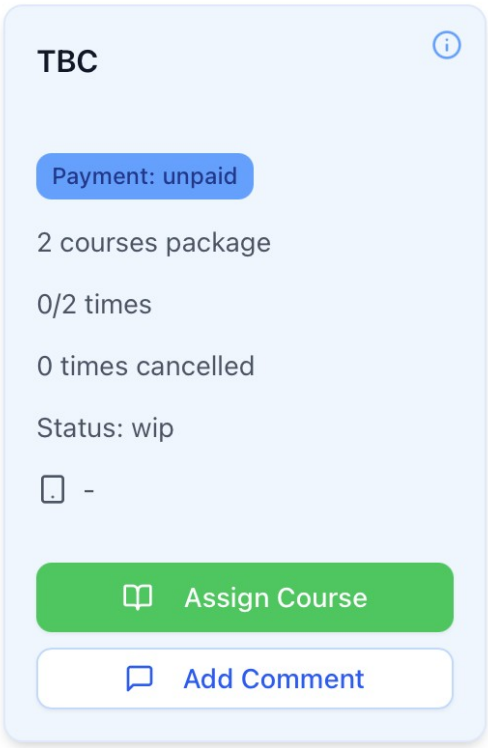
On the **Courses** page, the manager can view all courses currently offered by KDL. New courses can be created by entering details such as the course title, description, recommended age range, and learning medium. Additionally, the manager can directly add students to a course from this page.

Course Screen

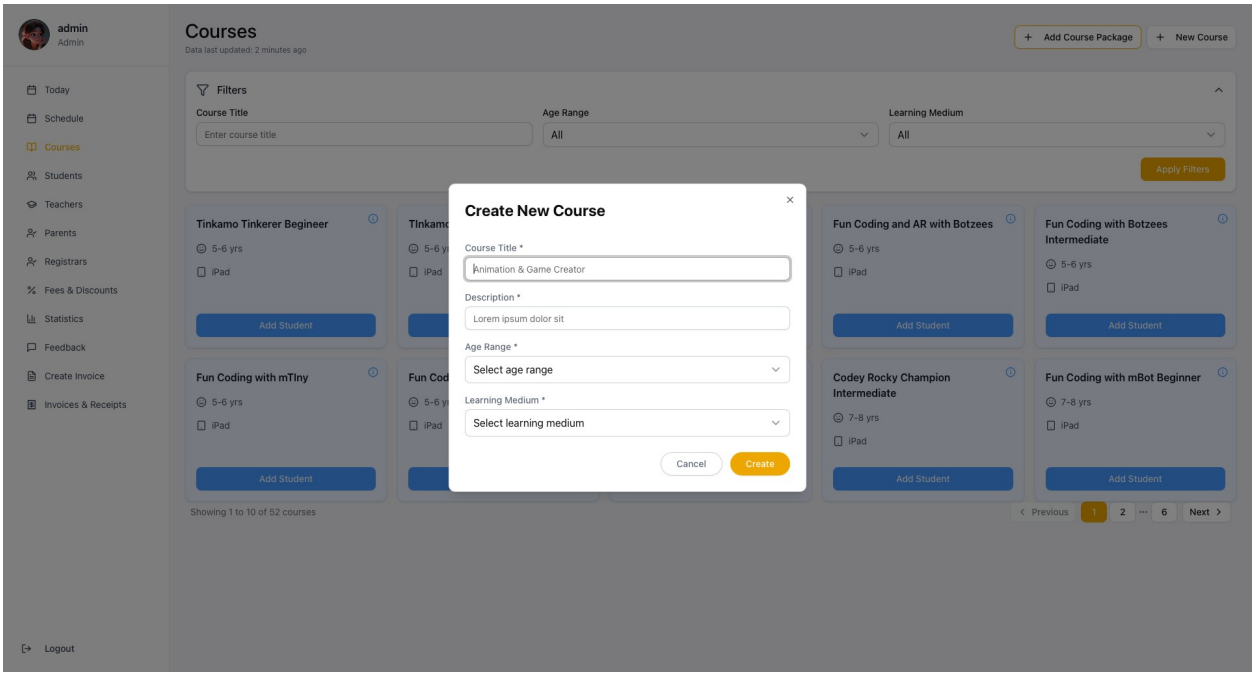


The manager can assign course packages to students. To streamline this process, when searching for a student, the manager can enter the student's name, nickname, or student ID. Once one of these fields is entered, the remaining fields will be automatically populated. After successfully adding a course package, a **TBC (To Be Confirmed)** course will be assigned to the student. This TBC course allows the course title to be specified at a later time. The purpose of the course package is to enable parents to purchase multiple courses in bulk at a discounted price. The procedure for assigning the actual course title is the same as when adding a new course to a student.

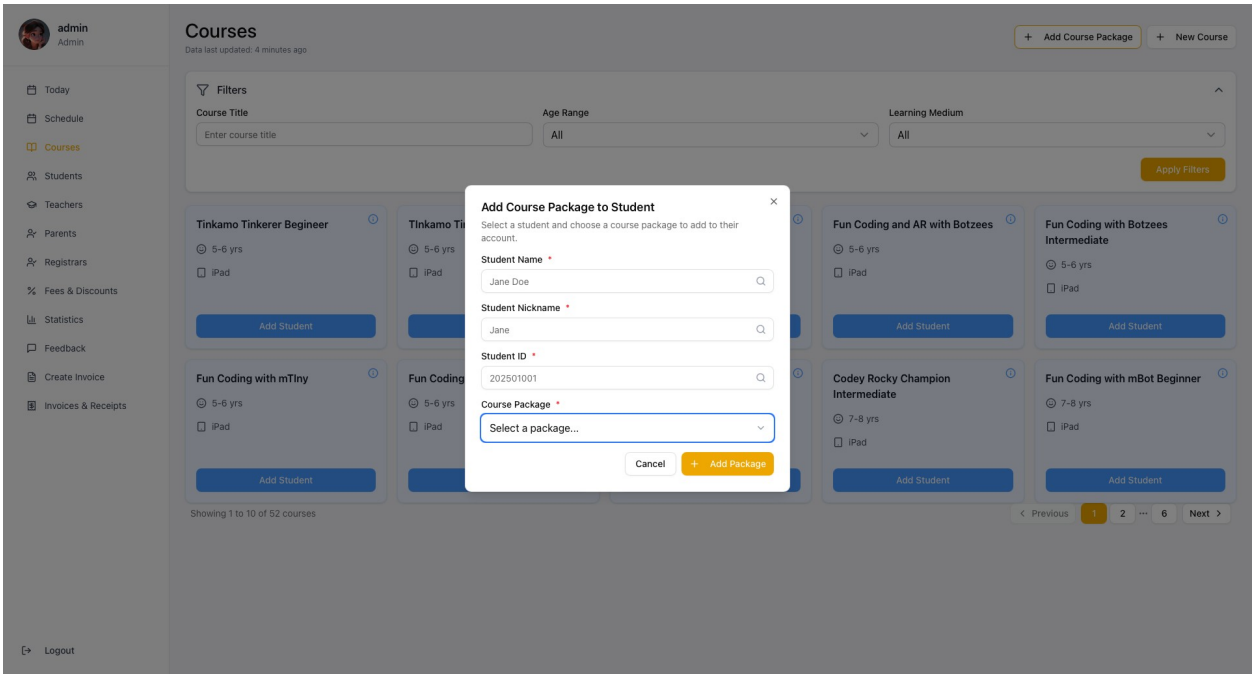
TBC course



Create new course



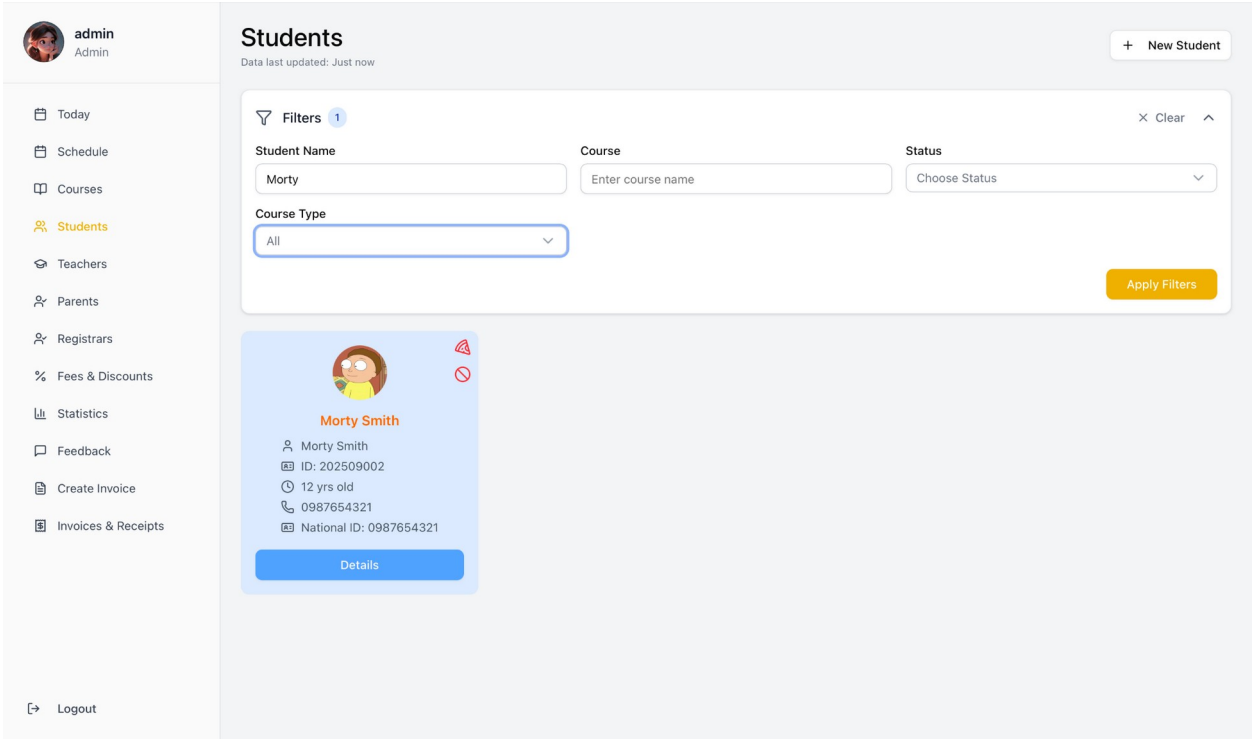
Add Course Package to Student



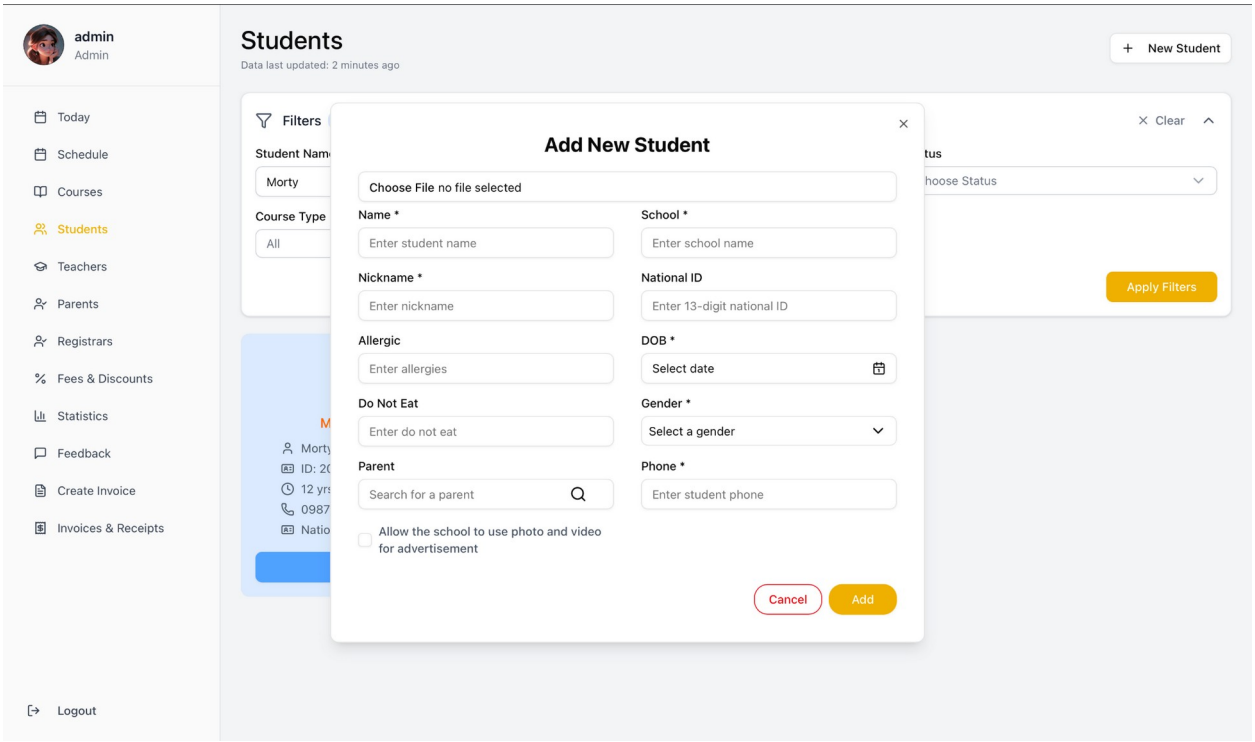
Students Page

On the **Students** page, the manager can view a list of all students. New students can be added by entering details such as photo, name, nickname, allergies, dietary restrictions, school, national ID, date of birth, gender, phone number, parent information, and permission for the use of photos or videos for advertising purposes. Fields marked with an asterisk (*) are required. If the manager enters invalid or incomplete information in these fields, appropriate error messages will be displayed.

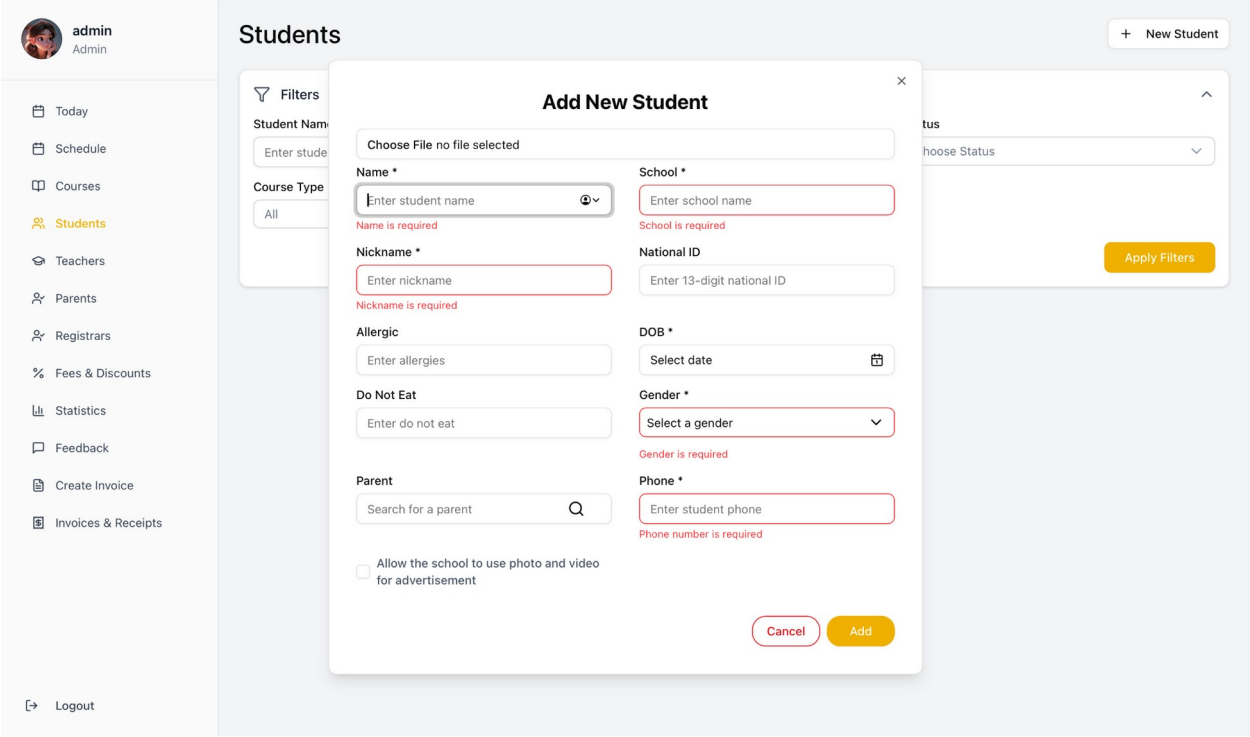
Student page with list of student



Register New Student

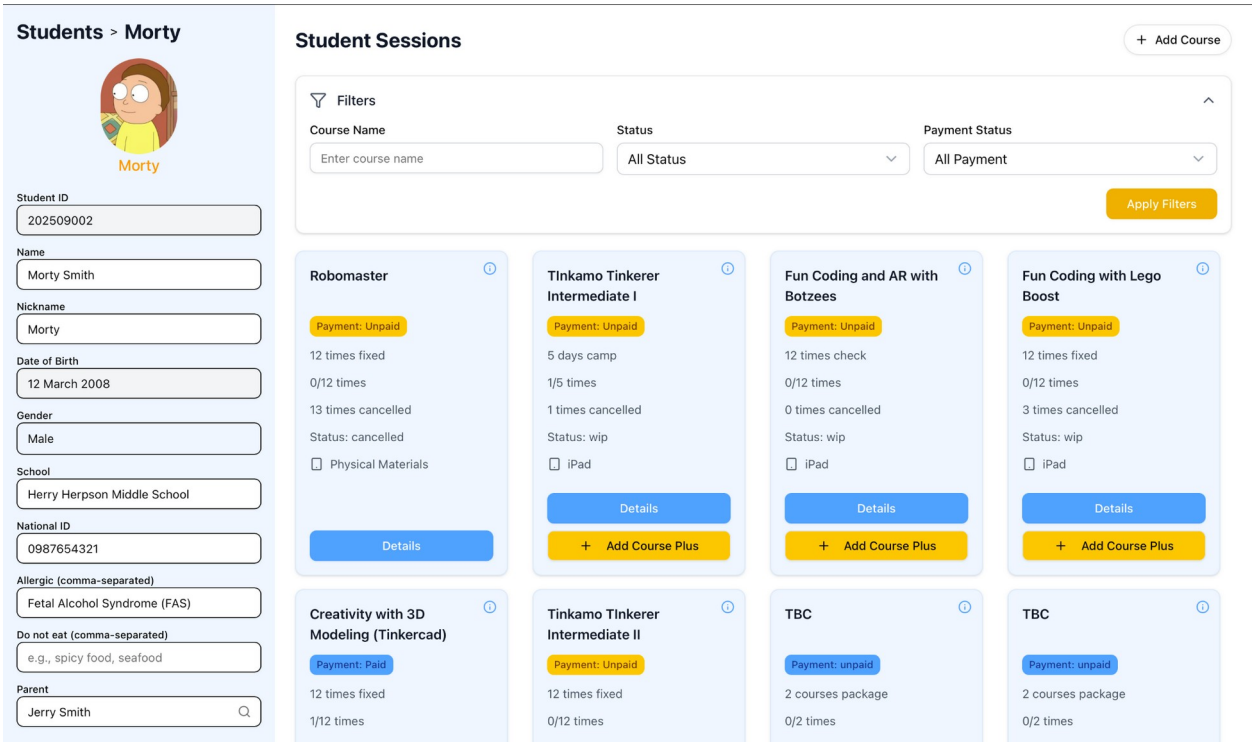


Display error if the required files are not filled



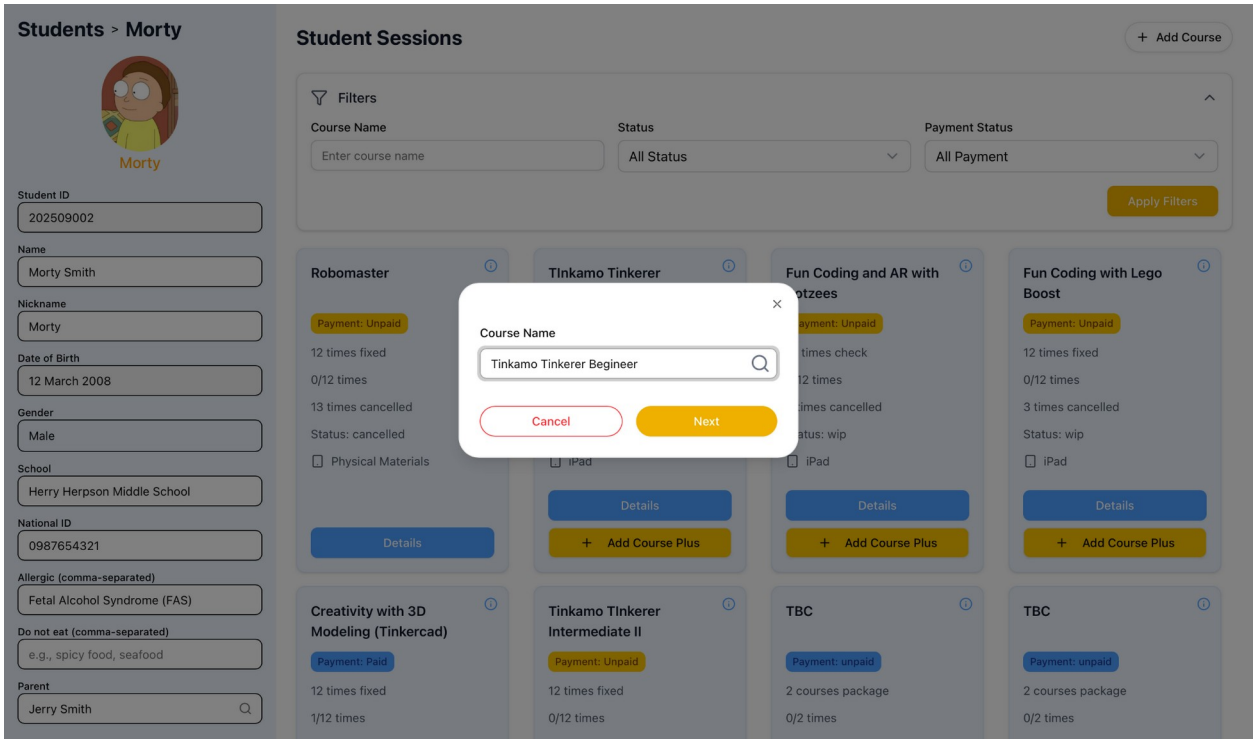
The manager can add courses to a student by clicking the **Add Course** button. By selecting **Details**, the manager can view the student's personal information as well as their course history.

Student personal detail



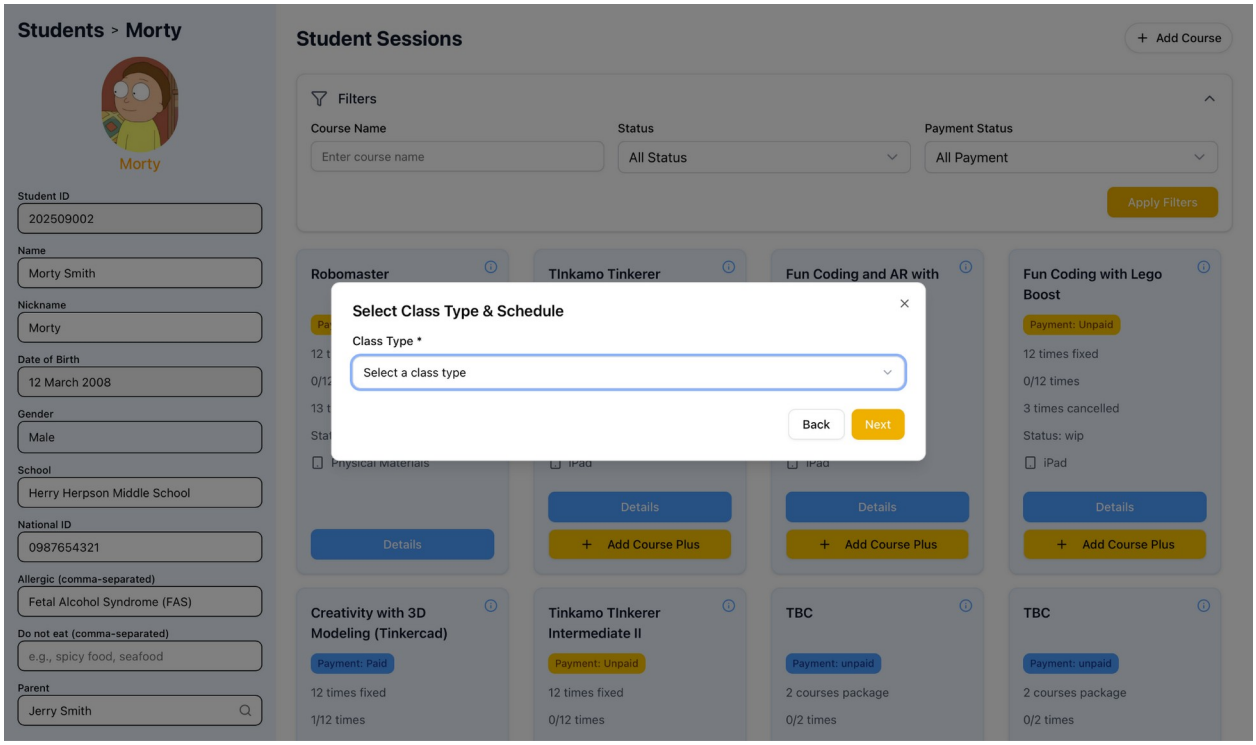
To ensure errors are promptly addressed and users are clearly informed, the system divides the course addition process into smaller, manageable steps. After clicking **Add Course**, the manager must enter a course name that already exists in the system. Once the user types at least three characters, the system will begin suggesting matching course names. The user can only proceed to the next step after entering valid data.

Student enrolls in a course



The next step is to select the class type. To prevent invalid entries, a dropdown list is provided from which the user can choose. The available options include **12 Times Fixed**, **12 Times Check**, **5 Days Camp Class**, and others. The user selects one option and proceeds. Each class type follows its own specific procedure.

Selecting class type

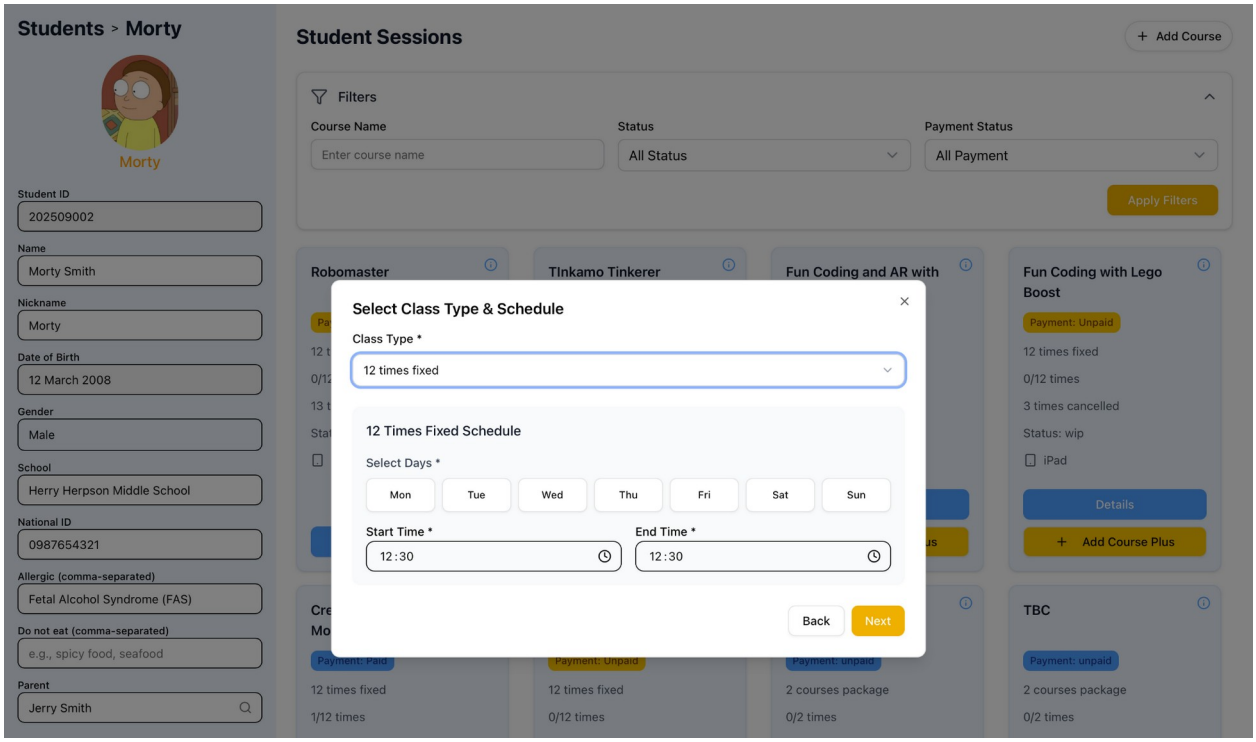


12 Times Fixed

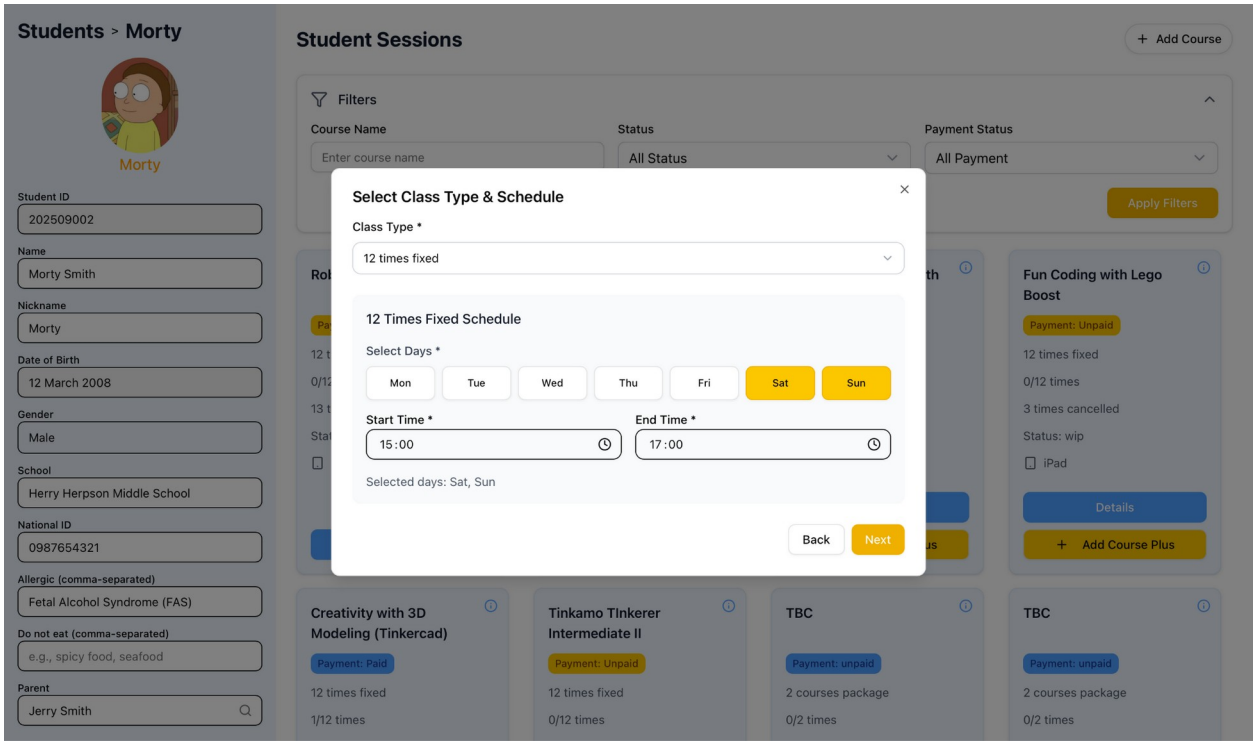
In the **12 Times Fixed** class type, the user can select the specific days and times the student will attend. Afterward, the user chooses the teacher and room. The system includes a mechanism that filters and displays only the teachers qualified to teach the selected subject, preventing the selection of an inappropriate teacher.

For example, a student may be scheduled to attend classes on Saturdays and Sundays from 3 PM to 5 PM with Teacher Meeseeks in the room named "Small."

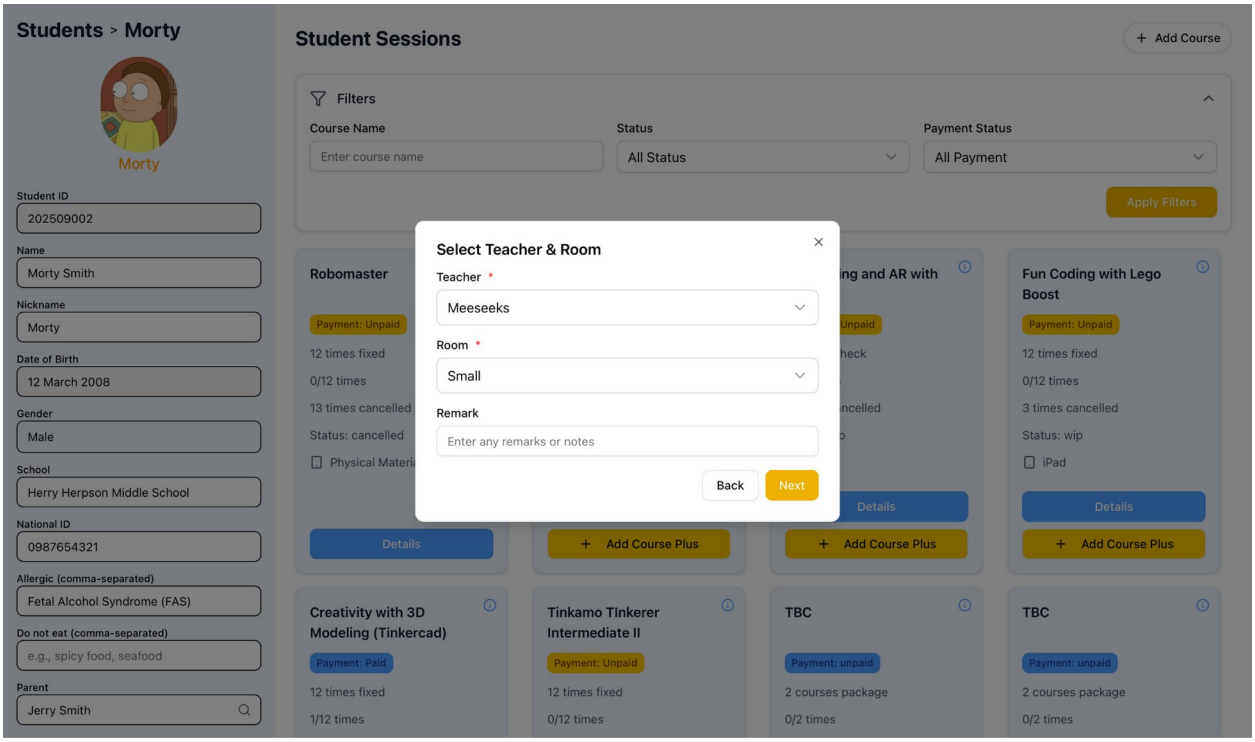
Class type as 12 times fixed screen



Class type as 12 times fixed with example input screen

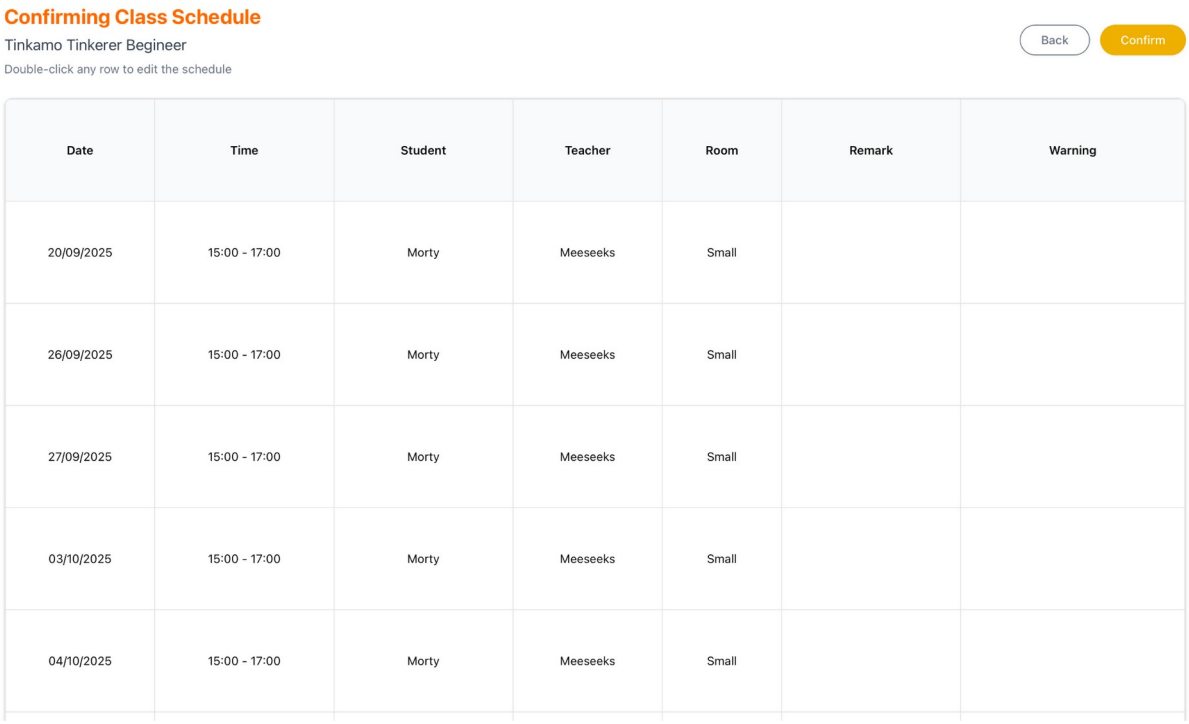


Selecting teacher and room

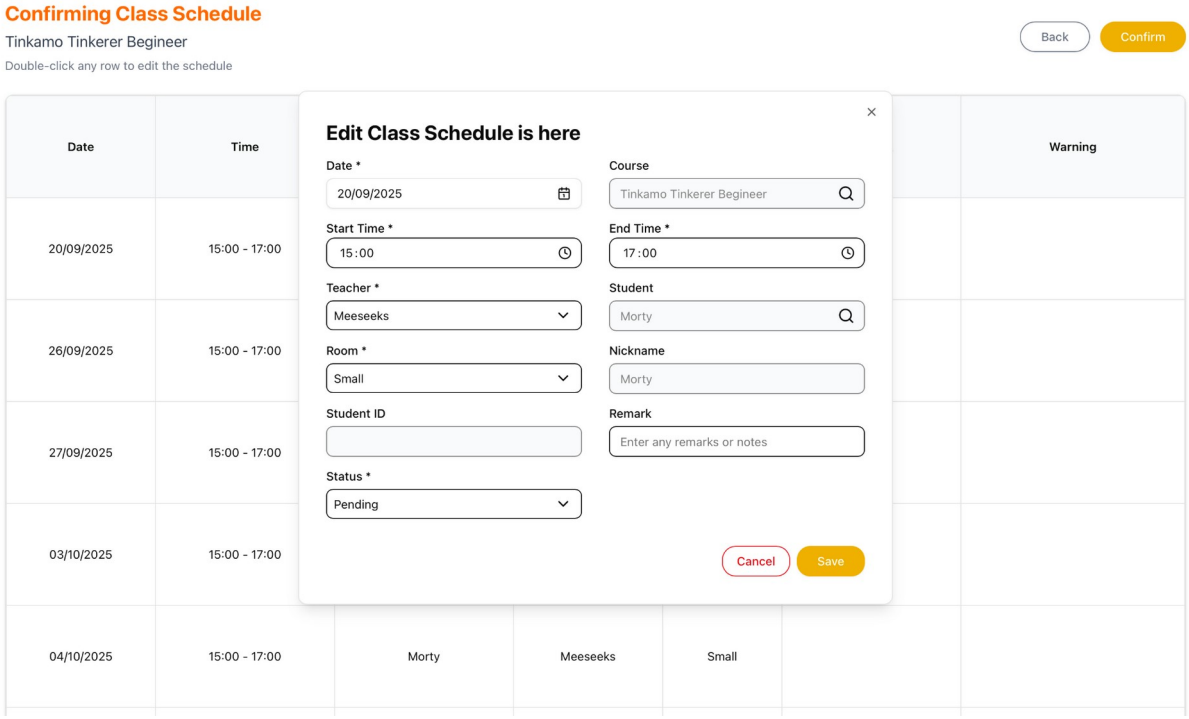


The final step before saving the schedule to the database is **Confirming the Class Schedule**. At this stage, the system allows the user to verify whether there are any scheduling conflicts. The schedule is generated based on the previously entered information. If a conflict is detected, the reason will be displayed in a warning column. The user can double-click any row to edit the details. To save the schedule and add the course to the student's record, the user must click confirm.

Confirm Schedule Screen



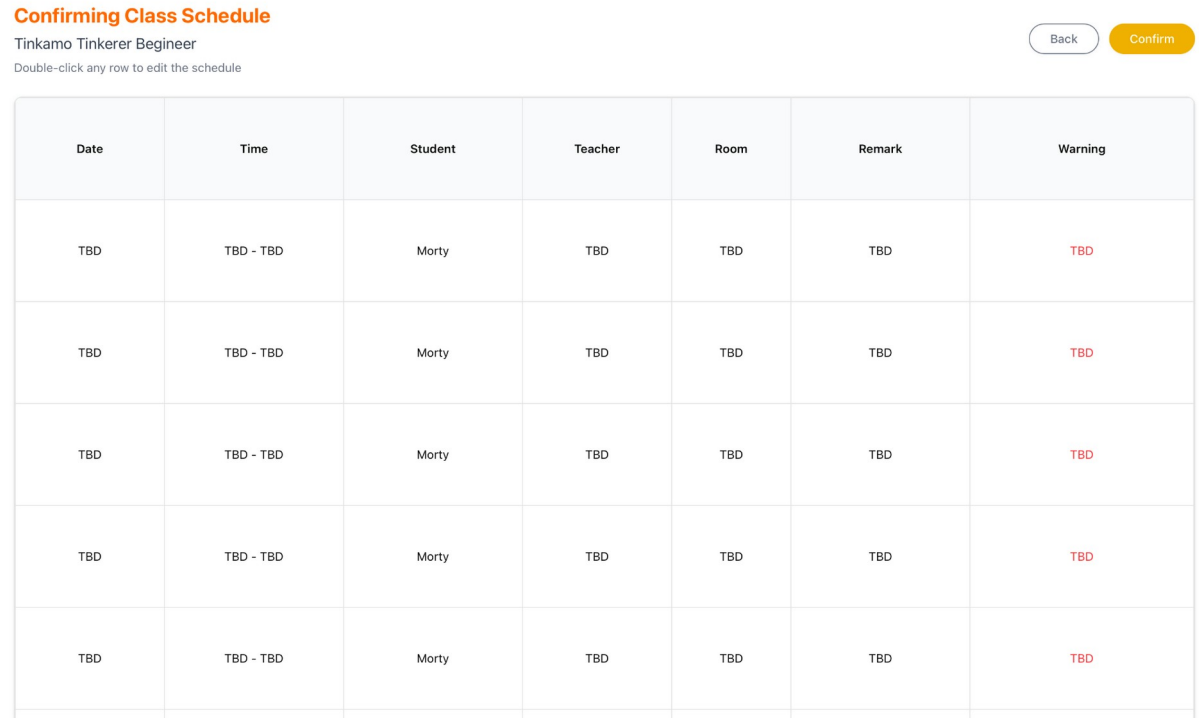
Editing each schedule in Confirming Schedule Screen



12 Times Check

In the **12 Times Check** class type, the user does not need to specify the day, time, teacher, or room in advance. Instead, the student informs KDL of their availability on an ongoing basis, allowing KDL to assign an available teacher and room accordingly.

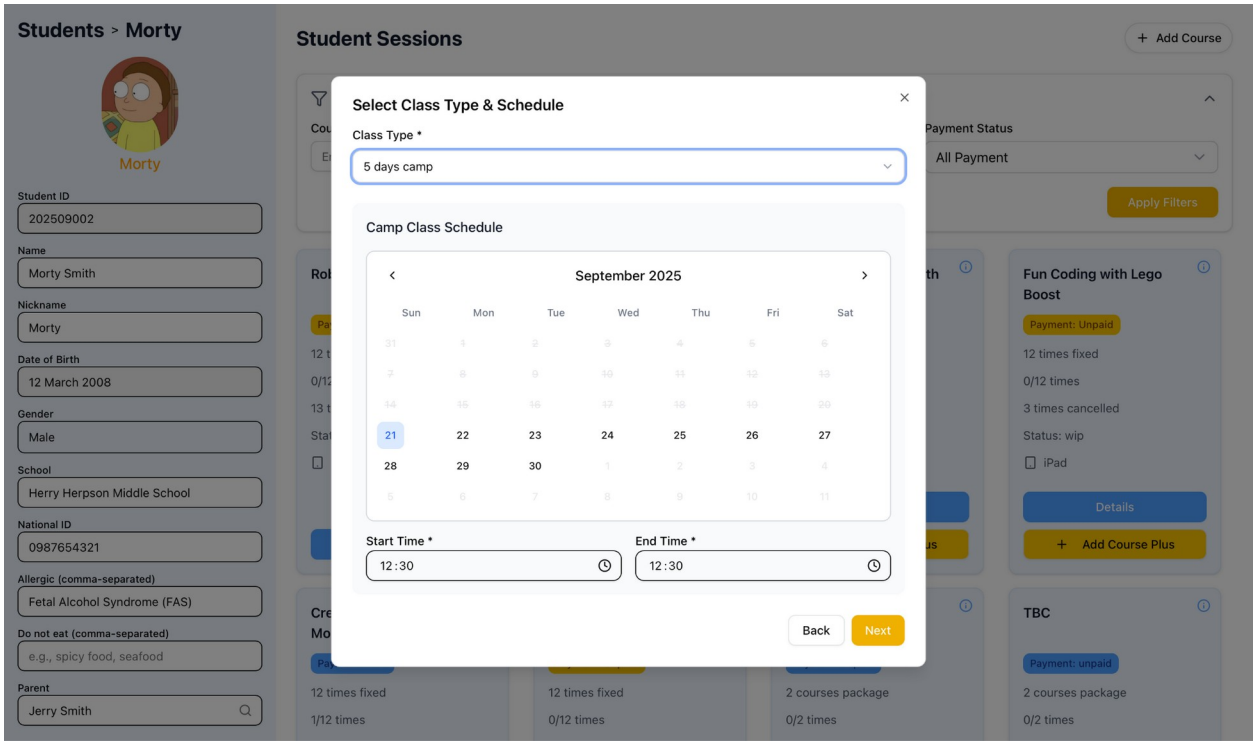
Confirming Schedule for 12 times check as class type



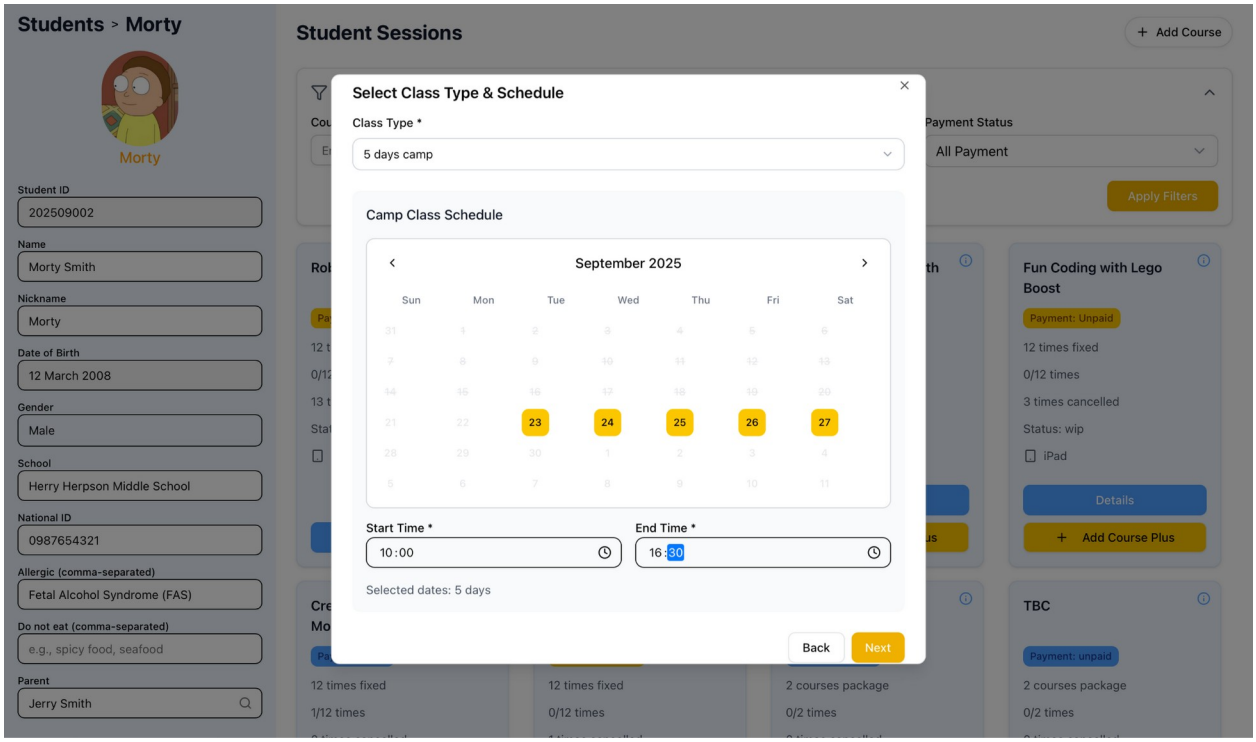
Camp class

In the **Camp Class** type, the user selects the dates and times the student will attend the class. After entering the necessary information, the user can confirm the schedule following the same procedure as in the **12 Times Fixed** class type.

Selecting Camp class as class type screen



Selecting Camp class as class type with example input screen



Confirming schedule with class conflict message screen

Confirming Class Schedule

Tinkamo Tinkerer Begineer

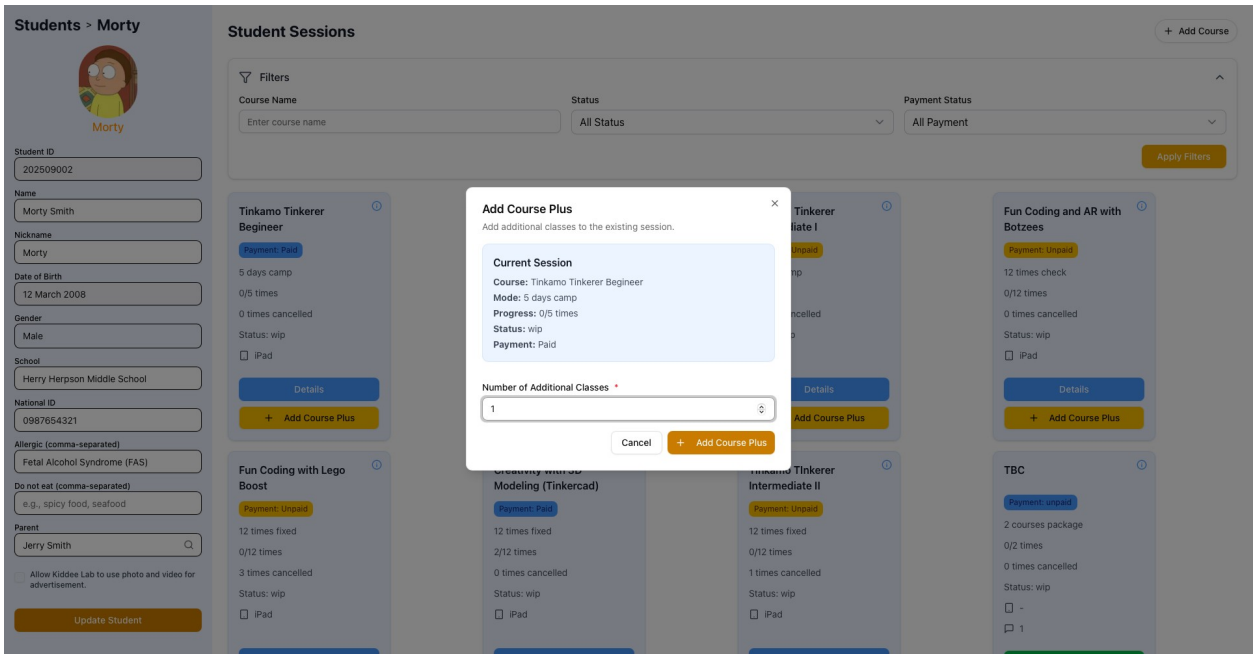
Double-click any row to edit the schedule

Back Confirm

Date	Time	Student	Teacher	Room	Remark	Warning
23/09/2025	10:00 - 16:30	Morty	Meeseeks	Small		Teacher Meeseeks is not available. Teacher Meeseeks is teaching Tinkamo TInkerer Intermediate II at 09:00 - 12:00.
24/09/2025	10:00 - 16:30	Morty	Meeseeks	Small		Student Morty Smith is not available. Student Morty Smith is learning Tinkamo TInkerer Intermediate II at 11:00 - 14:00.
25/09/2025	10:00 - 16:30	Morty	Meeseeks	Small		Teacher Meeseeks is not available. Student Morty Smith is learning Tinkamo TInkerer Intermediate II at 11:00 - 14:00.
26/09/2025	10:00 - 16:30	Morty	Meeseeks	Small		Teacher Meeseeks is not available. Student Morty Smith is learning Tinkamo TInkerer Intermediate II at 11:00 - 14:00.
27/09/2025	10:00 - 16:30	Morty	Meeseeks	Small		Teacher Meeseeks is not available. Student Morty Smith is learning Creativity with 3D Modeling (Tinkercad) at 10:00 - 12:00.

If a teacher determines that a student requires additional time to complete the course due to a slower learning pace, Kiddee Lab (KDL) may recommend that the parents purchase additional class sessions to accommodate the student's learning needs.

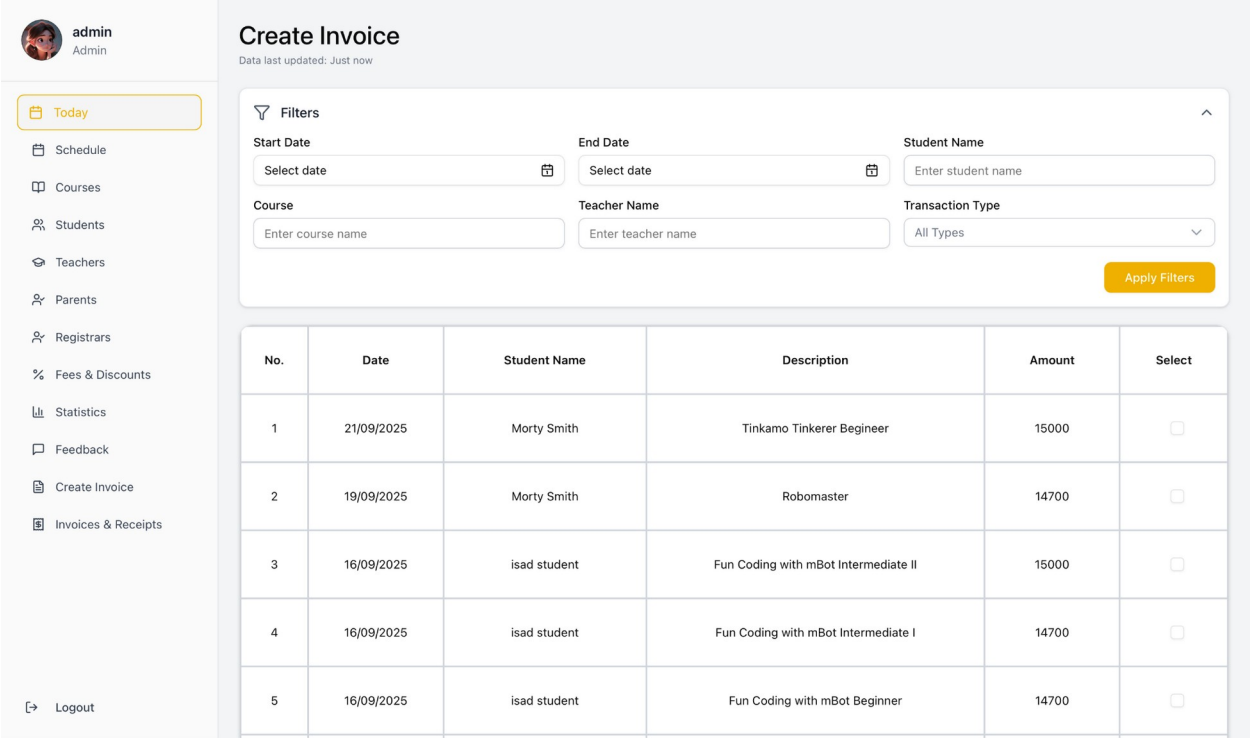
Buying additional class for a subject



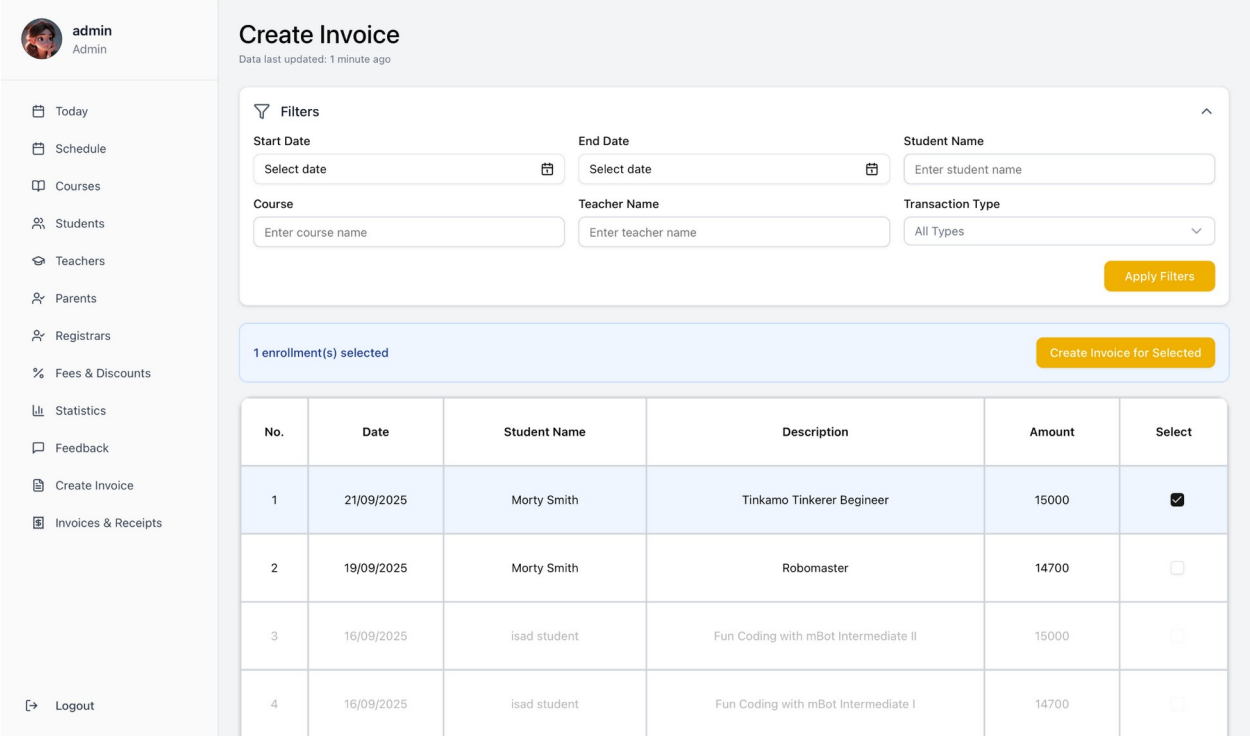
Create Invoice Page

After confirming the schedule when adding a course or package to a student, the corresponding course data for that student will be displayed on the **Create Invoice** page. To generate an invoice, the user must select the rows they wish to include. To prevent the creation of invoices containing multiple students, the system restricts selection to entries belonging to the same student only.

Create Invoice page with list of enrolled course screen



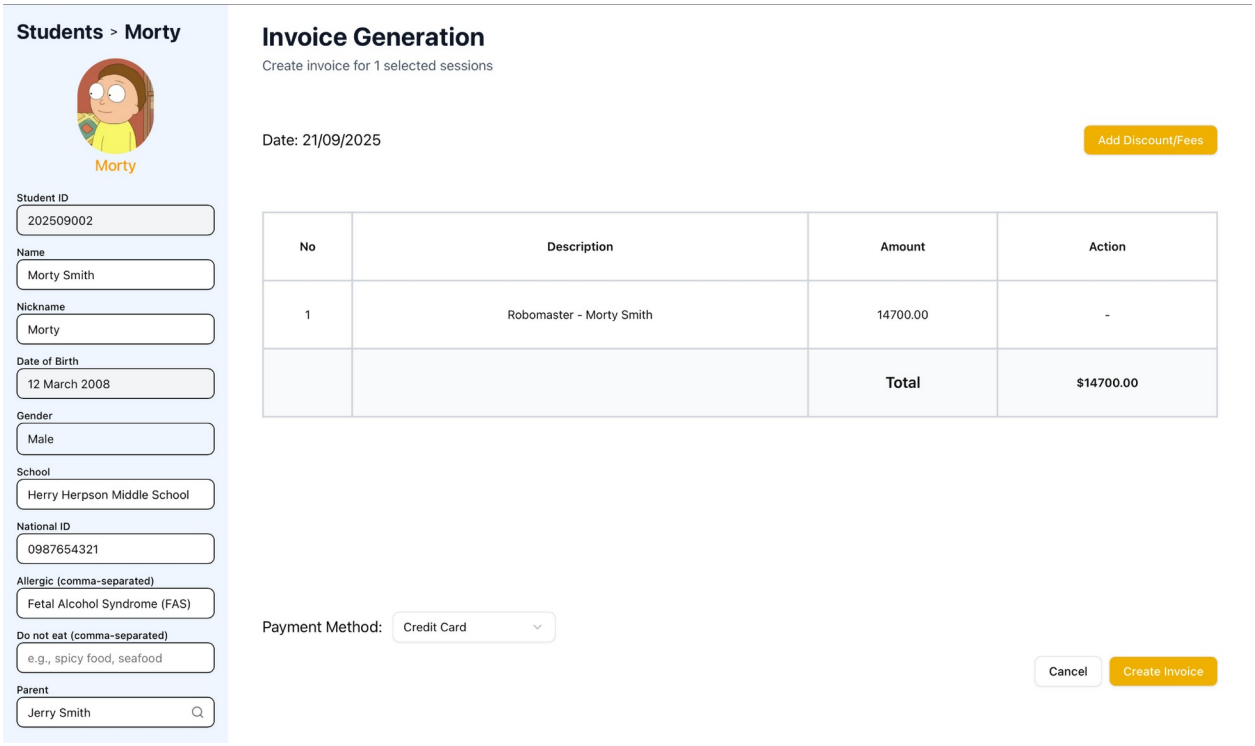
Create invoice for one student screen



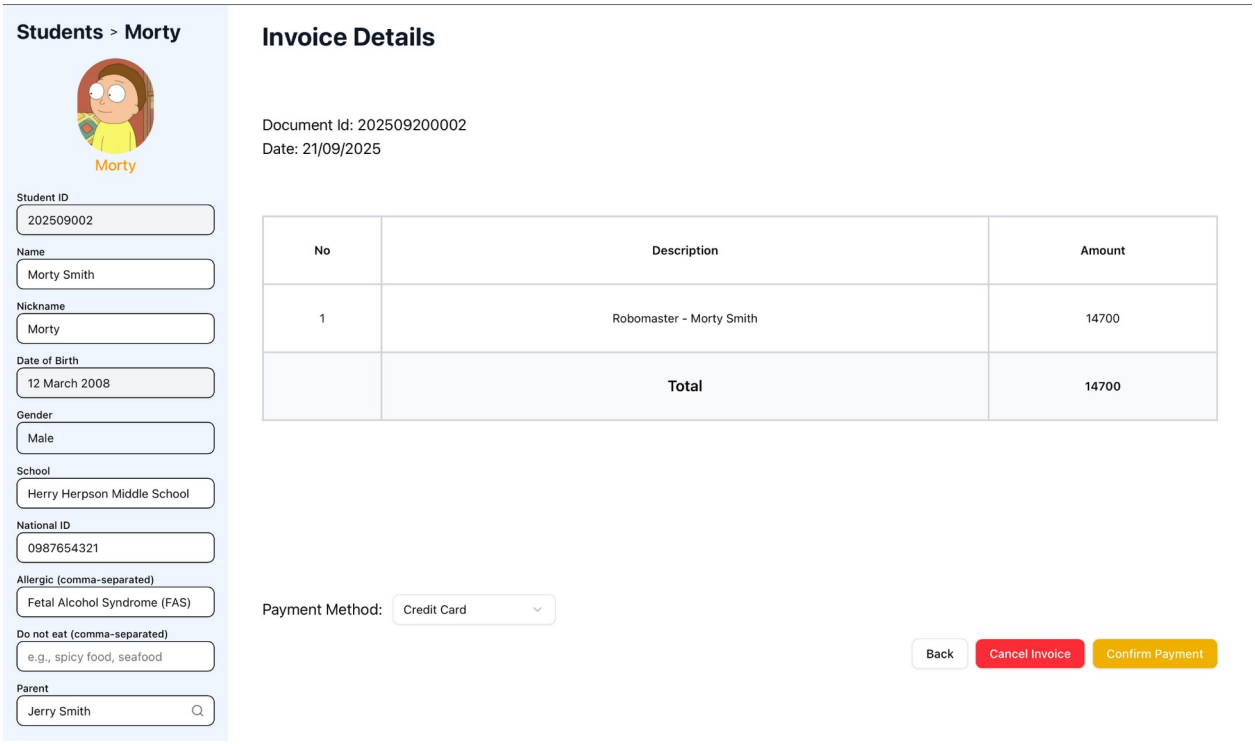
After creating an invoice, the user is directed to the **Invoice Generation** page, where they can add management fees or discounts and select a payment method. Once the **Create Invoice** button is clicked, the invoice is generated. The user must then manually verify whether the payment has been received, as the system does not support automated payment processing.

Upon successful payment confirmation, a receipt is generated, which can be downloaded and printed. If the payment is unsuccessful, the created invoice remains accessible on the **Invoices & Receipts** page for further action.

Invoice generation screen



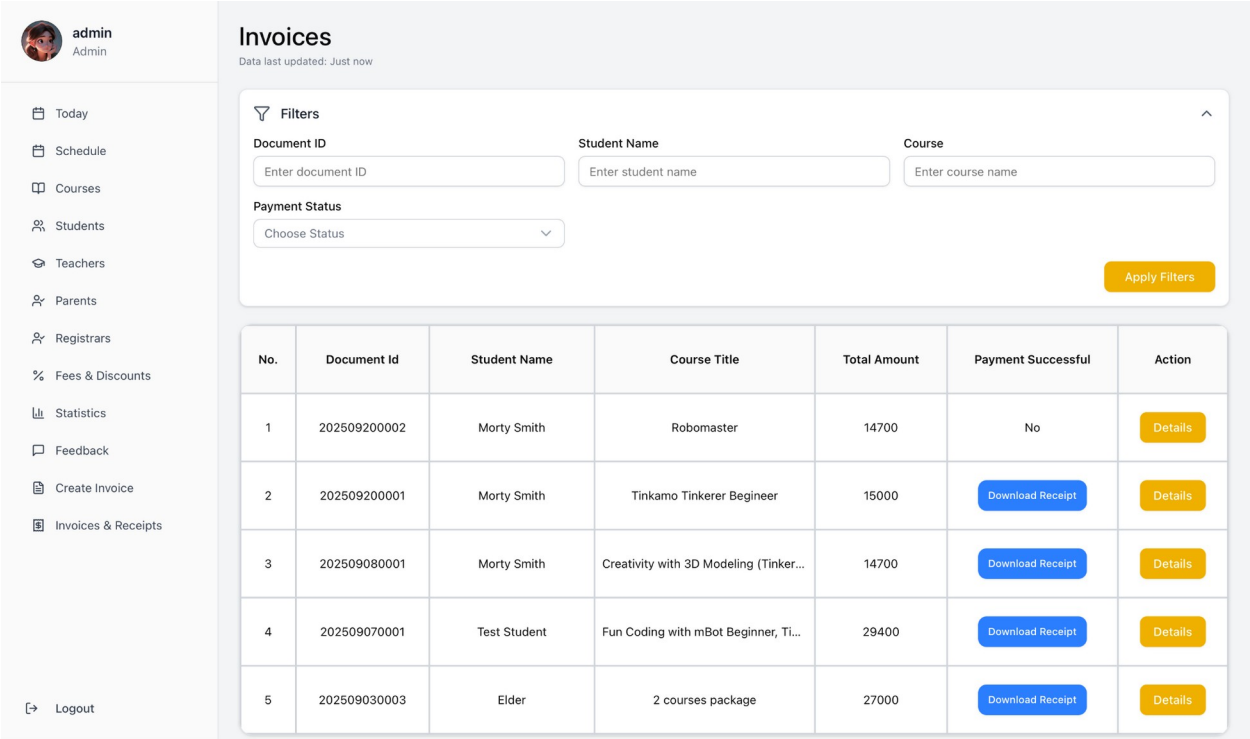
Confirming successful payment screen



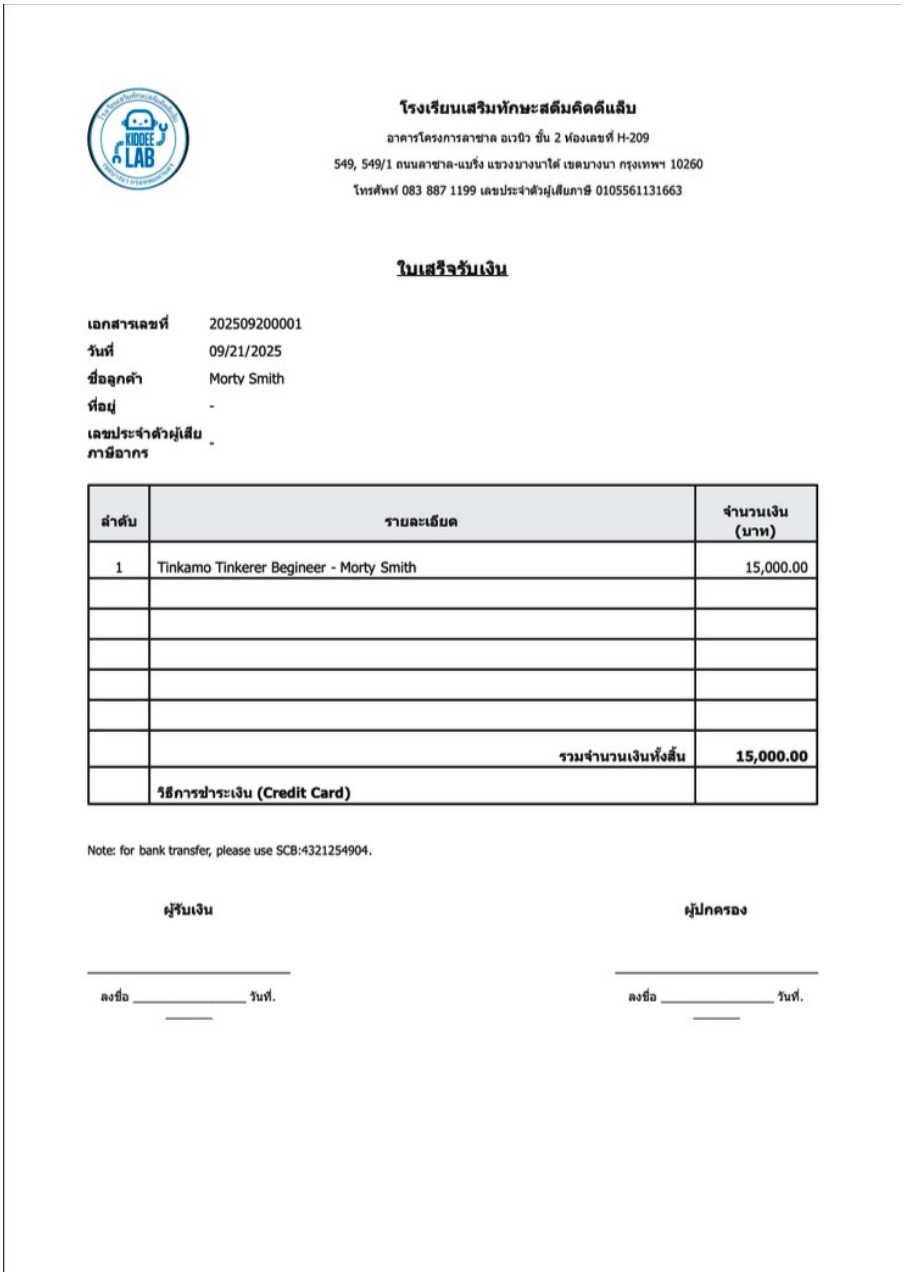
Invoices & Receipts Page

On the **Invoices & Receipts** page, users can view all receipts as well as outstanding invoices that have not yet been paid.

Invoices & Receipts screen



Generated Receipt



โรงเรียนเสริมทักษะสตีมคิดดีแล็บ
อาคารโครงการลาซาล อเวนิว ชั้น 2 ห้องเลขที่ H-209
549, 549/1 ถนนลาซาล-แบริ่ง แขวงบางนาใต้ เขตบางนา กรุงเทพฯ 10260
โทรศัพท์ 083 887 1199 เลขประจำตัวผู้เสียภาษี 0105561131663

ใบเสร็จรับเงิน

เอกสารเลขที่ 202509200001
วันที่ 09/21/2025
ชื่อลูกค้า Morty Smith
ที่อยู่ -
เลขประจำตัวผู้เสียภาษีอากร -

ลำดับ	รายละเอียด	จำนวนเงิน (บาท)
1	Tinkamo Tinkerer Begineer - Morty Smith	15,000.00
	รวมจำนวนเงินทั้งสิ้น	**15,000.00**
	วิธีการชำระเงิน (Credit Card)	

Note: for bank transfer, please use SCB:4321254904.

ผู้รับเงิน

ลงชื่อ _____ วันที่. _____

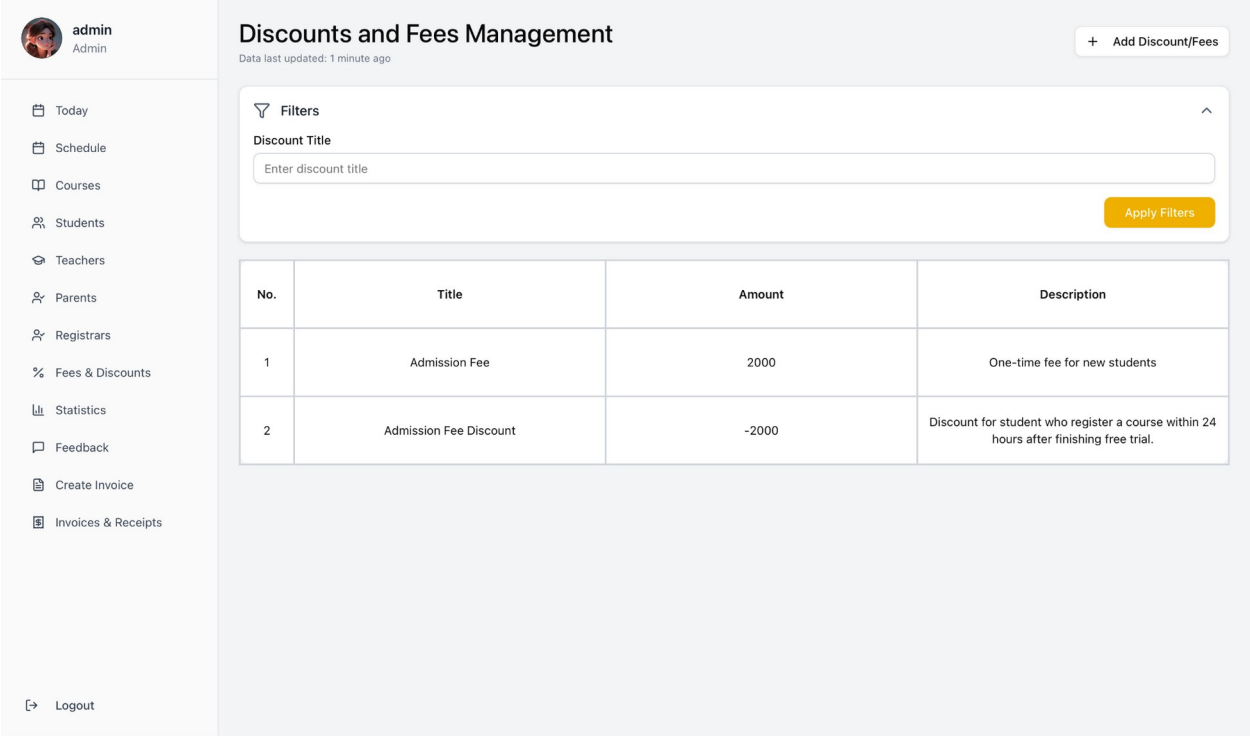
ผู้ปกครอง

ลงชื่อ _____ วันที่. _____

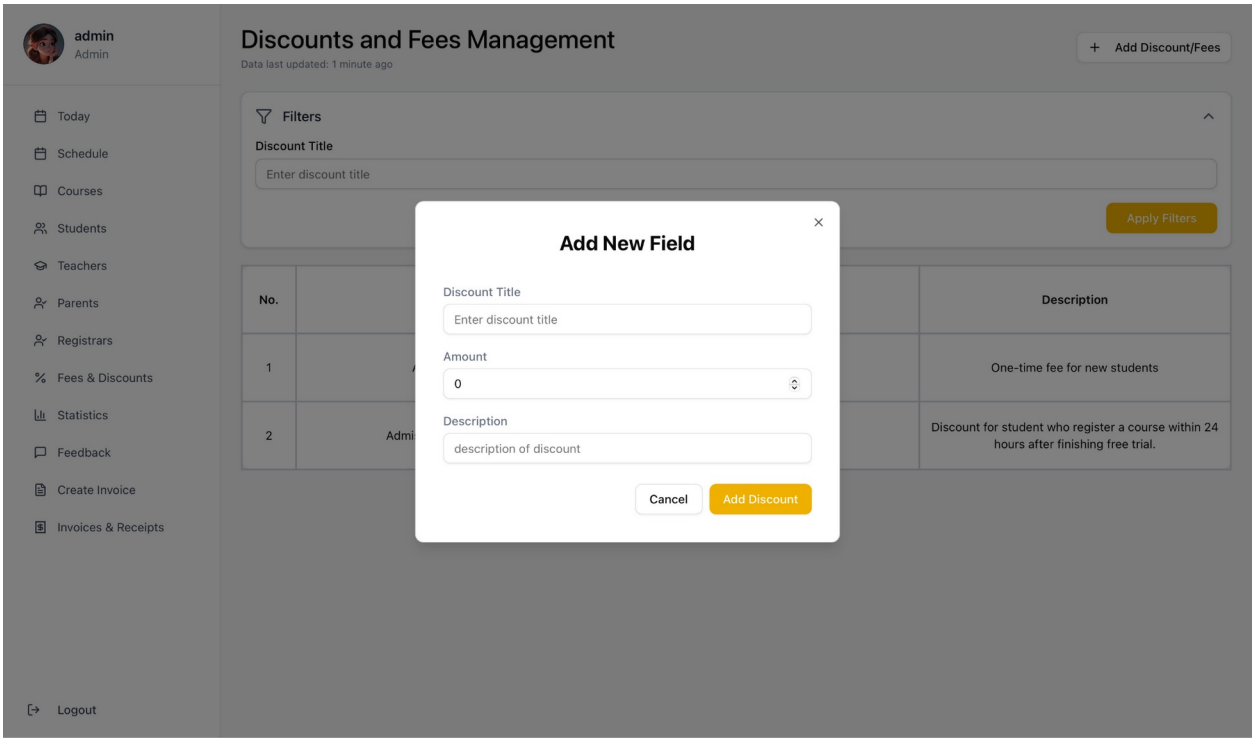
Fees & Discounts Page

On the **Fees & Discounts** page, the manager can view a list of all existing fees and discounts. New entries can be created by clicking the **Add Discount/Fees** button and completing the required information.

Fees & Discount Page Screen



Creating new fee/discount

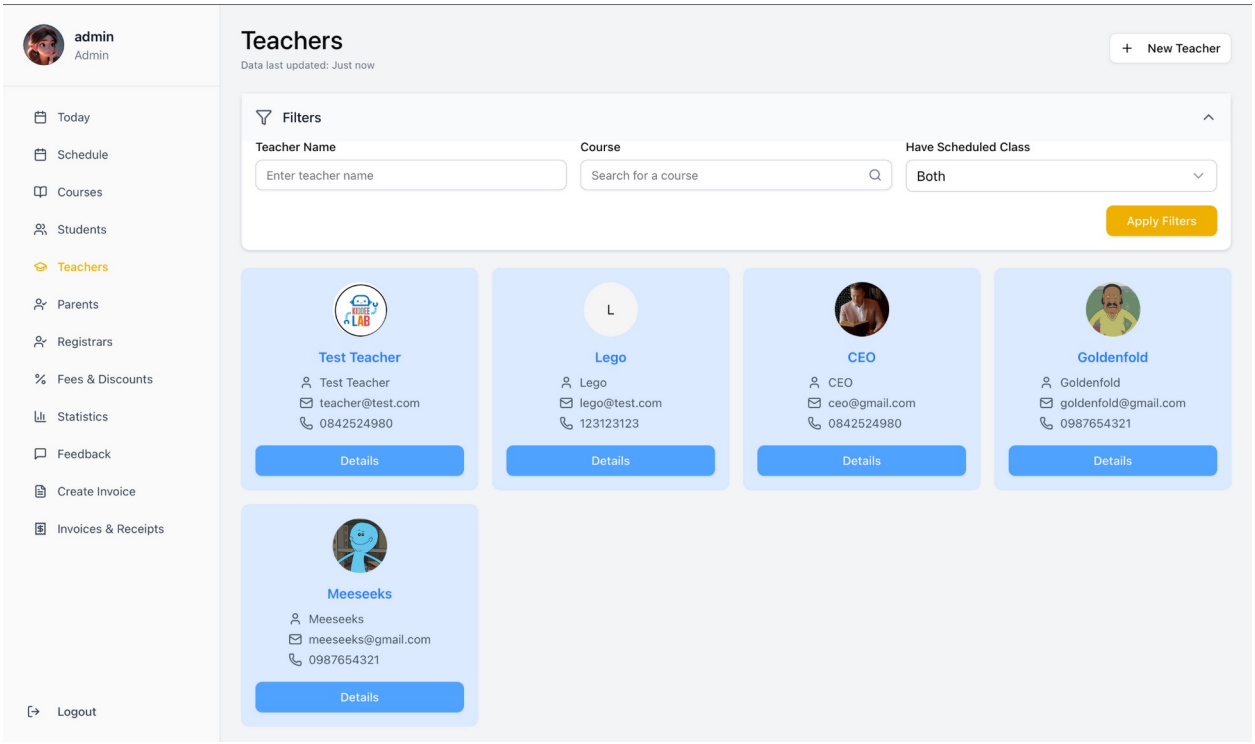


Teachers Page

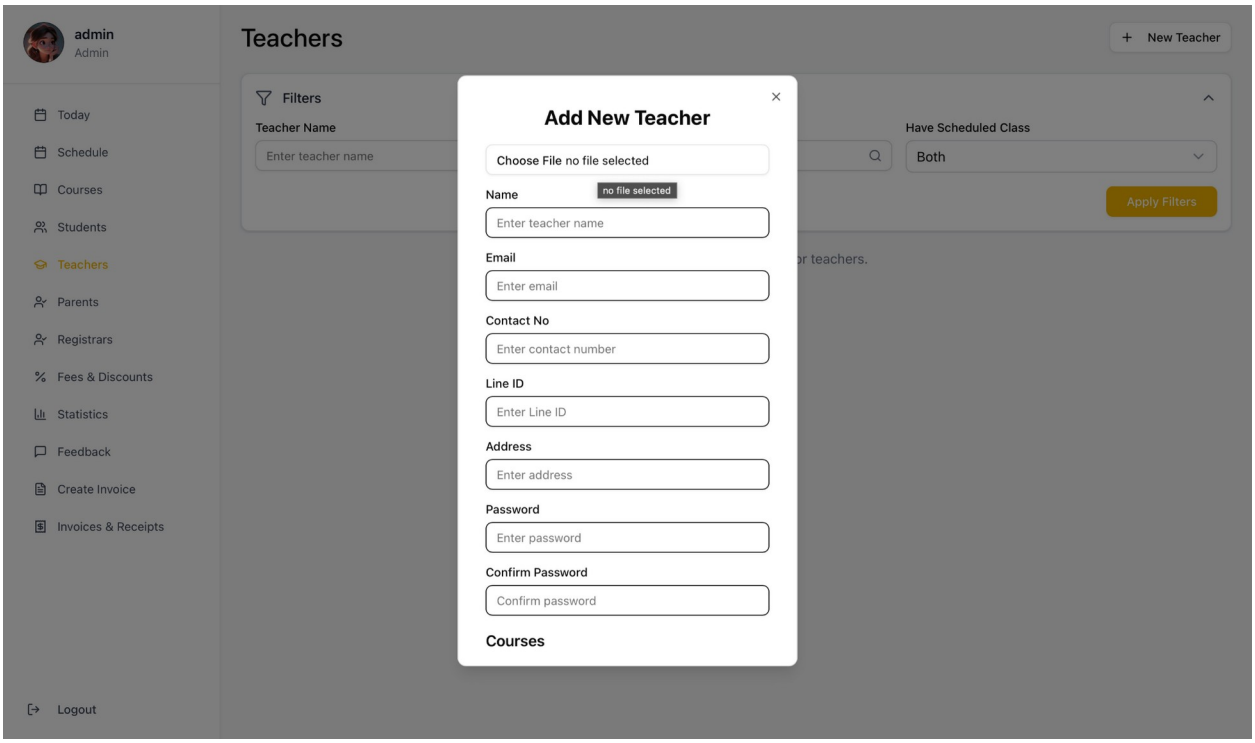
On the **Teachers** page, the manager can view a list of existing teachers. New teachers can be added by entering their details. The email and password provided during the creation process

will serve as the teacher's login credentials. Additionally, courses that the teacher is qualified to teach can be assigned within the system.

Teachers page with a list of teachers

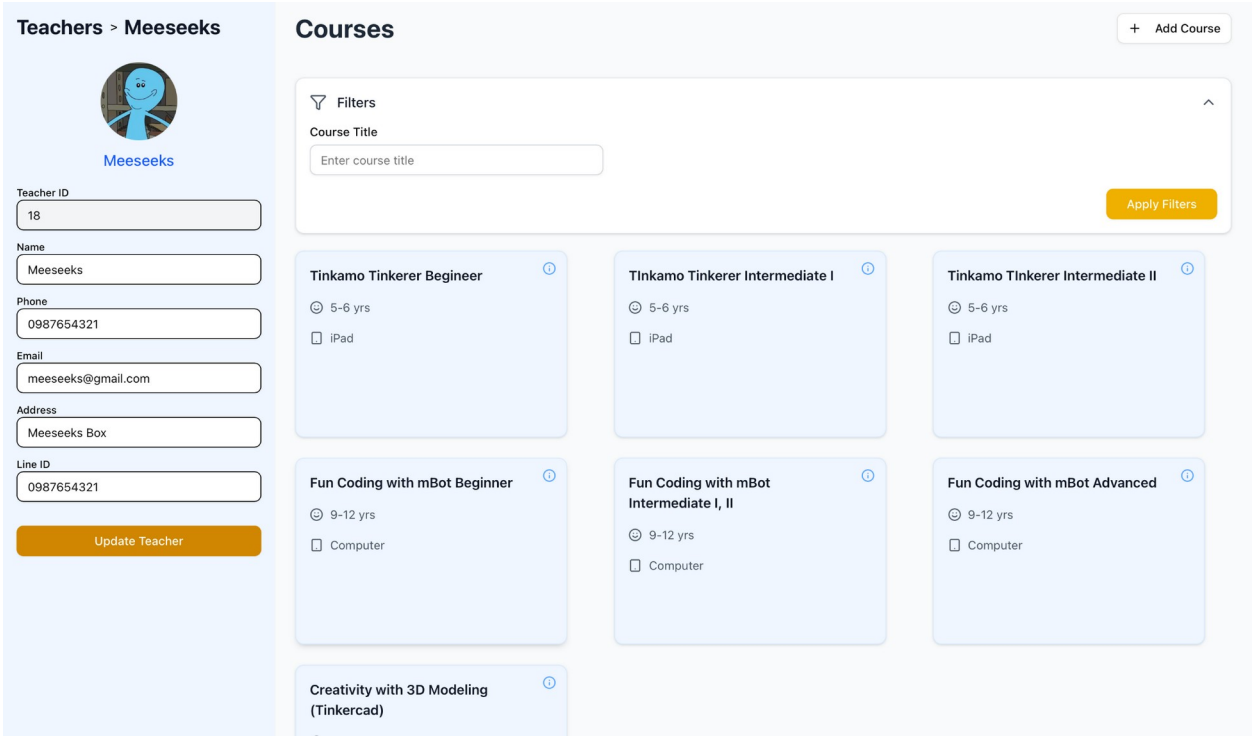


Creating a new teacher

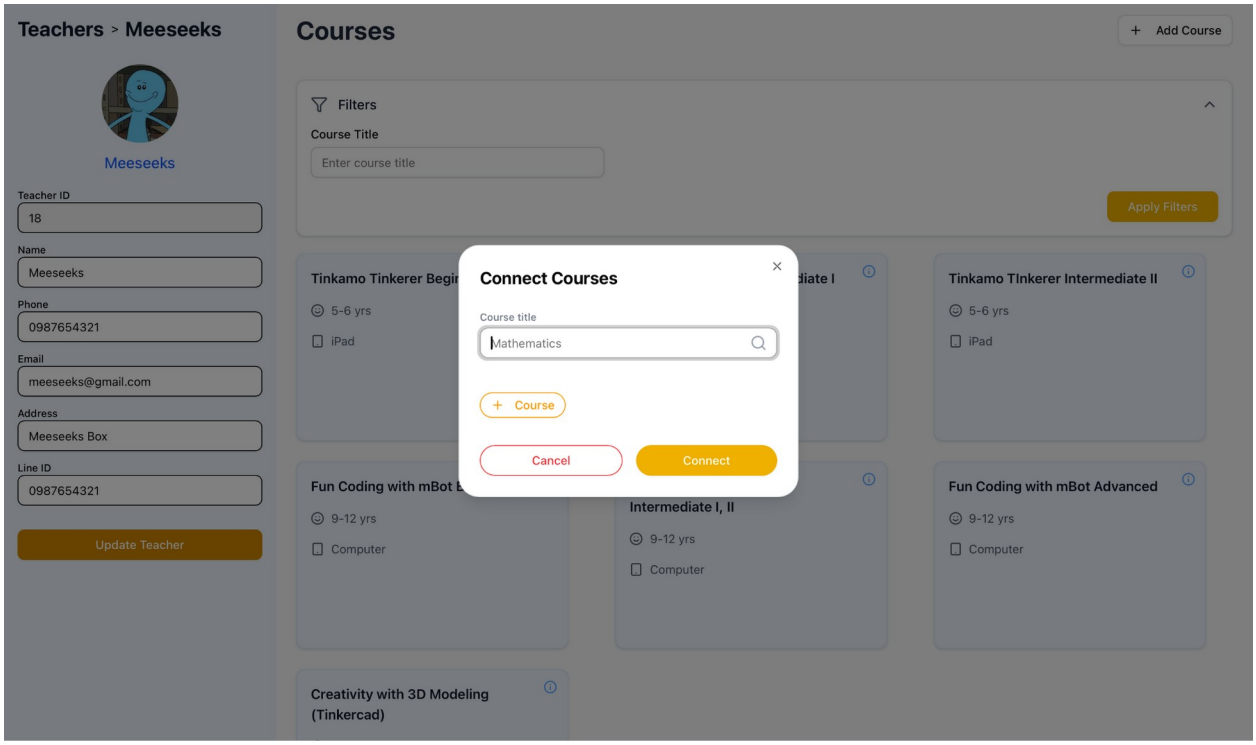


In the **Teacher Detail** view, the user can find the teacher's personal information along with a list of courses they are qualified to teach. By clicking the **Add Course** button, the user can assign additional courses to the teacher.

Teacher Detail Screen



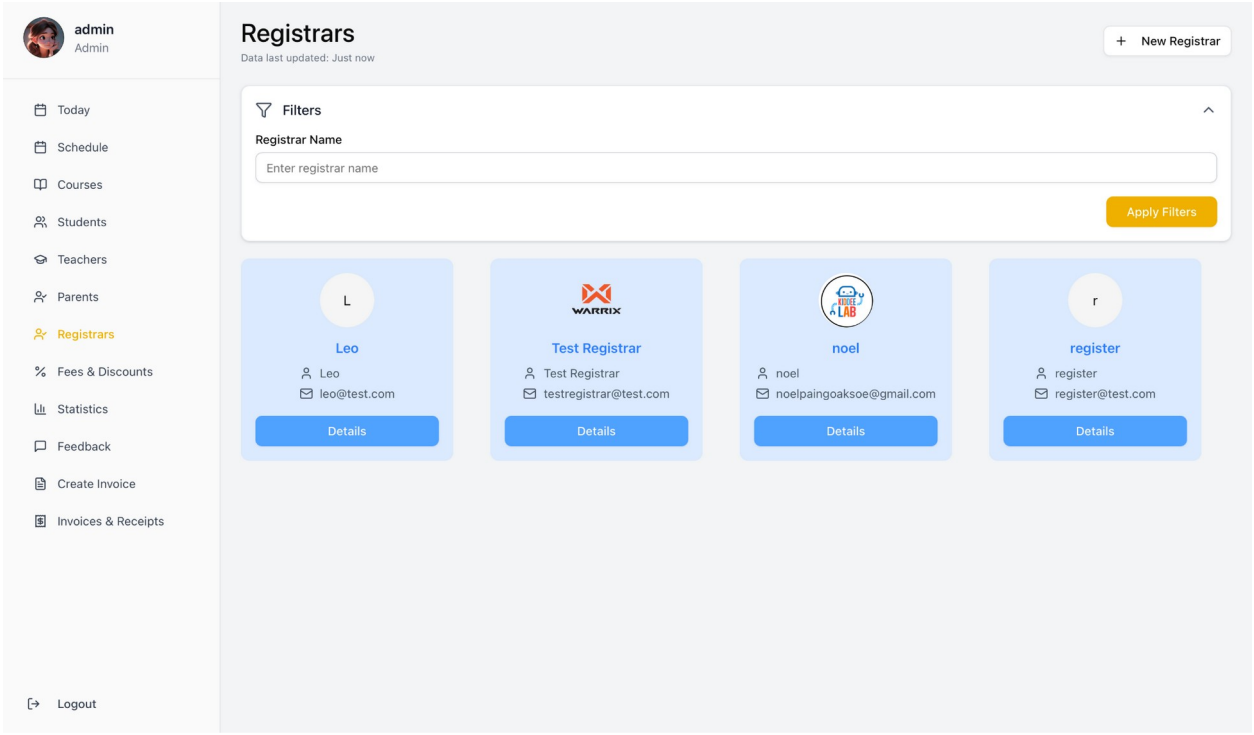
Connecting more course to teacher



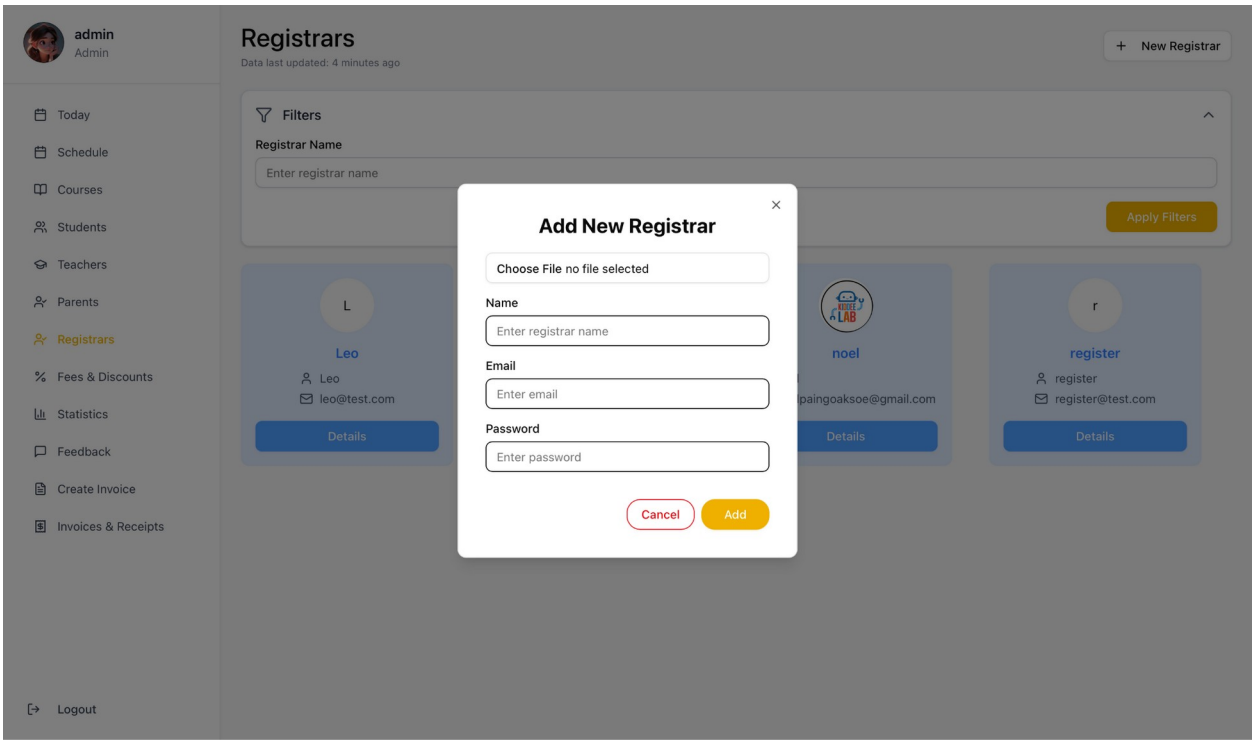
Registrars Page

On the **Registrars** page, the manager can view a list of existing registrars. New registrars can be added by entering their details. The email and password provided during the creation process will be used as their login credentials.

Registrars Page with a list of registrars



Creating a new registrar

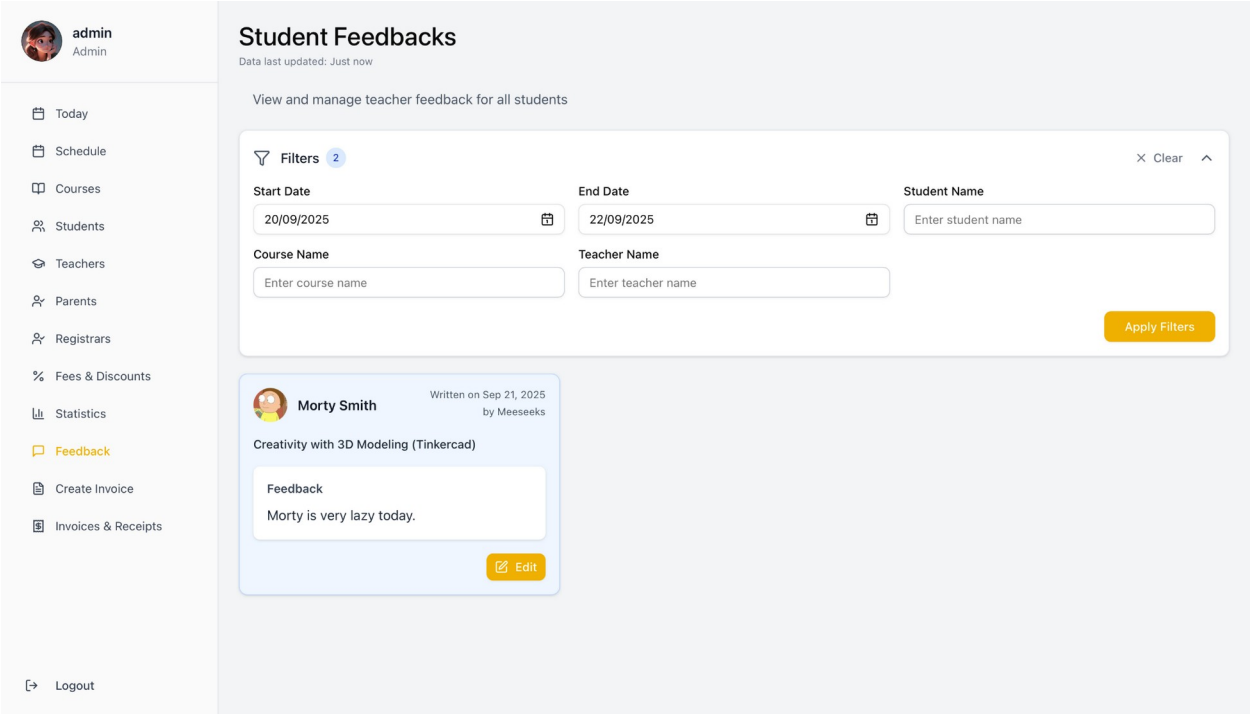


Feedback Page

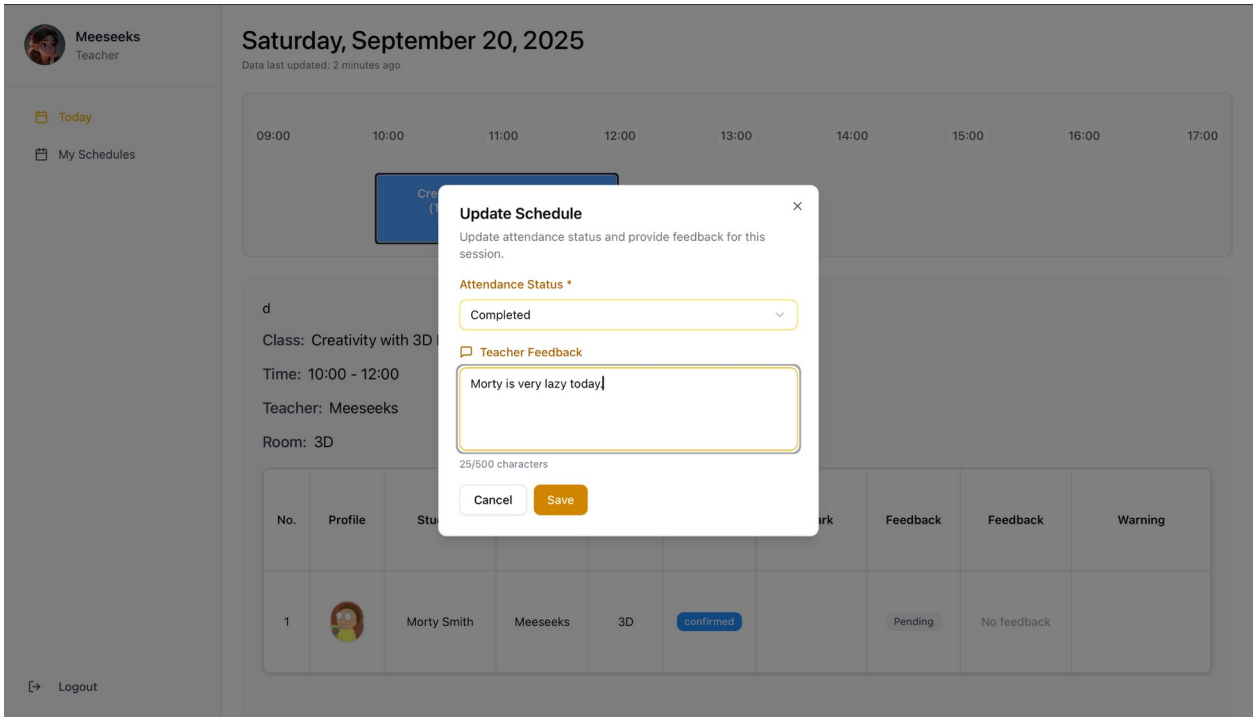
On the **Feedback** page, all feedback submitted by teachers is displayed. Users can modify and/or approve the feedback. Only approved feedback is marked as ready to be shown to parents

and can be accessed on the **Schedules** page or other relevant sections. The system also logs the author of each feedback entry for accountability.

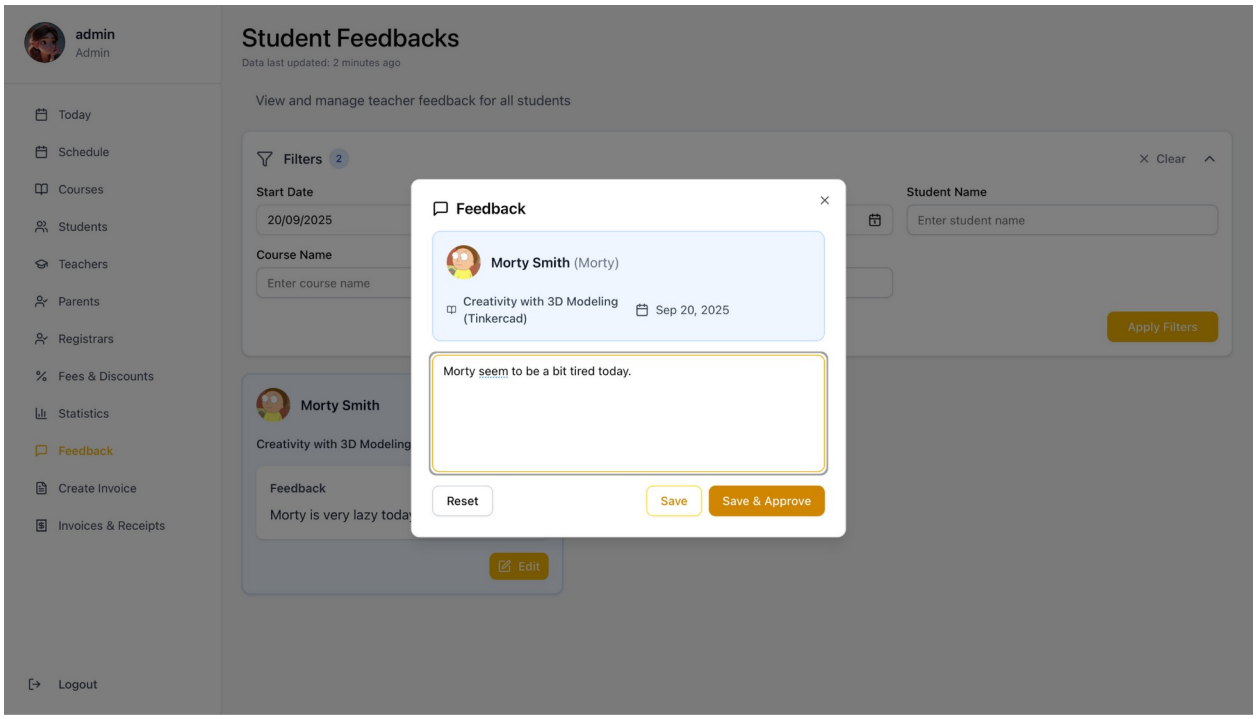
Student feedback page with a list of feedback



Modifying Original Feedback



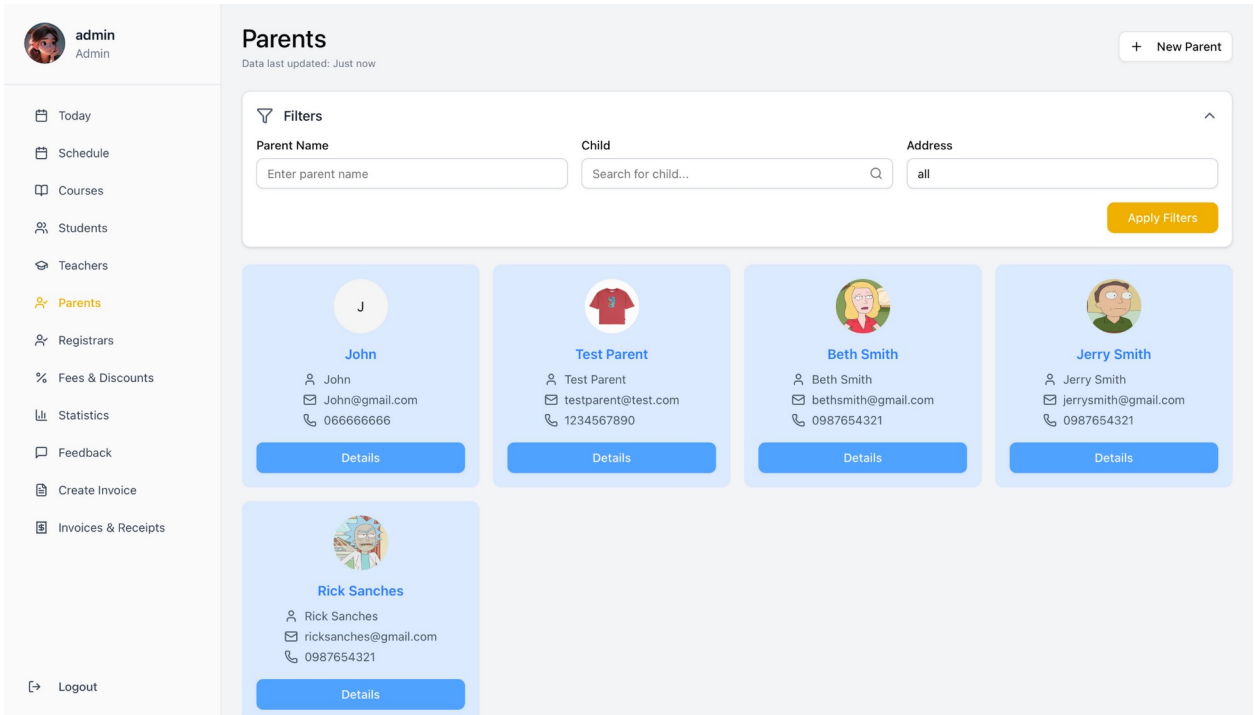
Modified & Approve feedback



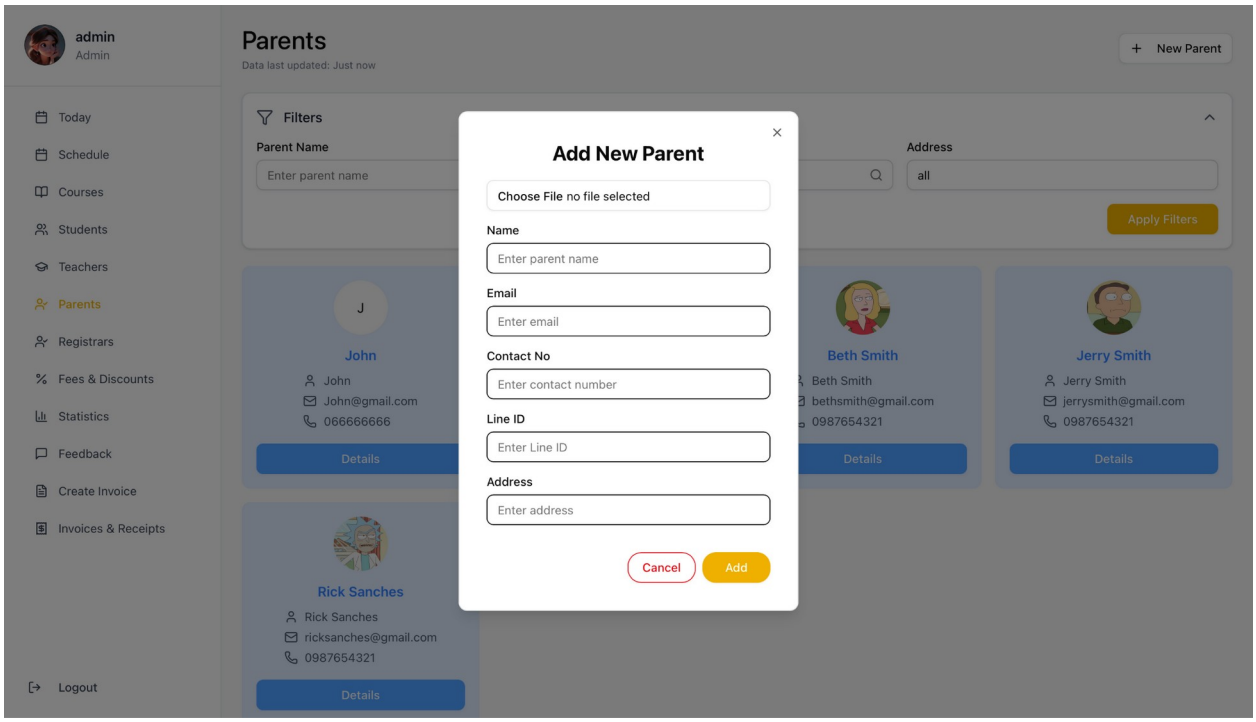
Parents Page

On the **Parents** page, the manager can view a list of existing parents. New parents can be added by entering their details.

Parent Page with a list of parents

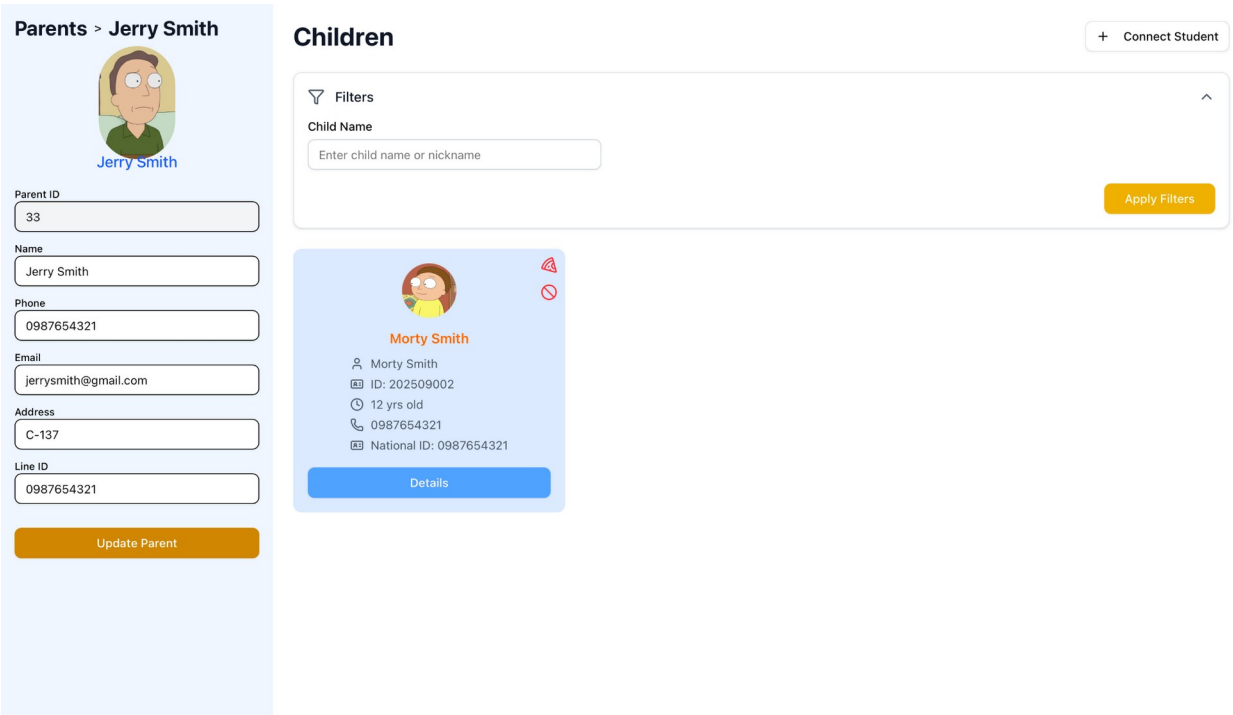


Creating new parent

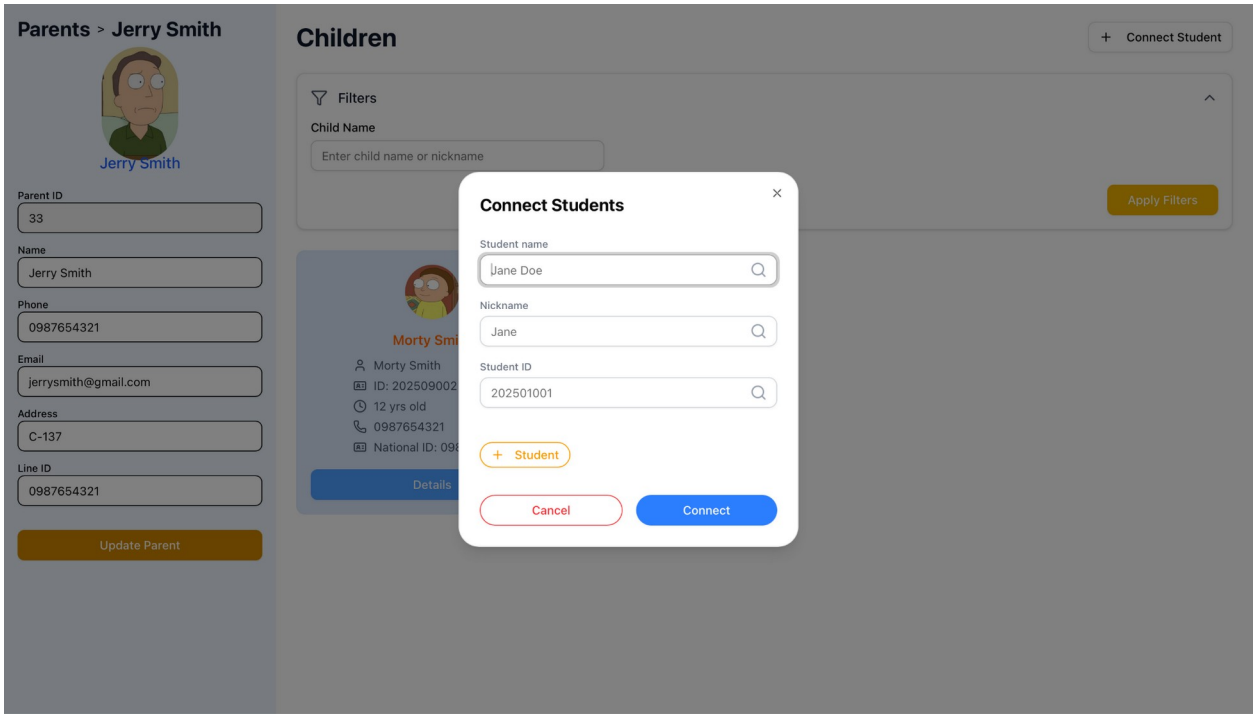


In the **Parent Details** view, the user can find the parent's personal information along with a list of their children. Additional children can be linked by clicking the **Connect Student** button.

Parent details screen



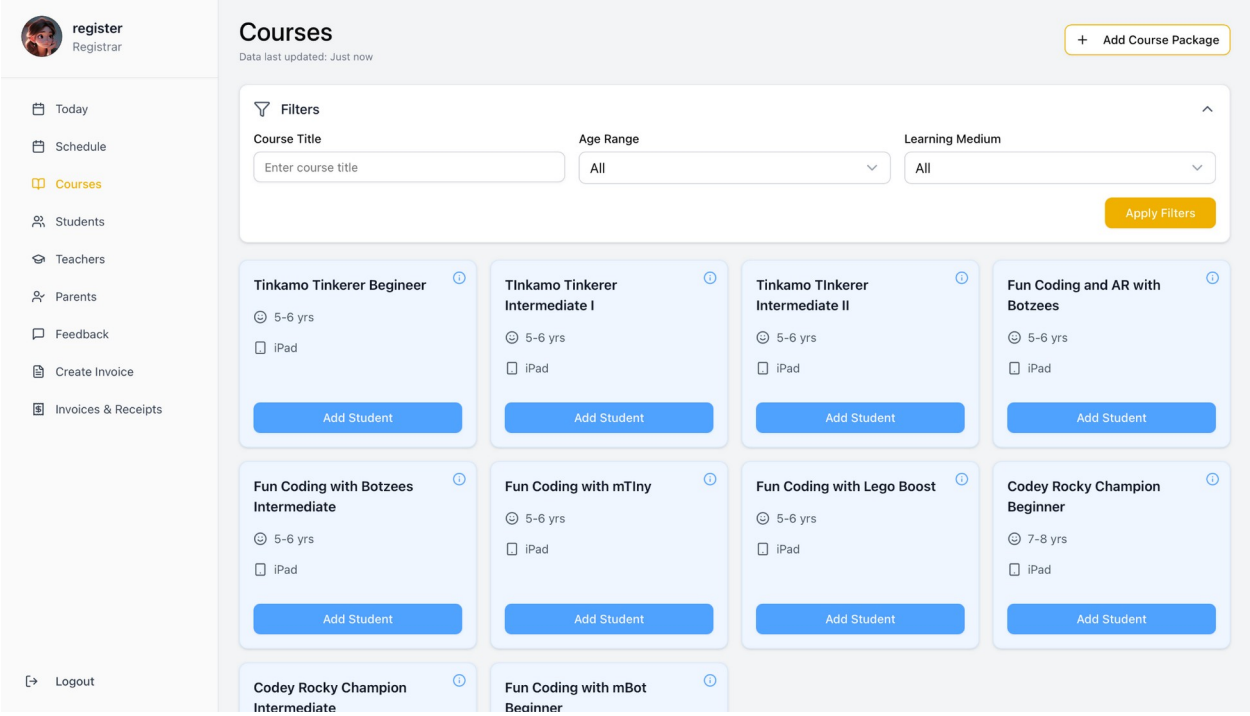
Connecting more students to parent



4.5.5 Registrar UI

Registrars can only access features specified in the Access Control List. Since all permitted features—except the **Courses**page—are the same as those available to managers, the registrar interface largely mirrors the manager UI. However, on the **Courses** page, registrars are not allowed to create new courses; therefore, the **New Course** button is not visible to them.

Courses Page in Registrar POV

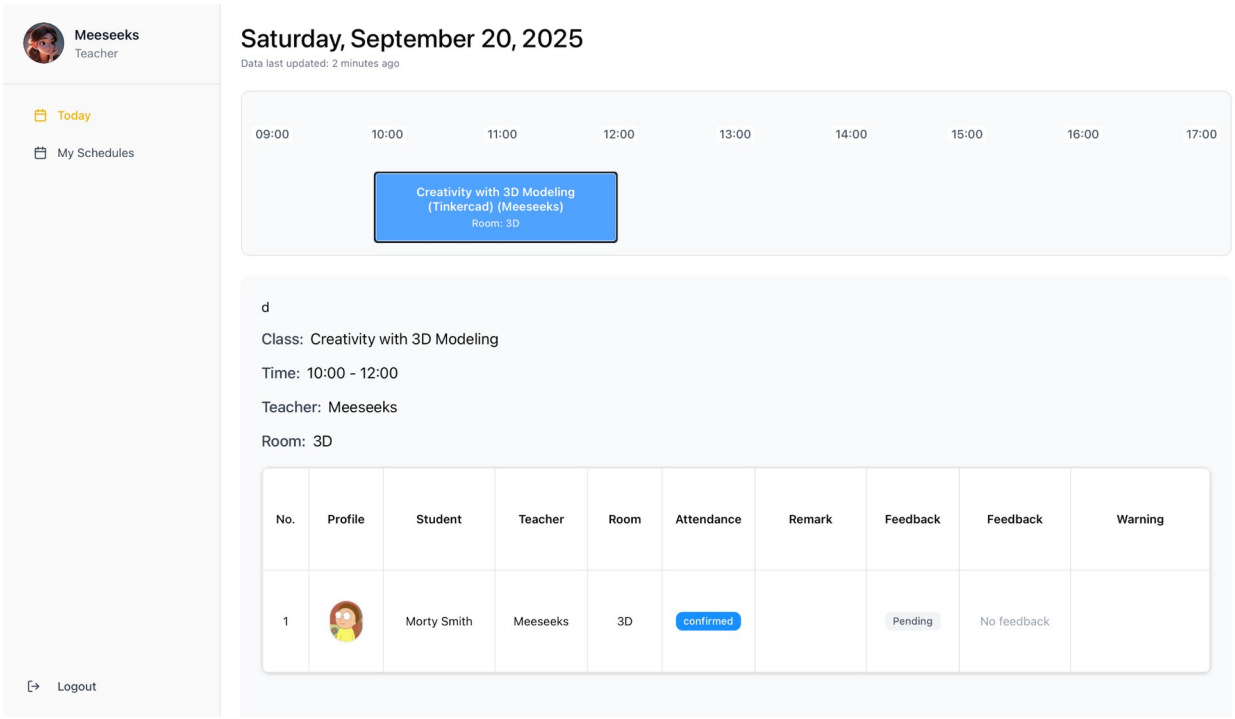


4.5.6 Teacher UI

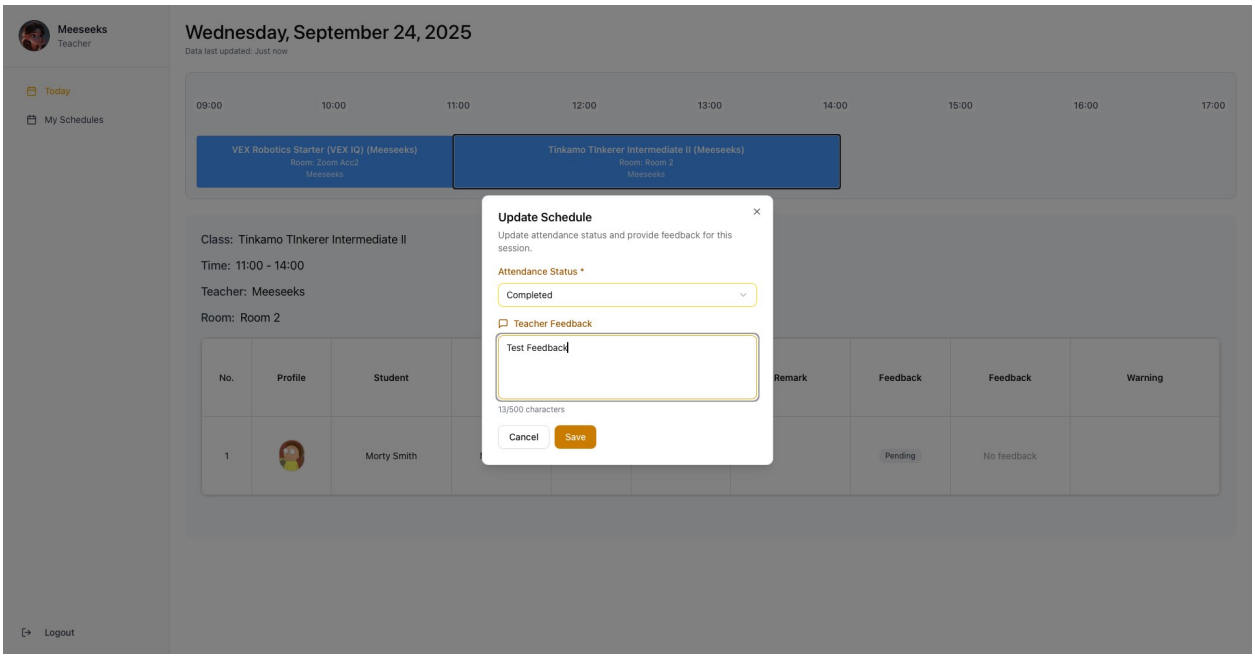
Teachers can only access features specified in the Access Control List.

In the **Today** feature, teachers can view the classes scheduled for the day along with the list of students attending each class. By double-clicking a student's row, teachers can update attendance status and add feedback. Additionally, teachers can access their own schedules, similar to the functionality available in the manager UI.

Web Application in Teacher POV



Check attendance and Log feedback



Chapter 5: Result

5.1 Functionality Achievements

The system is a web-based registration system designed primarily to assist registrars in scheduling classes without conflicts and to enable teachers to record student attendance and feedback efficiently. The scope of the system is defined by the following key features:

- Schedule Conflict Warning: Allows registrars to schedule classes while preventing overlaps in the availability of teachers, rooms, and students.
- Schedule Display: Provides detailed views of student and teacher schedules to facilitate planning and management.
- Feedback Recording: Enables teachers to submit feedback for students, which can subsequently be reviewed and approved by registrars before being finalized.
- Invoice and Receipt Generation: Automates the creation of invoices and receipts following Kiddee Lab's standardized format, streamlining the billing and payment process.

Today Page / Daily Overview

This is the default landing page for managers upon signing in to the system.

The Today feature displays all classes scheduled for the current day. By clicking on a specific class, the manager can view detailed information including the course name, assigned teacher, classroom, time, and the list of students expected to attend. By double-clicking a student's row, the manager can edit the student's details.

Schedules Page

On the Schedules page, managers can use the filtering function to search and narrow down student schedules. Available filters include start date, end date, student name, teacher, course, attendance status, sort order, room, and class type.

By double-clicking a schedule row, the manager can access and edit the associated student's details.After applying the filter, the manager can download the filtered schedule in either PDF or Excel format.

Courses Page

On the Courses page, the manager can view all courses currently offered by KDL. New courses can be created by entering details such as the course title, description, recommended age range, and learning medium. Additionally, the manager can directly add students to a course from this page.

The manager can assign course packages to students. To streamline this process, when searching for a student, the manager can enter the student's name, nickname, or student ID. Once one of these fields is entered, the remaining fields will be automatically populated. After successfully adding a course package, a TBC (To Be Confirmed) course will be assigned to the student. This TBC course allows the course title to be specified at a later time. The purpose of the course package is to enable parents to purchase multiple courses in bulk at a discounted price.

Students Page

On the Students page, the manager can view a list of all students. New students can be added by entering details such as photo, name, nickname, allergies, dietary restrictions, school, national ID, date of birth, gender, phone number, parent information, and permission for the use of photos or videos for advertising purposes. Fields marked with an asterisk (*) are required. If the manager enters invalid or incomplete information in these fields, appropriate error messages will be displayed.

12 Times Fixed

In the 12 Times Fixed class type, the user can select the specific days and times the student will attend. Afterward, the user chooses the teacher and room. The system includes a mechanism that filters and displays only the teachers qualified to teach the selected subject, preventing the selection of an inappropriate teacher.

For example, a student may be scheduled to attend classes on Saturdays and Sundays from 3 PM to 5 PM with Teacher Meeseeks in the room named "Small."

12 Times Check

In the 12 Times Check class type, the user does not need to specify the day, time, teacher, or room in advance. Instead, the student informs KDL of their availability on an ongoing basis, allowing KDL to assign an available teacher and room accordingly.

Camp Class

In the Camp Class type, the user selects the dates and times the student will attend the class. After entering the necessary information, the user can confirm the schedule following the same procedure as in the 12 Times Fixed class type.

Create Invoice Page

After confirming the schedule when adding a course or package to a student, the corresponding course data for that student will be displayed on the Create Invoice page. To generate an invoice, the user must select the rows they wish to include. To prevent the creation of invoices containing multiple students, the system restricts selection to entries belonging to the same student only.

After creating an invoice, the user is directed to the Invoice Generation page, where they can add management fees or discounts and select a payment method. Once the Create Invoice button is clicked, the invoice is generated. The user must then manually verify whether the payment has been received, as the system does not support automated payment processing. Upon successful payment confirmation, a receipt is generated, which can be downloaded and printed. If the payment is unsuccessful, the created invoice remains accessible on the Invoices & Receipts page for further action.

Invoices & Receipts Page

On the Invoices & Receipts page, users can view all receipts as well as outstanding invoices that have not yet been paid.

Fees & Discounts Page

On the Fees & Discounts page, the manager can view a list of all existing fees and discounts. New entries can be created by clicking the Add Discount/Fees button and completing the required information.

Teachers Page

On the Teachers page, the manager can view a list of existing teachers. New teachers can be added by entering their details. The email and password provided during the creation process will serve as the teacher’s login credentials. Additionally, courses that the teacher is qualified to teach can be assigned within the system.

In the Teacher Detail view, the user can find the teacher’s personal information along with a list of courses they are qualified to teach. By clicking the Add Course button, the user can assign additional courses to the teacher.

Registrars Page

On the Registrars page, the manager can view a list of existing registrars. New registrars can be added by entering their details. The email and password provided during the creation process will be used as their login credentials.

Feedback Page

On the Feedback page, all feedback submitted by teachers is displayed. Users can modify and/or approve the feedback. Only approved feedback is marked as ready to be shown to parents and can be accessed on the Schedules page or other relevant sections. The system also logs the author of each feedback entry for accountability.

Parents Page

On the Parents page, the manager can view a list of existing parents. New parents can be added by entering their details.

In the Parent Details view, the user can find the parent's personal information along with a list of their children. Additional children can be linked by clicking the Connect Student button.

5.2 Requirement Gathering & Testing

Table 14 Requirement Gathering and Testing Table

Date	Activity
11 April 2025	Online interview with clients to talk about their problems, goals and expectations.
21 April 2025	Follow-up interview with clients to confirm the project scope.
26 April 2025	On-site observation and interviews with the manager, registrar and teachers.
5 May 2025	Hands-on workshop to explore user interaction and gather additional feedback.
9 May 2025	Conducted User Acceptance Testing (UAT) with the system prototype.
20 May 2025	Made adjustments to the prototype and confirmed alignment with stakeholder needs.

5.3 User Acceptance Testing & Adjustments

User Acceptance Testing (UAT) with the real system is tested on 8 Sep 2025 Conducted at Kiddee Lab Company. The UAT activity and subsequent adjustments indicate iterative validation of the prototype with stakeholders; specific findings and bug lists are recorded in the project notes (not reproduced here).

5.4 Limitations

The system is limited to the functionalities listed above and does not currently support other administrative tasks such as payroll, detailed academic performance tracking, direct online payment processing, parental access to view their child’s information or performance, class cancellation requests, or viewing student timetables. Additionally, the project scope does not include a data migration plan; digitizing and importing existing paper-based student records or schedules from Excel files is outside the current implementation.

Chapter 6: Conclusion

The Kiddee Lab Registration System was developed to address the operational inefficiencies caused by manual scheduling, registration, and payment processes. Prior to this project, all administrative tasks were performed manually using spreadsheets and paper-based documents, resulting in duplicated work, inconsistent records, and difficulty in managing class schedules.

Through the implementation of this system, the registration process is now centralized, reducing human error and improving the accuracy of records. The system provides an organized platform where managers, registrars, and teachers can access the same data in real time. Features such as schedule conflict detection, automated invoice generation, and student information management have streamlined daily operations and significantly reduced the workload of administrative staff.

The design of the system focuses on usability and simplicity, ensuring that even non-technical users can easily navigate and perform their tasks. Its modular architecture, built using Node.js, React, and MySQL, allows for scalability and future enhancement. The ability to manage students, teachers, schedules, and payments within one system demonstrates how technology can effectively transform small-scale educational operations into efficient, data-driven environments.

In summary, the system successfully meets the main objectives established during the requirement analysis phase — it automates repetitive administrative tasks, improves information accessibility, and provides a reliable platform for internal management at Kiddee Lab.

6.1 Future Work

Although the Kiddee Lab Registration System currently meets the school’s main administrative needs, there are plans to further improve the system in the future. The main focus will be on integrating the system with the LINE application, which is widely used by parents in Thailand. By doing this:

- **Parent Access via LINE:** Parents will be able to access the system directly within the LINE app, eliminating the need to install a separate application. They can check their child's schedule, class details, and other relevant information conveniently.
- **Automated Notifications:** The system will send notifications to parents through LINE to inform them whether their children can attend upcoming classes. Notifications will be sent three times in advance of every class to ensure timely communication.

These improvements aim to make the system more convenient for parents, improve communication, and enhance overall usability.

Overall, this project demonstrates how a small educational institution can leverage web technologies to create customized solutions that reduce administrative complexity, enhance productivity, and prepare for sustainable digital transformation in the future.

References

Company Website : https://www.kiddeelab.co.th/
Company Materials: https://www.figma.com/board/TWr17JCSKN1GDBXpRry3va/Kiddee-Lab-Application?node-id=0-1&t=YyZufCT1HBtaze3L-1